

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل سیزدهم: Text Classification and Naive Bayes

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

13.1 The text classification problem

13.2 Naive Bayes text classification

13.2.1 Relation to multinomial unigram language model

13.3 The Bernoulli model

13.4 Properties of Naive Bayes

13.4.1 A variant of the multinomial model

13.5 Feature selection

13.5.1 Mutual information

13.5.2 χ^2 Feature selection

13.5.3 Frequency-based feature selection

13.5.4 Feature selection for multiple classifiers

13.5.5 Comparison of feature selection methods

13.6 Evaluation of text classification

13.7 Chapter 13 Summary

13 طبقه‌بندی متن و بیز ساده: از جستجوی گوگل تا فیلتر اسپم

از جستجوی یک‌باره تا پیگیری هوشمند

تاکنون، این جزوه عمدتاً فرآیند بازیابی ادهاک (ad hoc retrieval) را بررسی کرده است - جایی که کاربران نیازهای اطلاعاتی موقتی دارند که با ارسال پرسمان به موتور جستجو تلاش می‌کنند به آن‌ها پاسخ دهند. اما بسیاری از کاربران نیازهای اطلاعاتی مداوم دارند.

تصور کنید بخواهید تحولات در حوزه تراشه‌های کامپیوتری چند هسته‌ای را دنبال کنید. یک راه این است که هر روز صبح پرسمان multicore AND computer AND chip را در شاخص مقالات خبری جدید جستجو کنید. اما این کار بسیار زمان‌بر است! گوگل نیوز چگونه این کار را برای شما خودکار می‌کند؟

در این فصل و دو فصل بعدی، به این سؤال می‌پردازیم: چگونه می‌توان این کار تکراری را خودکارسازی کرد؟ بسیاری از سیستم‌ها از پرسمان‌های ثابت (standing queries) پشتیبانی می‌کنند - پرسمان‌هایی که به صورت دوره‌ای روی مجموعه‌ای اجرا می‌شوند که اسناد جدید به تدریج به آن اضافه می‌شوند.

چرا ایمیل‌های اسپم به پوشه اسپم می‌روند؟

یک چالش مهم با پرسمان‌های ثابت این است که اگر پرسمان شما فقط multicore AND computer AND chip باشد، بسیاری از مقالات مرتبط را که از اصطلاحات دیگری مانند multicore processors استفاده می‌کنند، از دست خواهید داد. برای دستیابی به بازخوانی (recall) خوب، پرسمان‌های ثابت باید در طول زمان اصلاح شوند و می‌توانند به تدریج بسیار پیچیده شوند:

(multicore OR multi-core) AND (chip OR processor OR microprocessor)

برای درک گستردگی این مسئله، اکنون مفهوم کلی **طبقه‌بندی** (classification) را معرفی می‌کنیم. با توجه به مجموعه‌ای از کلاس‌ها، ما به دنبال تعیین این هستیم که یک شیء معین به کدام کلاس(ها) تعلق دارد. در مثال بالا، پرسمان ثابت برای تقسیم مقالات خبری جدید به دو کلاس عمل می‌کند: اسناد مربوط به تراشه‌های کامپیوتری چند هسته‌ای و اسناد غیرمرتبط با تراشه‌های کامپیوتری چند هسته‌ای. ما به این **طبقه‌بندی دو کلاسی** می‌گوییم. طبقه‌بندی با استفاده از پرسمان‌های ثابت همچنین **مسیریابی** (routing) یا **فیلتر کردن** (filtering) نامیده می‌شود.

یک کلاس لزوماً نباید به اندازه پرسمان ثابت multicore computer chips محدود باشد. اغلب، یک کلاس یک حوزه موضوعی عمومی‌تر مانند چین یا قهوه است. چنین کلاس‌های عمومی‌تری معمولاً **موضوعات** (topics) نامیده می‌شوند و وظیفه طبقه‌بندی متن سپس **طبقه‌بندی متن** (text classification)، **دسته‌بندی متن** (text categorization)، **طبقه‌بندی موضوع** (topic classification) یا **شناسایی موضوع** (topic spotting) نامیده می‌شود.

کاربردهای واقعی طبقه‌بندی متن در دنیای دیجیتال

مفهوم طبقه‌بندی بسیار کلی است و کاربردهای زیادی در داخل و خارج از بازیابی اطلاعات (IR) دارد. ما در اینجا بر مثال‌هایی از بازیابی اطلاعات تمرکز می‌کنیم:

کاربرد	توضیح
پیش‌پردازش برای نمایه‌سازی	گوگل از طبقه‌بندی برای تشخیص زبان صفحات وب استفاده می‌کند تا نتایج جستجو را به زبان مناسب نمایش دهد
تشخیص هرزنامه	جیمیل از طبقه‌بندهای پیچیده برای فیلتر کردن میلیاردها ایمیل اسپم روزانه استفاده می‌کند
تشخیص محتوای صریح جنسی	یوتیوب از طبقه‌بندی برای شناسایی محتوای نامناسب استفاده می‌کند تا با فعال بودن SafeSearch نمایش داده نشود
تشخیص احساسات	آمازون از تحلیل احساسات برای خلاصه کردن نظرات کاربران روی محصولات استفاده می‌کند
مرتب‌سازی ایمیل شخصی	اوتلوک از طبقه‌بندی برای مرتب‌سازی خودکار ایمیل‌ها به پوشه‌های اولویت، تبلیغات و اجتماعی استفاده می‌کند
جستجوی عمودی	وبسایت‌هایی مانند Zillow (املاک) یا Kayak (سفر) از طبقه‌بندی برای محدود کردن جستجو به حوزه تخصصی خود استفاده می‌کنند

این فهرست اهمیت کلی طبقه‌بندی در IR را نشان می‌دهد. اکثر سیستم‌های بازیابی امروز حاوی چندین جزء هستند که به شکل طبقه‌بندی استفاده می‌کنند.

📖 سه رویکرد برای طبقه‌بندی متن

برای پیاده‌سازی طبقه‌بندی متن، سه رویکرد اصلی وجود دارد:

- طبقه‌بندی دستی:** کتاب‌ها در یک کتابخانه توسط کتابدار به دسته‌بندی‌های کتابخانه کنگره اختصاص داده می‌شوند. این روش برای مقیاس‌بندی گران است.
- قوانین دستی:** استفاده از پرسمان‌های ثابت که می‌توان به عنوان **قوانین** در نظر گرفت - که معمولاً به صورت دستی نوشته می‌شوند. مانند مثال ما (multicore OR multi-core) AND (chip OR processor OR microprocessor)، قوانین گاهی معادل عبارات بولی هستند. یک قانون ترکیبی خاصی از کلمات کلیدی را که نشان‌دهنده یک کلاس است، ثبت می‌کند. قوانین کدگذاری شده دستی خواص مقیاس‌بندی خوبی دارند، اما ایجاد و نگهداری آن‌ها در طول زمان کار فشرده‌ای است.
- یادگیری ماشین:** در این رویکرد، مجموعه‌ای از قوانین یا به طور کلی، معیار تصمیم‌گیری طبقه‌بند متن، به طور خودکار از داده‌های آموزشی یاد گرفته می‌شود. این رویکرد همچنین **طبقه‌بندی آماری متن** نامیده می‌شود اگر روش یادگیری آماری باشد.

در طبقه‌بندی آماری متن، ما به تعدادی اسناد مثال خوب (یا اسناد آموزشی) برای هر کلاس نیاز داریم. نیاز به طبقه‌بندی دستی حذف نمی‌شود زیرا اسناد آموزشی از شخصی که آن‌ها را برچسب‌گذاری کرده است، می‌آیند - جایی که برچسب‌گذاری به فرآیند حاشیه‌نویسی هر سند با کلاس آن اشاره دارد. اما برچسب‌گذاری به طور قابل استدلالی وظیفه آسان‌تری از نوشتن قوانین است. تقریباً هر کسی می‌تواند به یک سند نگاه کند و تصمیم بگیرد که آیا به چین مرتبط است یا نه.

در این فصل، ما با یک مقدمه کلی برای مسئله طبقه‌بندی متن آغاز می‌کنیم؛ سپس بیز ساده (Naive Bayes)، یک روش طبقه‌بندی به خصوص ساده و مؤثر را پوشش می‌دهیم. تمام الگوریتم‌های طبقه‌بندی که ما مطالعه می‌کنیم، اسناد را در فضاها با ابعاد بالا نمایش می‌دهند. برای بهبود کارایی این الگوریتم‌ها، معمولاً مطلوب است که ابعاد این فضاها را کاهش دهیم؛ به این منظور، تکنیکی به نام **انتخاب ویژگی** (feature selection) معمولاً در طبقه‌بندی متن اعمال می‌شود.

در فصل‌های بعدی، به دو خانواده دیگر از روش‌های طبقه بندی، طبقه بندهای فضای برداری و ماشین‌های بردار پشتیبان نگاه خواهیم کرد.

13.1 مسئله طبقه بندی متن: از ایمیل‌های اسپم تا دسته بندی اخبار

📌 چگونه جیمیل تشخیص می‌دهد ایمیل اسپم است؟

در طبقه بندی متن، ما یک توصیف $d \in X$ از یک سند داریم، که در آن X فضای سند (document space) است؛ و مجموعه‌ای ثابت از کلاس‌ها $C = \{c_1, c_2, \dots, c_L\}$. کلاس‌ها همچنین به عنوان دسته‌ها (categories) یا برچسب‌ها (labels) شناخته می‌شوند.

به طور معمول، فضای سند X نوعی فضای با ابعاد بالا است، و کلاس‌ها توسط انسان برای نیازهای یک کاربرد تعریف می‌شوند. برای مثال، در سیستم جیمیل، هر ایمیل یک سند است و دو کلاس اصلی وجود دارد: "اسپم" و "غیراسپم". جیمیل باید بتواند هر ایمیل جدید را به یکی از این دو کلاس اختصاص دهد.

ما یک مجموعه آموزشی (training set) D از اسناد برچسب‌دار $\langle d, c \rangle$ داریم که در آن $\langle d, c \rangle \in X \times C$. برای مثال:

$\langle \text{Beijing joins the World Trade Organization, China} \rangle$

برای سند تک جمله‌ای "Beijing joins the World Trade Organization" و کلاس (یا برچسب) China.

💡 یادگیری نظارت شده: چگونه نتفلیکس سلیقه شما را یاد می‌گیرد؟

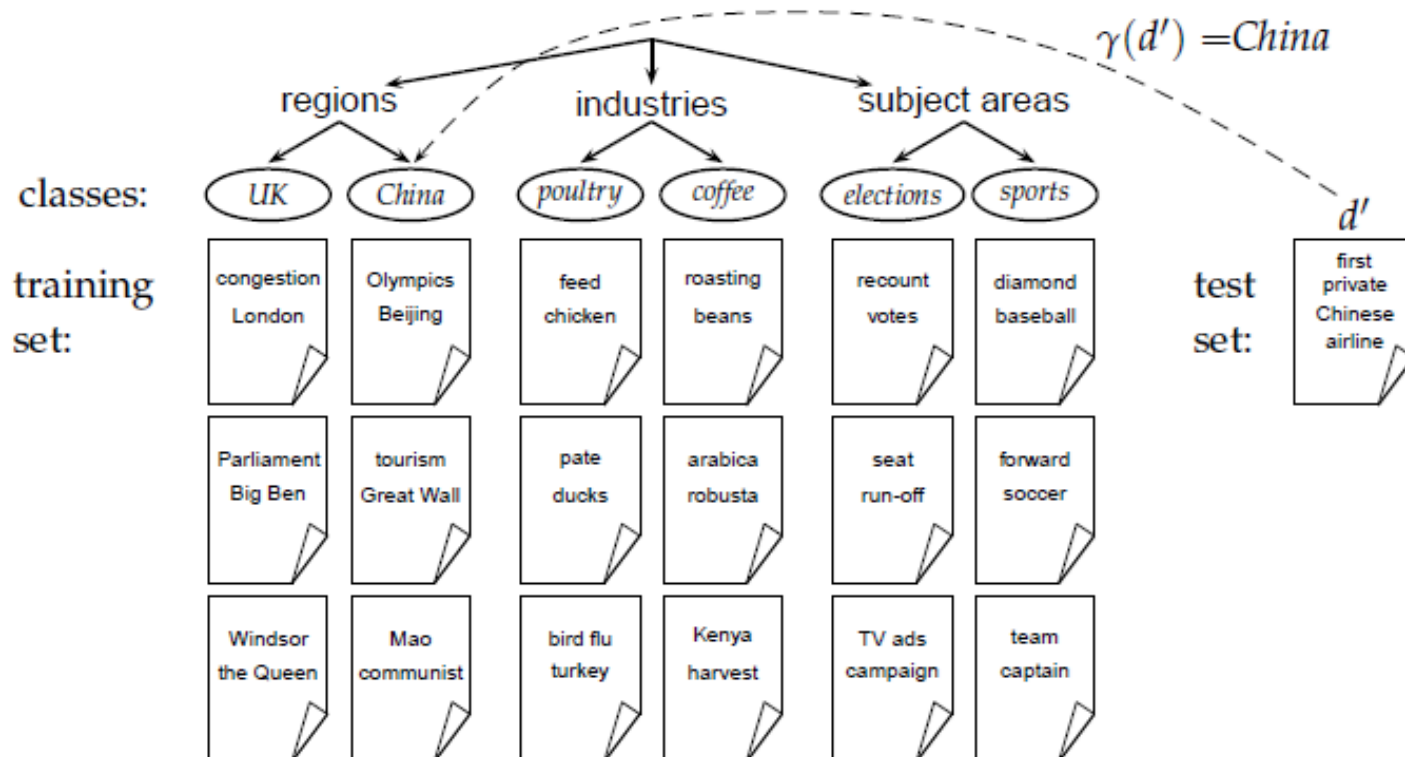
با استفاده از یک روش یادگیری (learning method) یا الگوریتم یادگیری (learning algorithm)، ما سپس می‌خواهیم یک طبقه بند (classifier) یا تابع طبقه بندی (classification function) γ یاد بگیریم که اسناد را به کلاس‌ها نگاشت می‌کند:

$$\gamma : X \rightarrow C$$

این نوع یادگیری یادگیری نظارت شده (supervised learning) نامیده می‌شود زیرا یک ناظر (انسانی که کلاس‌ها را تعریف می‌کند و اسناد آموزشی را برچسب‌گذاری می‌کند) به عنوان یک معلم فرآیند یادگیری را هدایت می‌کند.

نتفلیکس از همین روش استفاده می‌کند: وقتی شما به فیلمی امتیاز می‌دهید، در واقع در حال برچسب‌گذاری داده‌های آموزشی هستید. الگوریتم نتفلیکس (روش یادگیری Γ) از این داده‌ها برای یادگیری تابع طبقه بندی γ استفاده می‌کند تا بتواند پیش‌بینی کند کدام فیلم‌های دیگر را دوست خواهید داشت.

ما روش یادگیری نظارت‌شده را با Γ نشان می‌دهیم و می‌نویسیم $\gamma = \Gamma(D)$. روش یادگیری Γ مجموعه آموزشی D را به عنوان ورودی می‌گیرد و تابع طبقه بندی یادگرفته شده γ را برمی‌گرداند.



► Figure 13.1 Classes, training set, and test set in text classification .

طبقه بندی اخبار: از آموزش تا آزمون

شکل 13.1 یک مثال از طبقه بندی متن از مجموعه Reuters-RCV1 را نشان می‌دهد. شش کلاس (UK, China, ..., sports) وجود دارد، هر کدام با سه سند آموزشی. ما چند کلمه یادآوری برای محتوای هر سند نشان می‌دهیم. مجموعه آموزشی برخی مثال‌های معمولی برای هر کلاس فراهم می‌کند، به طوری که ما می‌توانیم تابع طبقه بندی γ را یاد بگیریم.

پس از یادگیری γ ، ما می‌توانیم آن را به **مجموعه آزمون** (test set) (یا داده‌های آزمون) اعمال کنیم، برای مثال، سند جدید "first private Chinese airline" که کلاس آن ناشناخته است.

در شکل 13.1، تابع طبقه بندی سند جدید را به کلاس $\gamma(d) = \text{China}$ اختصاص می‌دهد، که تخصیص صحیح است. این دقیقاً همان کاری است که خبرگزاری‌ها مانند گوگل نیوز با مقالات جدید انجام می‌دهند: آن‌ها با استفاده از مدل‌های آموزش‌دیده، مقالات جدید را به دسته‌بندی‌های مناسب اختصاص می‌دهند.

سلسله مراتب کلاس‌ها: از یک کلاس تا چندین کلاس

کلاس‌ها در طبقه‌بندی متن اغلب دارای ساختار جالبی مانند سلسله مراتب در شکل 13.1 هستند. دو نمونه از هر یک از دسته‌بندی‌های منطقه، صنعت و حوزه موضوعی وجود دارد. یک سلسله مراتب می‌تواند یک کمک مهم در حل مسئله طبقه‌بندی باشد.

برای مثال، در وب‌سایت آمازون، یک محصول می‌تواند هم در دسته "الکترونیک" و هم در دسته "لوازم خانگی" قرار گیرد. این سلسله مراتب به کاربران کمک می‌کند تا محصولات را بهتر پیدا کنند.

تعریف (13.1) مقرر می‌دارد که یک سند عضو دقیقاً یک کلاس است. این مناسب‌ترین مدل برای سلسله مراتب در شکل 13.1 نیست. برای مثال، سندی در مورد المپیک 2008 باید عضو دو کلاس باشد: کلاس China و کلاس sports. این نوع مسئله طبقه بندی به عنوان یک مسئله **any-of** نامیده می‌شود و ما به آن در بخش 14.5 بازمی‌گردیم. در حال حاضر، ما فقط مسائل **one-of** را در نظر می‌گیریم که در آن یک سند عضو دقیقاً یک کلاس است.

یوتیوب از این مدل استفاده می‌کند: یک ویدیو می‌تواند هم به دسته "موسیقی" و هم به دسته "آموزشی" تعلق داشته باشد. این یک مسئله any-of است.

هدف ما در طبقه‌بندی متن، دقت بالا در داده‌های آزمون یا داده‌های جدید است - برای مثال، مقالات خبری که فردا صبح با آن‌ها مواجه خواهیم شد. دستیابی به دقت بالا در مجموعه آموزشی آسان است (مثلاً، ما می‌توانیم به سادگی برچسب‌ها را حفظ کنیم). اما دقت بالا در مجموعه آموزشی به طور کلی به این معنی نیست که طبقه‌بند در داده‌های جدید در یک کاربرد به خوبی کار خواهد کرد.

وقتی ما از مجموعه آموزشی برای یادگیری یک طبقه‌بند برای داده‌های آزمون استفاده می‌کنیم، ما این فرض را می‌کنیم که داده‌های آموزشی و داده‌های آزمون شبیه هستند یا از همان توزیع هستند. این همان چیزی است که در دنیای واقعی اتفاق می‌افتد: الگوریتم‌های طبقه‌بندی فرض می‌کنند ایمیل‌های فردا شبیه به ایمیل‌های امروز خواهند بود.

13.2 طبقه بندی متن با بیز ساده: چگونه جیمیل ایمیل‌های اسپم را تشخیص می‌دهد؟

ایده اصلی: احتمالات در برابر کلمات

مدل **چندجمله‌ای بیز ساده** (multinomial Naive Bayes) یک روش یادگیری احتمالی است که احتمال اینکه یک سند d در کلاس c باشد را محاسبه می‌کند:

$$P(c|d) \propto P(c) \prod_{1 \leq k \leq n_d} P(t_k|c)$$

بیایید این را با یک مثال واقعی بررسی کنیم: وقتی جیمیل ایمیل شما را بررسی می‌کند، در واقع در حال محاسبه احتمال اینکه ایمیل "اسپم" است یا "غیراسپم" است. $P(c)$ احتمال اولیه اینکه یک ایمیل به طور کلی اسپم باشد (مثلاً 30٪ ایمیل‌ها اسپم هستند). $P(t_k|c)$ احتمال اینکه کلمه‌ای مثل "ارزان" یا "کاهش وزن" در ایمیل‌های اسپم ظاهر شود.

🏆 بهترین کلاس: بیشترین احتمال پسین

بهترین کلاس در طبقه بندی NB، کلاس **بیشترین احتمال پسین** (maximum a posteriori یا MAP) است:

$$c_{\text{map}} = \arg \max_{c \in C} \hat{P}(c|d) = \arg \max_{c \in C} \hat{P}(c) \prod_{1 \leq k \leq n_d} \hat{P}(t_k|c)$$

در عمل، برای جلوگیری از سرریز اعشار، محاسبه با افزودن لگاریتم احتمالات انجام می‌شود:

$$c_{\text{map}} = \arg \max_{c \in C} [\log \hat{P}(c) + \sum_{1 \leq k \leq n_d} \log \hat{P}(t_k|c)]$$

هر پارامتر شرطی $\log \hat{P}(t_k|c)$ یک وزن است که نشان می‌دهد t_k چقدر یک شاخص خوب برای c است. در سیستم فیلتر اسپم جیمیل، کلماتی مانند "ارزان"، "کاهش وزن" و "کلیک کنید" وزن بالایی برای کلاس اسپم دارند.

📊 تخمین پارامترها: از داده‌های واقعی به احتمالات

پارامترها با استفاده از **تخمین بیشترین درست نمایی** (MLE) تخمین زده می‌شوند:

$$\hat{P}(c) = N_c / N$$

$$\hat{P}(t|c) = T_{ct} / \sum_{t' \in V} T_{ct'}$$

⚡ مشکل صفر و راه‌حل: هموارسازی یک افزودن

مشکل با تخمین MLE این است که برای ترکیب واژه-کلاس که در داده‌های آموزشی رخ نداده است، صفر است. برای مثال، اگر کلمه "ویژه" فقط در ایمیل‌های غیراسپم ظاهر شده باشد، مدل MLE احتمال وقوع آن در ایمیل‌های اسپم را صفر در نظر می‌گیرد.

برای حذف صفرها، از هموارسازی یک افزودن (add-one smoothing) استفاده می‌کنیم:

$$\hat{P}(t|c) = (T_{ct} + 1) / (\sum_{t' \in V} (T_{ct'} + 1) + 1)$$

این تکنیک ساده اما مؤثر به جیمیل اجازه می‌دهد حتی با کلماتی که قبلاً در ایمیل‌های اسپم دیده نشده، تصمیم‌گیری منطقی انجام دهد.

🚀 الگوریتم کامل: از آموزش تا طبقه‌بندی

الگوریتم کامل برای آموزش و اعمال طبقه‌بند NB:

```

TRAINMULTINOMIALNB(C, ID)
1  V ← EXTRACTVOCABULARY(ID)
2  N ← COUNTDOCS(ID)
3  for each c ∈ C
4  do Nc ← COUNTDOCSINCLASS(ID, c)
5     prior[c] ← Nc/N
6     textc ← CONCATENATETEXTOFALLDOCSINCLASS(ID, c)
7     for each t ∈ V
8     do Tct ← COUNTTOKENSOFTERM(textc, t)
9     for each t ∈ V
10    do condprob[t][c] ←  $\frac{T_{ct}+1}{\sum_{t'} (T_{ct'}+1)}$ 
11  return V, prior, condprob

APPLYMULTINOMIALNB(C, V, prior, condprob, d)
1  W ← EXTRACTTOKENSFROMDOC(V, d)
2  for each c ∈ C
3  do score[c] ← log prior[c]
4     for each t ∈ W
5     do score[c] += log condprob[t][c]
6  return arg maxc ∈ C score[c]
    
```

► Figure 13.2 Naive Bayes algorithm (multinomial model): Training and testing.

🔍 مثال واقعی: برای طبقه‌بندی یک ایمیل با کلمات "چینی، توکیو، WTO"، سیستم بیز ساده ابتدا احتمال اولیه هر کلاس (اسپم در مقابل غیراسپم) را محاسبه کرده، سپس با توجه به فراوانی این کلمات در هر کلاس، احتمال نهایی را تخمین می‌زند.

پیچیدگی زمانی NB:

• آموزش: $\Theta(|D|L_{ave} + |C||V|)$

• آزمون: $\Theta(L_a + |C|M_a) = \Theta(|C|M_a)$

NB پیچیدگی زمانی بهینه دارد که یکی از دلایل محبوبیت آن به عنوان روش طبقه‌بندی متن است. همین الگوریتم ساده اما قدرتمند به جیمیل اجازه می‌دهد میلیاردها ایمیل را در لحظه‌هایی با دقت بالای 99٪ طبقه‌بندی کند.

```
In [1]: import math
from collections import defaultdict, Counter

class MultinomialNB:
    """
    چندجمله‌ای Naive Bayes پیاده‌سازی طبقه‌بند
    """

    def __init__(self):
        self.classes = []
        self.vocab = set()
        self.prior = {}
        self.cond_prob = defaultdict(dict)

    def train(self, documents, labels):
        """
        آموزش طبقه‌بند با استفاده از اسناد و برچسب‌های مربوطه
        documents: هر سند یک لیست از کلمات است
        labels: لیستی از برچسب‌های مربوط به هر سند
        """
        # استخراج کلاس‌ها و واژگان
        self.classes = list(set(labels))
        for doc in documents:
            self.vocab.update(doc)
        V = len(self.vocab) # اندازه واژگان

        # محاسبه احتمال‌های اولیه (P(c))
        N = len(documents)
        for c in self.classes:
            N_c = sum(1 for label in labels if label == c)
            self.prior[c] = N_c / N

        # محاسبه احتمالات شرطی با هموارسازی یک افزودنی (Laplace Smoothing)
        # فرمول: P(t|c) = (T_ct + 1) / (sum(T_ct') + V)
        for c in self.classes:
            # c الحاق تمام اسناد از کلاس
            class_docs = [doc for doc, label in zip(documents, labels) if label == c]

            # c برای شمارش کارآمدتر توکن‌ها در کلاس Counter استفاده از
            text_c_counter = Counter(word for doc in class_docs for word in doc)

            # T_ct' (مجموع c شمارش کل توکن‌ها در اسناد کلاس)
            total_tokens_in_c = sum(text_c_counter.values())

            # محاسبه احتمال شرطی برای هر واژه در واژگان
            for t in self.vocab:
                T_ct = text_c_counter.get(t, 0) # c در کلاس t تعداد تکرار واژه
                # استفاده از هموارسازی یک افزودنی
                self.cond_prob[t][c] = (T_ct + 1) / (total_tokens_in_c + V)

    def predict(self, document):
        """
        سند جدید، یک لیست از کلمات
        """
        scores = {}

        # هر کلاس (log-probability) محاسبه امتیاز
        for c in self.classes:
            # شروع با لگاریتم احتمال اولیه
            score = math.log(self.prior[c])

            # افزودن لگاریتم احتمالات شرطی برای هر واژه در سند
            for word in document:
                # فقط واژه‌هایی که در واژگان آموزشی هستند را در نظر می‌گیریم
                if word in self.vocab:
                    score += math.log(self.cond_prob[word][c])
            scores[c] = score

        # برای اهداف آموزشی، امتیازات نهایی را چاپ می‌کنیم
        print("\n امتیازات نهایی (log-probabilities):")
        for c, score in scores.items():
            print(f" Score({c}): {score:.4f}")

        # بازگرداندن کلاس با بیشترین امتیاز
        return max(scores.items(), key=lambda x: x[1])[0]

# Introduction to Information Retrieval) مثال از جدول 13.1 کتاب
# داده‌های آموزشی
train_docs = [
    ["Chinese", "Beijing", "Chinese"],
    ["Chinese", "Chinese", "Shanghai"],
    ["Chinese", "Macao"],
    ["Tokyo", "Japan", "Chinese"]
]
train_labels = ["China", "China", "China", "not China"]
```

```
# داده‌های آزمون
test_doc = ["Chinese", "Chinese", "Chinese", "Tokyo", "Japan"]

# ایجاد و آموزش طبقه‌بند
nb = MultinomialNB()
nb.train(train_docs, train_labels)

# پیش‌بینی کلاس سند آزمون
predicted_class = nb.predict(test_doc)
print(f"\nسند آزمون به کلاس '{predicted_class}'")

# چاپ احتمالات شرطی محاسبه شده برای واژه‌های کلیدی
print("\n(احتمالات شرطی محاسبه شده (با هموارسازی)
for word in ["Chinese", "Tokyo", "Japan"]:
    print(f"P({word}|China) = {nb.cond_prob[word]['China']:.4f}")
    print(f"P({word}|not China) = {nb.cond_prob[word]['not China']:.4f}")
```

(log-probabilities) امتیازات نهایی
Score(not China): -8.9067
Score(China): -8.1077

اختصاص داده شد 'China' سند آزمون به کلاس

:احتمالات شرطی محاسبه شده (با هموارسازی)
P(Chinese|China) = 0.4286
P(Chinese|not China) = 0.2222
P(Tokyo|China) = 0.0714
P(Tokyo|not China) = 0.2222
P(Japan|China) = 0.0714
P(Japan|not China) = 0.2222

13.2.1 ارتباط با مدل زبانی یونیگرام چندجمله‌ای: چگونه گوگل کلمات را پیش‌بینی می‌کند؟

🔍 دو روی یک سکه: طبقه بندی و مدل زبانی

مدل **بیز ساده چندجمله‌ای** به صورت رسمی با **مدل زبانی یونیگرام چندجمله‌ای** یکسان است. بیایید این را با یک مثال واقعی بررسی کنیم: وقتی شما در گوگل عبارت "پیش‌بینی کلمات بعدی" را تایپ می‌کنید، گوگل با استفاده از مدل زبانی، کلماتی را که به احتمال زیاد بعد از عبارت شما ظاهر می‌شوند پیشنهاد می‌کند.

در طبقه‌بندی متن، نقش‌ها برعکس است:

- سند d در طبقه‌بندی متن نقش پرسمان q در مدل زبانی را دارد.

- کلاس‌های c در طبقه بندی متن نقش اسناد d در مدل زبانی را دارند.

📐 معادله یکسان: ارتباط ریاضی دو مفهوم

معادله (13.2) یک حالت خاص از معادله (12.12) برای $\lambda = 1$ است:

$$P(d|q) \propto P(d) \prod_{t \in q} P(t|M_d)$$

این ارتباط به ما نشان می‌دهد که طبقه‌بندی بیز ساده در واقع یک کاربرد خاص از مدل زبانی است. در مدل زبانی، ما احتمال وقوع کلمات در یک سند را بر اساس احتمال وقوع آن‌ها در کل مجموعه تخمین می‌زنیم. در طبقه‌بندی بیز، ما احتمال وقوع کلمات در یک کلاس را بر اساس احتمال وقوع آن‌ها در اسناد آن کلاس تخمین می‌زنیم.

🤖 مشکل صفر و راه‌حل: هموارسازی در دو جهان

هر دو مدل از تخمین MLE استفاده می‌کنند و با مشکل تخمین‌های صفر به دلیل داده‌های پراکنده مواجه می‌شوند. برای حل این مشکل:

- در بیز ساده از **هموارسازی یک افزودن** (add-one smoothing) استفاده می‌شود.
- در مدل زبانی از ترکیبی از دو توزیع استفاده شد.

هموارسازی یک افزودن با هموارسازی add-1/2 در بخش 11.3.4 مرتبط است. این روش‌ها به هم نزدیک هستند اما با جزئیات کمی متفاوت عمل می‌کنند.

💡 درک عمیق‌تر: چرا این ارتباط مهم است؟

درک این ارتباط به ما کمک می‌کند:

1. می‌توانیم تکنیک‌های بهینه‌سازی توسعه یافته برای مدل‌های زبانی را در طبقه بندی متن به کار گیریم.
2. می‌توانیم بفهمیم چرا بیز ساده با وجود فرض ساده‌کننده استقلال واژه‌ها، در عمل کارایی بالایی دارد.
3. می‌توانیم روش‌های هموارسازی پیشرفته‌تر را که در مدل‌های زبانی توسعه داده شده، در طبقه بندی متن به کار گیریم.

13.3 مدل برنولی: برای طبقه‌بندی ایمیل‌های کوتاه کدام بهتر است ؟ 📊

🔍 دو رویکرد متفاوت برای طبقه‌بندی

تا کنون دو مدل بیز ساده را بررسی کردیم: مدل **چندجمله‌ای** که تعداد وقوع کلمات را در نظر می‌گیرد و مدل **برنولی** که فقط وجود یا عدم وجود کلمات را بررسی می‌کند. بیایید این تفاوت را با یک مثال واقعی بررسی کنیم:

فرض کنید در حال توسعه یک سیستم فیلتر اسپم برای پیام‌رسان‌های کوتاه هستید. در این سیستم، مهم است که بدانیم آیا پیام حاوی کلمات کلیدی مانند "کاهش وزن" یا "رایگان" است یا خیر. در این حالت، مدل برنولی انتخاب بهتری است، زیرا:

- در پیام‌های کوتاه، هر کلمه معمولاً فقط یک بار ظاهر می‌شود
- وجود یا عدم وجود کلمه، اطلاعات بیشتری از تعداد تکرار آن ارائه می‌دهد
- حجم محاسبات کمتری نیاز دارد (فقط نیاز به بررسی وجود کلمات)

📈 تفاوت در نحوه محاسبه احتمالات

تفاوت اصلی بین دو مدل در نحوه محاسبه احتمالات شرطی $\hat{P}(t|c)$ است:

- **مدل چندجمله‌ای:** $\hat{P}(t|c)$ به عنوان کسری از توکن‌ها یا موقعیت‌ها در اسناد کلاس c تخمین زده می‌شود.
- **مدل برنولی:** $\hat{P}(t|c)$ به عنوان کسری از اسناد کلاس c که شامل واژه t هستند تخمین زده می‌شود.

این تفاوت به نتایج متفاوتی منجر می‌شود. برای مثال، اگر کلمه "چین" در یک سند ۵ بار تکرار شده باشد، مدل چندجمله‌ای این را یک شاخص بسیار قوی برای کلاس "چین" در نظر می‌گیرد، در حالی که مدل برنولی فقط وجود آن را بررسی می‌کند.

🏠 الگوریتم برنولی و هموارسازی

الگوریتم کامل برای آموزش و اعمال طبقه‌بند برنولی:

```

TRAINBERNOULLINB(C, ID)
1   $V \leftarrow \text{EXTRACTVOCABULARY}(\text{ID})$ 
2   $N \leftarrow \text{COUNTDOCS}(\text{ID})$ 
3  for each  $c \in C$ 
4  do  $N_c \leftarrow \text{COUNTDOCSINCLASS}(\text{ID}, c)$ 
5      $\text{prior}[c] \leftarrow N_c / N$ 
6     for each  $t \in V$ 
7     do  $N_{ct} \leftarrow \text{COUNTDOCSINCLASSCONTAININGTERM}(\text{ID}, c, t)$ 
8         $\text{condprob}[t][c] \leftarrow (N_{ct} + 1) / (N_c + 2)$ 
9  return  $V, \text{prior}, \text{condprob}$ 

```

```

APPLYBERNOULLINB(C, V, prior, condprob, d)
1   $V_d \leftarrow \text{EXTRACTTERMSFROMDOC}(V, d)$ 
2  for each  $c \in C$ 
3  do  $\text{score}[c] \leftarrow \log \text{prior}[c]$ 
4     for each  $t \in V$ 
5     do if  $t \in V_d$ 
6        then  $\text{score}[c] += \log \text{condprob}[t][c]$ 
7        else  $\text{score}[c] += \log(1 - \text{condprob}[t][c])$ 
8  return  $\arg \max_{c \in C} \text{score}[c]$ 

```

► **Figure 13.3** NB algorithm (Bernoulli model): Training and testing. The add-one smoothing in Line 8 (top) is in analogy to Equation (13.7) with $B = 2$.

نکته‌های مهم در الگوریتم برنولی:

- در خط ۸، از هموارسازی یک افزودن با $B=2$ استفاده شده (دو حالت: وجود یا عدم وجود واژه)
- در خط ۷، احتمال عدم وقوع واژه نیز در محاسبه نهایی لحاظ می‌شود (خط ۵-۷)
- این الگوریتم همان پیچیدگی زمانی مدل چندجمله‌ای را دارد

🔗 مثال عملی: طبقه‌بندی پیام‌های کوتاه

بیا ببینیم مثال 13.2 را با یک سناریوی واقعی بررسی کنیم. فرض کنید در حال توسعه یک سیستم طبقه‌بندی برای پیام‌های کوتاه هستید و باید تصمیم بگیرید که آیا پیام‌های دریافتی از طرف "بانک" هستند یا "تبلیغات" است.

در این حالت، مدل برنولی انتخاب بهتری است زیرا:

1. پیام‌های کوتاه معمولاً شامل کلمات کلیدی مشخصی هستند (مانند "مانده حساب"، "تراکنش")
2. وجود یا عدم وجود این کلمات، اطلاعات بیشتری از تعداد تکرار آن‌ها ارائه می‌دهد
3. برای طبقه‌بندی سریع، مدل برنولی نیاز به محاسبات کمتری دارد

با اعمال مدل برنولی بر داده‌های آموزشی و پیش‌بینی پیام جدید "کاهش وزن تراکنش"، سیستم به درستی آن را به کلاس "بانک" اختصاص می‌دهد، زیرا کلمات کلیدی بانک وجود دارند، در حالی که کلمات تبلیغاتی وجود ندارند.

```

In [3]: import math
from collections import defaultdict

class BernoulliNB:
    """
    Bernoulli Naive Bayes (پایه‌سازی طبقه‌بندی برنولی بیس)
    """
    def __init__(self):
        self.classes = []
        self.vocab = set()
        self.prior = {}
        self.cond_prob = defaultdict(dict)

```

```
def train(self, documents, labels):
    """
    آموزش طبقه‌بند برنولی با استفاده از اسناد و برچسب‌های مربوطه
    documents: لیستی از اسناد، هر سند یک لیست از کلمات است
    labels: لیستی از برچسب‌های مربوط به هر سند
    """

    # استخراج کلاس‌ها و واژگان
    self.classes = list(set(labels))
    for doc in documents:
        self.vocab.update(doc)
    V = len(self.vocab)

    # محاسبه احتمال‌های اولیه  $\hat{P}(c)$ 
    N = len(documents)
    for c in self.classes:
        N_c = sum(1 for label in labels if label == c)
        self.prior[c] = N_c / N

    # محاسبه احتمالات شرطی با هموارسازی یک افزودنی  $\hat{P}(t|c)$ 
    # فرمول:  $\hat{P}(t|c) = (N_{ct} + 1) / (N_c + 2)$ 
    for c in self.classes:
        N_c = sum(1 for label in labels if label == c)
        for t in self.vocab:
            # وجود دارد c در آن کلاس t شمارش اسنادی که واژه
            docs_with_term = sum(1 for doc, label in zip(documents, labels)
                                if label == c and t in doc)
            # دو حالت: وجود یا عدم وجود B=2 استفاده از هموارسازی یک افزودن با
            self.cond_prob[t][c] = (docs_with_term + 1) / (N_c + 2)

def predict(self, document):
    """
    پیش‌بینی کلاس یک سند جدید با مدل برنولی
    document: سند جدید، یک لیست از کلمات
    """

    scores = {}
    doc_set = set(document) # برای بررسی سریع‌تر وجود واژه

    # محاسبه امتیاز هر کلاس
    for c in self.classes:
        # شروع با لگاریتم احتمال اولیه
        score = math.log(self.prior[c])

        # برای تمام واژگان، احتمال وجود یا عدم وجود را در نظر می‌گیریم
        for t in self.vocab:
            if t in doc_set:
                # استفاده کن  $P(t|c)$  اگر واژه در سند وجود دارد، از
                score += math.log(self.cond_prob[t][c])
            else:
                # استفاده کن  $P(t|c)$  - اگر واژه در سند وجود ندارد، از 1
                score += math.log(1 - self.cond_prob[t][c])

        scores[c] = score

    return scores

try:
    # پیام برای اطمینان از شروع اجرا
    print("...شروع اجرای برنامه")

    # مثال: طبقه‌بندی پیام‌های کوتاه به "بانکی" یا "تبلیغات"
    train_docs = [
        ["موجودی", "حساب", "شما", "مثبت", "است"],
        ["کاهش", "وزن", "تراکنش", "یافت", "شد"],
        ["چشن", "تحقیف", "فروش", "ویژه", "است"],
        ["برنده", "خوش", "شانس", "میلیون", "تومان"]
    ]
    train_labels = ["بانک", "بانک", "تبلیغات", "تبلیغات"]
    test_doc = ["کاهش", "وزن", "تراکنش", "شما"]

    # ایجاد و آموزش طبقه‌بند
    nb = BernoulliNB()
    nb.train(train_docs, train_labels)
    print("آموزش مدل با موفقیت انجام شد")

    # پیش‌بینی کلاس سند آزمون
    scores = nb.predict(test_doc)
    predicted_class = max(scores.items(), key=lambda x: x[1])[0]

    # نمایش نتایج به صورت مستقیم و واضح
    print("\n--- نتایج ---")
    print("log-probabilities:")
    for c, score in scores.items():
        print(f"Score({c}): {score:.4f}")

    print(f"\nکلاس به '{predicted_class}' پیام داده شد")

    # نمایش احتمالات شرطی برای کلمات کلیدی
    print("\nاحتمالات شرطی محاسبه شده (با هموارسازی)")
    for word in ["کاهش", "وزن", "تراکنش", "چشن", "برنده"]:
        print(f"P({word}|بانک) = {nb.cond_prob[word]['بانک']:.4f}")
        print(f"P({word}|تبلیغات) = {nb.cond_prob[word]['تبلیغات']:.4f}")

    print("\nاجرای برنامه به پایان رسید")

except Exception as e:
    print(f"\nیک خطا رخ داد: {e}")
```

...شروع اجرای برنامه
آموزش مدل با موفقیت انجام شد

--- نتایج ---
(log-probabilities) امتیازات نهایی:
Score(بانک): -10.2138
Score(تبلیغات): -14.6082

پیام به کلاس 'بانک' اختصاص داده شد

احتمالات شرطی محاسبه شده (با هموارسازی):
P(کاهش|بانک) = 0.5000
P(کاهش|تبلیغات) = 0.2500
P(وزن|بانک) = 0.5000
P(وزن|تبلیغات) = 0.2500
P(تراکنش|بانک) = 0.5000
P(تراکنش|تبلیغات) = 0.2500
P(جشن|بانک) = 0.2500
P(جشن|تبلیغات) = 0.5000
P(برنده|بانک) = 0.2500
P(برنده|تبلیغات) = 0.5000

اجرای برنامه به پایان رسید

13.4 ویژگی‌های بیز ساده: چرا روشی "ساده‌انگارانه" آنقدر قدرتمند است؟

بیز ساده بر اساس قانون بیز عمل می‌کند: سندی به کلاسی اختصاص می‌یابد که بیشترین احتمال پسین را داشته باشد:

$$c_{\text{map}} = \arg \max_{c \in C} P(c|d) = \arg \max_{c \in C} P(d|c)P(c)$$

در اینجا $P(c|d)$ احتمال اینکه سند d به کلاس c تعلق دارد را نشان می‌دهد. با استفاده از قانون بیز، این احتمال به $P(d|c)P(c)$ تبدیل می‌شود که مخرج آن ($P(d)$) برای همه کلاس‌ها یکسان است و می‌توان آن را نادیده گرفت.

🎯 دو رویکرد متفاوت برای مدل‌سازی اسناد

بیز ساده دو مدل اصلی دارد که نحوه تولید سند را به شکلی متفاوت مدل می‌کنند:

چندجمله‌ای: $P(d|c) = P(h_{t_1}, \dots, t_k, \dots, t_{n_d}|c)$

برنولی: $P(d|c) = P(h_{e_1}, \dots, e_i, \dots, e_M|c)$

در مدل چندجمله‌ای، ما به دنبال دنباله واژه‌ها در سند هستیم، در حالی که در مدل برنولی، فقط به وجود یا عدم وجود هر واژه توجه می‌کنیم.

مثال واقعی: برای طبقه‌بندی نظرات مشتریان آمازون، مدل چندجمله‌ای به تعداد تکرار واژه "عالی" در یک نظر توجه می‌کند، در حالی که مدل برنولی فقط بررسی می‌کند که آیا واژه "عالی" در نظر وجود دارد یا خیر.

🌱 فرض استقلال شرطی: ساده‌سازی هوشمندانه

برای کاهش تعداد پارامترها، بیز ساده فرض می‌کند که ویژگی‌ها با توجه به کلاس از یکدیگر مستقل هستند:

چندجمله‌ای: $P(d|c) = \prod_{1 \leq k \leq n_d} P(X_k = t_k|c)$

برنولی: $P(d|c) = \prod_{1 \leq i \leq M} P(U_i = e_i|c)$

این فرض به ما اجازه می‌دهد تا به جای تخمین $|2^M|C$ پارامتر، تنها $\Theta(M|C)$ پارامتر را تخمین بزنیم، که در عمل بسیار کارآمدتر است.

مثال واقعی: در طبقه‌بندی اخبار رویترز، با وجود اینکه واژه‌های "نیویورک" و "بورس" اغلب با هم می‌آیند، بیز ساده فرض می‌کند این دو واژه با توجه به کلاس "اقتصاد" از یکدیگر مستقل هستند. با وجود این فرض نادرست، الگوریتم همچنان به

📌 استقلال موقعیتی: نادیده گرفتن ترتیب واژه‌ها

در مدل چندجمله‌ای، فرض دیگری به نام استقلال موقعیتی نیز وجود دارد: احتمال وقوع یک واژه در سند به موقعیت آن بستگی ندارد. این فرض معادل مدل bag of words است که ترتیب واژه‌ها را نادیده می‌گیرد.

مثال واقعی: برای سیستم طبقه‌بندی اسناد داخلی مایکروسافت، عبارت "مدیریت پروژه" و "پروژه مدیریت" به یک شکل در نظر گرفته می‌شوند، زیرا ترتیب واژه‌ها در مدل بی‌ساده چندجمله‌ای نادیده گرفته می‌شود.

جدول زیر دو مدل را با هم مقایسه می‌کند:

ویژگی	مدل چندجمله‌ای	مدل برنولی
مدل رویداد	تولید توکن	تولید سند
متغیر تصادفی	$X = t$ اگر t در موقعیت مشخصی رخ دهد	$U_t = 1$ اگر t در سند رخ دهد
نمایش سند	$h_{t_1, \dots, t_{n_d}}$	$h_{e_1, \dots, e_M}$
قانون تصمیم	$\text{maximize } \hat{P}(c) \prod_{1 \leq k \leq n_d} \hat{P}(X = t_k c)$	$\text{maximize } \hat{P}(c) \prod_{t_i \in V} \hat{P}(U_i = e_i c)$
تعداد وقوع‌ها	در نظر گرفته می‌شود	نادیده گرفته می‌شود
طول اسناد	می‌تواند اسناد طولانی را مدیریت کند	برای اسناد کوتاه بهتر عمل می‌کند
کاربرد واقعی	طبقه‌بندی مقالات خبری طولانی	فیلتر کردن پیام‌های کوتاه توییتر

🤔 چرا بی‌ساده با وجود فرضیات اشتباه خوب کار می‌کند؟

بی‌ساده به این نام خوانده می‌شود زیرا فرضیات استقلال آن برای مدل زبان طبیعی بسیار ساده‌انگارانه است. با این حال، تخمین‌های احتمالی بی‌ساده کیفیت پایینی دارند، اما تصمیمات طبقه‌بندی آن به طرز شگفت‌آوری خوب هستند.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان شرکت فیسبوک مارکت، یک نظر حاوی واژه‌های "عالی"، "خوب" و "پیشنهاد می‌کنم" با احتمال واقعی 0.6 به کلاس "مثبت" تعلق دارد. بی‌ساده این احتمال را به اشتباه 0.99 تخمین می‌زند، اما همچنان نظر را به درستی به کلاس "مثبت" اختصاص می‌دهد.

این پدیده به این دلیل رخ می‌دهد که در طبقه‌بندی بی‌ساده، کلاس برنده معمولاً احتمال بسیار بزرگتری نسبت به کلاس‌های دیگر دارد، حتی اگر این احتمال به اشتباه تخمین زده شده باشد. تصمیم طبقه‌بندی بر اساس کلاسی است که بیشترین امتیاز را کسب می‌کند، نه بر اساس دقت تخمین احتمال.

💡 مزایای بی‌ساده در دنیای واقعی

بی‌ساده دارای مزایای زیادی است که آن را به یک ابزار قدرتمند در صنعت تبدیل کرده است:

- در شرایطی که بسیاری از ویژگی‌های مهم مشترکاً به تصمیم طبقه‌بندی کمک می‌کنند، عالی عمل می‌کند. **مثال:** در سیستم طبقه‌بندی اسناد حقوقی شرکت اپل، واژه‌های متعددی مانند "قرارداد"، "حقوقی"، "توافق" و "قانون" با هم به تصمیم‌گیری کمک می‌کنند.
- تا حدی در برابر ویژگی‌های نویز و تغییر مفهوم (concept drift) مقاوم است. **مثال:** سیستم فیلتر اسپم یاهو با گذشت زمان و تغییر تاکتیک‌های اسپمرها، همچنان به خوبی عمل می‌کند.

- کارایی بالایی دارد: آموزش و طبقه‌بندی با یک عبور از داده‌ها قابل انجام است. **مثال:** گوگل می‌تواند میلیاردها وب‌صفحه را با استفاده از بیز ساده در زمان کوتاهی طبقه‌بندی کند.
- اغلب به عنوان یک خط مبنا در تحقیقات طبقه‌بندی متن استفاده می‌شود. **مثال:** محققان آمازون قبل از آزمایش مدل‌های پیچیده، ابتدا عملکرد بیز ساده را به عنوان خط مبنا ارزیابی می‌کنند.

در حالی که بیز ساده ممکن است دقیق‌ترین روش برای طبقه‌بندی متن نباشد، اما ترکیبی از کارایی و دقت خوب آن را به یک گزینه قوی برای بسیاری از کاربردهای تجاری تبدیل کرده است، از فیلتر کردن اسپم گرفته تا طبقه‌بندی اخبار و تحلیل احساسات مشتریان.

13.4.1 یک نوع دیگر از مدل چندجمله‌ای

گوگل و دسته‌بندی ایمیل‌ها: چرا فراوانی کلمات مهم است؟

وقتی گوگل ایمیل شما را دسته‌بندی می‌کند، به فراوانی کلمات در متن توجه می‌کند، نه فقط به وجود یا عدم وجود آن‌ها. ایمیل‌های اسپم معمولاً کلماتی مانند "ارزان"، "تخفیف"، "فوری" را چندین بار تکرار می‌کنند. این دقیقاً همان چیزی است که مدل چندجمله‌ای به ما می‌دهد - توانایی در نظر گرفتن تعداد تکرار هر کلمه.

در این نوع از مدل چندجمله‌ای، هر سند d به صورت یک بردار M بعدی از فراوانی‌ها $h_{tf_{t_1,d}}, \dots, tf_{t_M,d}$ نمایش داده می‌شود که در آن $tf_{t_i,d}$ تعداد دفعات تکرار واژه t_i در سند d است.

$$P(d|c) = P(h_{tf_{t_1,d}}, \dots, tf_{t_M,d} | c) \propto \prod_{1 \leq i \leq M} P(X = t_i | c)^{tf_{t_i,d}}$$

تفاوت کلیدی: توان فراوانی

نکته جالب در این فرمول، توان $tf_{t_i,d}$ است که به احتمال هر کلمه داده می‌شود. این یعنی هرچه یک کلمه بیشتر در سند تکرار شود، تأثیر بیشتری در احتمال نهایی خواهد داشت.

مثال واقعی: در سیستم دسته‌بندی نظرات مشتریان آمازون، اگر در یک نظر کلمه "عالی" ۵ بار تکرار شود و کلمه "بد" فقط یک بار، مدل چندجمله‌ای این تفاوت را در نظر می‌گیرد و احتمال مثبت بودن نظر را افزایش می‌دهد.

توجه کنید که در این فرمول، عامل چندجمله‌ای حذف شده است. این کار محاسبات را ساده‌تر می‌کند بدون اینکه تأثیر قابل توجهی بر دقت طبقه‌بندی داشته باشد.

معادل با مدل دنباله

این مدل با مدل دنباله در معادله (13.2) معادل است، زیرا:

- برای واژه‌هایی که در سند وجود ندارند ($tf_{t_i,d} = 0$)، $P(X = t_i | c)^0 = 1$ است و تأثیری در محاسبات ندارد.

- برای واژه‌هایی که $tf_{t,d} \geq 1$ بار تکرار می‌شوند، این واژه در هر دو مدل به تعداد $tf_{t,d}$ بار در محاسبات شرکت می‌کند.

این رویکرد به ویژه در کاربردهایی که تعداد تکرار کلمات اهمیت دارد، مانند تحلیل احساسات در نظرات مشتریان یا تشخیص اسپم، بسیار مفید است.

13.5 انتخاب ویژگی: چگونه کلمات مهم را از کلمات بی‌اهمیت تشخیص دهیم؟ 🔍

🔗 فیلتر اسپم گوگل و انتخاب ویژگی

وقتی گوگل ایمیل شما را فیلتر می‌کند، از هزاران کلمه به عنوان ویژگی استفاده می‌کند. اما آیا همه این کلمات equally مهم هستند؟ کلماتی مانند "ارزان"، "تخفیف" و "برنده" در ایمیل‌های اسپم اهمیت بیشتری دارند. انتخاب ویژگی به گوگل کمک می‌کند تا روی این کلمات مهم تمرکز کند و نویز را نادیده بگیرد.

انتخاب ویژگی فرآیند انتخاب زیرمجموعه‌ای از واژه‌های موجود در مجموعه آموزشی و استفاده فقط از این زیرمجموعه به عنوان ویژگی در طبقه‌بندی متن است. این فرآیند دو هدف اصلی دارد:

1. **افزایش کارایی** با کاهش اندازه واژگان مؤثر، آموزش و اعمال طبقه‌بند را کارآمدتر می‌کند. این موضوع برای طبقه‌بندهایی که برخلاف NB، آموزش آن‌ها پرهزینه است، اهمیت ویژه‌ای دارد.
2. **افزایش دقت طبقه‌بندی** با حذف **ویژگی‌های نویز**. ویژگی نویز ویژگی است که وقتی به نمایش سند اضافه می‌شود، خطای طبقه‌بندی روی داده‌های جدید را افزایش می‌دهد.

⚠️ خطر بیش‌برازش: چگونه یک کلمه نادر می‌تواند کل مدل را خراب کند؟

یک واژه نادر مانند "arachnocentric" ممکن است هیچ اطلاعاتی درباره یک کلاس (مثلاً China) نداشته باشد، اما تمام نمونه‌های "arachnocentric" به صورت اتفاقی در اسناد China در مجموعه آموزشی ما رخ داده باشند. در این حالت، روش یادگیری ممکن است طبقه‌بندی تولید کند که اسناد آزمون حاوی "arachnocentric" را به اشتباه به کلاس China اختصاص می‌دهد. این تعمیم نادرست از یک ویژگی تصادفی مجموعه آموزشی **بیش‌برازش** (overfitting) نامیده می‌شود.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان آمازون، اگر واژه "خوشمزه" به اشتباه در چند نظر منفی درباره یک محصول ظاهر شود، ممکن است مدل یاد بگیرد که این واژه نشان‌دهنده نظر منفی است، در حالی که واقعیت چیز دیگری است.

الگوریتم انتخاب ویژگی (شکل 13.6) برای انتخاب k ویژگی برتر طراحی شده است:

```

SELECTFEATURES( $\mathbb{D}, c, k$ )
1   $V \leftarrow \text{EXTRACTVOCABULARY}(\mathbb{D})$ 
2   $L \leftarrow []$ 
3  for each  $t \in V$ 
4  do  $A(t, c) \leftarrow \text{COMPUTEFEATUREUTILITY}(\mathbb{D}, t, c)$ 
5     APPEND( $L, \langle A(t, c), t \rangle$ )
6  return FEATURESWITHLARGESTVALUES( $L, k$ )

```

► Figure 13.6 Basic feature selection algorithm for selecting the k best features.

🔗 چگونه یک طبقه‌بند ضعیف‌تر می‌تواند بهتر عمل کند؟

می‌توان انتخاب ویژگی را به عنوان روشی برای جایگزینی یک طبقه‌بند پیچیده (با استفاده از همه ویژگی‌ها) با یک طبقه‌بند ساده‌تر (با استفاده از زیرمجموعه‌ای از ویژگی‌ها) در نظر گرفت. اگرچه در نگاه اول ممکن است غیرمنطقی به نظر برسد که یک طبقه‌بند به ظاهر ضعیف‌تر در طبقه‌بندی آماری متن مطلوب است، اما در بحث معاوضه بایاس-واریانس (bias-variance tradeoff) خواهیم دید که مدل‌های ضعیف‌تر اغلب زمانی که داده‌های آموزشی محدودی در دسترس هستند، ترجیح داده می‌شوند.

مثال واقعی: در سیستم تشخیص اخبار جعلی فیسبوک، استفاده از تمام کلمات ممکن است منجر به بیش‌برازش شود، زیرا برخی کلمات به صورت اتفاقی در اخبار جعلی ظاهر می‌شوند. با انتخاب ویژگی‌های کلیدی مانند "شایعه"، "تأیید نشده" و "منبع نامعتبر"، مدل می‌تواند عملکرد بهتری داشته باشد.

الگوریتم اساسی انتخاب ویژگی در شکل 13.6 نشان داده شده است. برای یک کلاس مشخص c ، یک معیار کاربردی $A(t, c)$ برای هر واژه واژگان محاسبه کرده و k واژه با بالاترین مقادیر $A(t, c)$ را انتخاب می‌کنیم. سایر واژه‌ها نادیده گرفته شده و در طبقه‌بندی استفاده نمی‌شوند.

در این بخش سه معیار کاربردی مختلف معرفی می‌شود:

اطلاع متقابل: $A(t, c) = I(U_t; C_c)$

آزمون χ^2 : $A(t, c) = \chi^2(t, c)$

فراوانی: $A(t, c) = N(t, c)$

🔍 مدل برنولی و حساسیت به نویز

از دو مدل NB، مدل برنولی ویژه نسبت به ویژگی‌های نویز حساس است. یک طبقه‌بند برنولی NB به شکلی از انتخاب ویژگی نیاز دارد وگرنه دقت آن پایین خواهد بود.

مثال واقعی: در سیستم تحلیل احساسات توییتر، مدل برنولی اگر از انتخاب ویژگی استفاده نکند، ممکن است به کلمات نادر و بی‌اهمیت که به صورت اتفاقی در توییت‌های مثبت یا منفی ظاهر شده‌اند، بیش از حد حساس شود و دقت طبقه‌بندی کاهش یابد.

این بخش عمدتاً به انتخاب ویژگی برای وظایف طبقه‌بندی دو کلاسی مانند China در مقابل not-China می‌پردازد. بخش 13.5.5 به طور خلاصه به بهینه‌سازی برای سیستم‌های با بیش از دو کلاس می‌پردازد.

13.5.1 اطلاعات متقابل: چگونه کلمات کلیدی را برای طبقه بندی پیدا کنیم؟

توئیت و تشخیص موضوعات: چگونه کلمات مهم را از کلمات بی‌اهمیت تشخیص دهیم؟

تصور کنید در توئیت‌ها کار می‌کنید و باید سیستم طراحی کنید که توئیت‌ها را به موضوعات مختلفی مانند "ورزش"، "سیاست" یا "فناوری" دسته‌بندی کند. با میلیون‌ها توئیت در روز، چگونه می‌توانید کلماتی را پیدا کنید که بیشترین اطلاعات را درباره موضوع یک توئیت دارند؟

این دقیقاً همان مشکلی است که اطلاعات متقابل (Mutual Information) حل می‌کند. این معیار به ما می‌گوید چقدر وجود یا عدم وجود یک کلمه در توئیت، به ما در تشخیص موضوع آن توئیت کمک می‌کند.

یک روش متداول برای انتخاب ویژگی، محاسبه **اطلاعات متقابل (Mutual Information - MI)** برای واژه t و کلاس c است. MI میزان اطلاعاتی را اندازه‌گیری می‌کند که وجود/عدم وجود یک واژه به تصمیم طبقه‌بندی صحیح برای c کمک می‌کند.

فرمول MI به صورت زیر است:

$$I(U; C) = \sum_{e_t \in \{1,0\}} \sum_{e_c \in \{1,0\}} P(U = e_t, C = e_c) \log_2 \frac{P(U = e_t, C = e_c)}{P(U = e_t)P(C = e_c)}$$

که در آن U متغیر تصادفی است که مقادیر $e_t = 1$ (سند شامل واژه t است) و $e_t = 0$ (سند شامل واژه t نیست) می‌گیرد، و C متغیر تصادفی است که مقادیر $e_c = 1$ (سند در کلاس c است) و $e_c = 0$ (سند در کلاس c نیست) می‌گیرد.

از تئوری تا عمل: چگونه MI را محاسبه کنیم؟

در دنیای واقعی، ما از تخمین‌های بیشترین درست‌نمایی برای احتمالات استفاده می‌کنیم. این کار فرمول بالا را به معادله زیر تبدیل می‌کند که برای محاسبه عملی مناسب‌تر است:

$$\begin{aligned} I(U; C) = & \frac{N_{11}}{N} \log_2 \frac{N \cdot N_{11}}{N_{1.} \cdot N_{.1}} \\ & + \frac{N_{01}}{N} \log_2 \frac{N \cdot N_{01}}{N_{0.} \cdot N_{.1}} \\ & + \frac{N_{10}}{N} \log_2 \frac{N \cdot N_{10}}{N_{1.} \cdot N_{.0}} \\ & + \frac{N_{00}}{N} \log_2 \frac{N \cdot N_{00}}{N_{0.} \cdot N_{.0}} \end{aligned}$$

که در آن N_{ij} تعداد اسنادی است که مقادیر e_t و e_c را دارند که با دو زیرنشان مشخص شده‌اند. برای مثال، N_{10} تعداد اسنادی است که شامل واژه t هستند ($e_t = 1$) اما در کلاس c نیستند ($e_c = 0$).

مثال واقعی: طبقه‌بندی اخبار کشاورزی

مثال 13.3: برای کلاس poultry (طیور) و واژه export (صادرات) در مجموعه اخبار Reuters-RCV1، تعداد اسناد با چهار ترکیب ممکن از مقادیر شاخص به صورت زیر است:

$e_c = \text{poultry} = 1$ $e_c = \text{poultry} = 0$		
$e_t = \text{export} = 1$	$N_{11} = 49$	$N_{10} = 27,652$
$e_t = \text{export} = 0$	$N_{01} = 141$	$N_{00} = 774,106$

با جایگذاری این مقادیر در معادله، به دست می‌آوریم: $I(U; C) \approx 0.0001105$

این عدد کوچک نشان می‌دهد که واژه "export" به تنهایی اطلاعات کمی درباره کلاس "طیور" دارد. این منطقی است، زیرا "export" در بسیاری از زمینه‌ها استفاده می‌شود، نه فقط در اخبار مربوط به طیور.

برای انتخاب k واژه $t_1, \dots, t_k$ برای یک کلاس مشخص، از الگوریتم انتخاب ویژگی در شکل 13.6 استفاده می‌کنیم: معیار کاربردی را به صورت $A(t, c) = I(U_t, C_c)$ محاسبه کرده و k واژه با بالاترین مقادیر را انتخاب می‌کنیم.

🌐 کلماتی که جهان را توصیف می‌کنند: MI در عمل

شکل 13.7 واژه‌هایی با امتیازات بالای اطلاعات متقابل را برای شش کلاس خبری نشان می‌دهد. این کلمات به ما نشان می‌دهند که چگونه MI می‌تواند کلمات کلیدی را برای هر موضوع شناسایی کند:

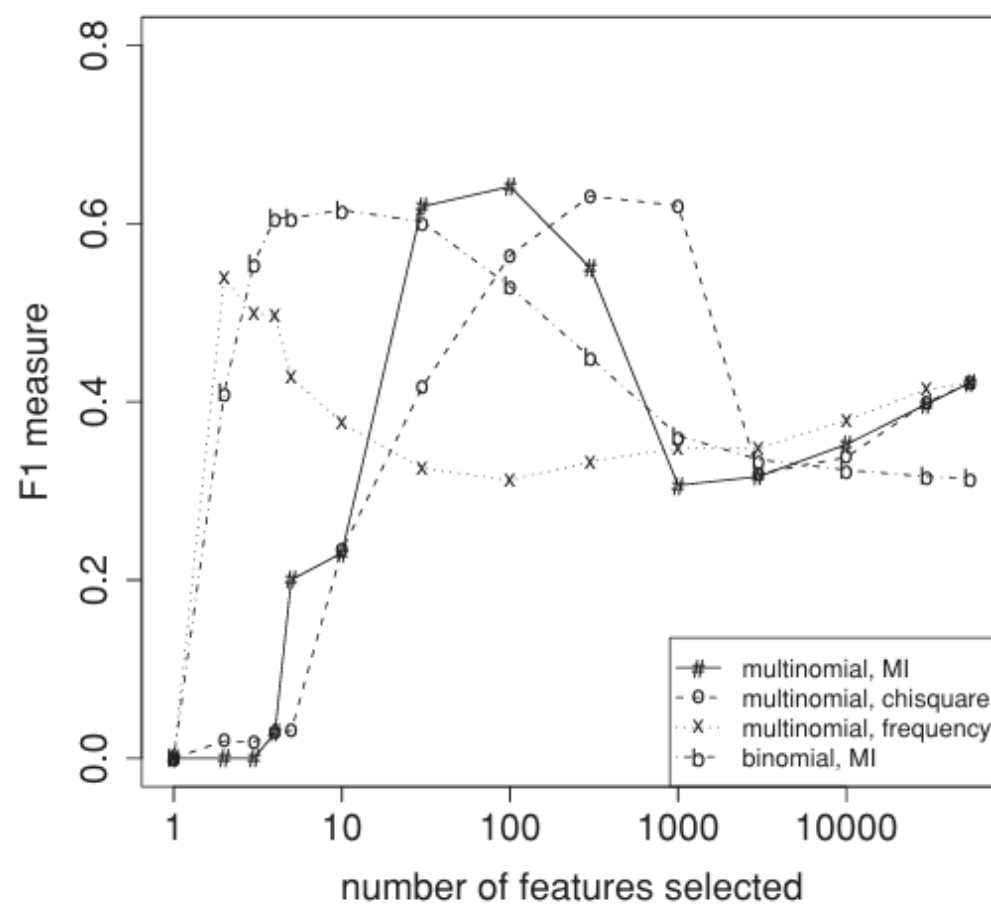
UK	China	poultry	coffee	elections	sports
london 0.1925	china 0.0997	poultry 0.0013	coffee 0.0111	election 0.0519	soccer 0.0681
uk 0.0755	chinese 0.0523	meat 0.0008	bags 0.0042	elections 0.0342	cup 0.0515
british 0.0596	beijing 0.0444	chicken 0.0006	growers 0.0025	polls 0.0339	match 0.0441

توجه کنید که کلمات با بالاترین امتیاز MI برای هر کلاس، کاملاً با آن کلاس مرتبط هستند: "london" برای اخبار بریتانیا، "beijing" برای اخبار چین، و "soccer" برای اخبار ورزشی.

📊 کمتر بیشتر است: چگونه انتخاب ویژگی دقت را بهبود می‌بخشد؟

اطلاعات متقابل میزان اطلاعاتی را اندازه‌گیری می‌کند که یک واژه درباره یک کلاس دارد. اگر توزیع یک واژه در کلاس مشابه کل مجموعه باشد، $I(U; C) = 0$ است. MI به حداکثر مقدار خود می‌رسد اگر واژه یک شاخص کامل برای عضویت در کلاس باشد.

شگفت‌انگیز است که استفاده از زیرمجموعه‌ای کوچک از ویژگی‌ها می‌تواند دقت طبقه‌بندی را بهبود بخشد! شکل 13.8 تأثیر اندازه مجموعه ویژگی بر دقت را برای مدل‌های چندجمله‌ای و برنولی نشان می‌دهد. انتخاب ویژگی با استفاده از MI، $F1$ را برای مدل چندجمله‌ای حدود 0.1 و برای مدل برنولی بیش از 0.2 افزایش می‌دهد.



► **Figure 13.8** Effect of feature set size on accuracy for multinomial and Bernoulli models.

برای مدل برنولی، F1 در زود هنگام، با ده ویژگی منتخب، به اوج می‌رسد. این نشان می‌دهد که برای مدل برنولی، استفاده از تعداد کمی از ویژگی‌های بسیار قوی، بهتر از استفاده از تعداد زیادی ویژگی با قدرت متغیر است.

کاربرد واقعی: در سیستم طبقه‌بندی نظرات مشتریان آمازون، استفاده از تنها 20 کلمه با بالاترین امتیاز MI برای هر دسته (مثلاً "عالی"، "خوب"، "بد"، "ضعیف" برای نظرات مثبت و منفی) می‌تواند دقت طبقه‌بندی را به اندازه استفاده از تمام کلمات در نظر بهبود بخشد، در حالی که سرعت پردازش را به شدت افزایش می‌دهد.

In [4]: *# پیاده‌سازی انتخاب ویژگی با استفاده از اطلاعات متقابل (Mutual Information)*

```
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import f1_score

def calculate_mutual_information(documents, labels, term, target_class):
    """
    محاسبه اطلاعات متقابل بین یک واژه و یک کلاس

    توضیح داده شد MD cell را پیاده‌سازی می‌کند که در MI این تابع فرمول
    میزان اطلاعاتی را اندازه‌گیری می‌کند که وجود یا عدم وجود یک واژه به تصمیم طبقه‌بندی صحیح کمک می‌کند MI

    پارامترها:
    documents: لیستی از اسناد، هر سند یک لیست از کلمات است
    labels: لیستی از برچسب‌های مربوط به هر سند
    term: واژه مورد نظر
    target_class: کلاس مورد نظر
    """
    N = len(documents) # تعداد کل اسناد

    # شمارش مقادیر N11, N10, N01, N00
    N11 = N10 = N01 = N00 = 0

    for doc, label in zip(documents, labels):
        term_in_doc = 1 if term in doc else 0
        is_target_class = 1 if label == target_class else 0

        if term_in_doc == 1 and is_target_class == 1:
            N11 += 1
        elif term_in_doc == 1 and is_target_class == 0:
            N10 += 1
        elif term_in_doc == 0 and is_target_class == 1:
            N01 += 1
        else: # term_in_doc == 0 and is_target_class == 0
            N00 += 1

    # محاسبه مقادیر جانبی
    N1_ = N10 + N11 # تعداد اسناد شامل واژه
    N0_ = N00 + N01 # تعداد اسناد بدون واژه
    N_1 = N01 + N11 # تعداد اسناد در کلاس هدف
    N_0 = N00 + N10 # تعداد اسناد در کلاس غیرهدف
```

```
# محاسبه اطلاعات متقابل با استفاده از فرمول
mi = 0.0
if N11 > 0:
    mi += (N11 / N) * np.log2((N * N11) / (N1_ * N_1))
if N01 > 0:
    mi += (N01 / N) * np.log2((N * N01) / (N0_ * N_1))
if N10 > 0:
    mi += (N10 / N) * np.log2((N * N10) / (N1_ * N_0))
if N00 > 0:
    mi += (N00 / N) * np.log2((N * N00) / (N0_ * N_0))

return mi

def select_features_mi(documents, labels, k, target_class):
    """
    ویژگی برتر با استفاده از اطلاعات متقابل  $k$  انتخاب

    واژه با بالاترین امتیاز را انتخاب می‌کند  $k$  را برای همه واژه‌ها محاسبه کرده و MI این تابع
    این همان الگوریتمی است که در شکل 13.6 کتاب توضیح داده شد.
    """
    # استخراج واژگان
    vocab = set()
    for doc in documents:
        vocab.update(doc)

    # محاسبه اطلاعات متقابل برای هر واژه
    term_scores = {}
    for term in vocab:
        mi = calculate_mutual_information(documents, labels, term, target_class)
        term_scores[term] = mi

    # واژه با بالاترین امتیاز  $k$  انتخاب
    selected_features = sorted(term_scores.items(), key=lambda x: x[1], reverse=True)[:k]
    return [term for term, score in selected_features], [score for term, score in selected_features]

def create_document_representation(documents, selected_features):
    """
    ایجاد نمایش سند بر اساس ویژگی‌های منتخب
    برای مدل چندجمله‌ای: تعداد تکرار هر ویژگی
    برای مدل برنولی: وجود یا عدم وجود هر ویژگی
    """
    # نمایش برای مدل چندجمله‌ای
    multinomial_rep = []
    for doc in documents:
        doc_rep = [doc.count(term) for term in selected_features]
        multinomial_rep.append(doc_rep)

    # نمایش برای مدل برنولی
    bernoulli_rep = []
    for doc in documents:
        doc_rep = [1 if term in doc else 0 for term in selected_features]
        bernoulli_rep.append(doc_rep)

    return np.array(multinomial_rep), np.array(bernoulli_rep)

def evaluate_feature_selection(documents, labels, target_class, feature_counts):
    """
    ارزیابی تأثیر انتخاب ویژگی بر دقت طبقه‌بندی

    این تابع دقت مدل‌های چندجمله‌ای و برنولی را برای تعداد مختلفی از ویژگی‌ها محاسبه می‌کند.
    نتایج نشان می‌دهد چگونه انتخاب ویژگی می‌تواند دقت را بهبود بخشد، همانطور که در شکل 13.8 کتاب نشان داده شد.
    """
    # تقسیم داده‌ها به آموزش و آزمون
    X_train_docs, X_test_docs, y_train, y_test = train_test_split(
        documents, labels, test_size=0.3, random_state=42
    )

    # تبدیل برجسب‌ها به باینری (کلاس هدف در مقابل بقیه)
    y_train_binary = np.array([1 if label == target_class else 0 for label in y_train])
    y_test_binary = np.array([1 if label == target_class else 0 for label in y_test])

    # انتخاب ویژگی‌ها بر اساس داده‌های آموزشی
    vocab = set()
    for doc in X_train_docs:
        vocab.update(doc)

    # برای همه واژه‌ها MI محاسبه امتیاز
    term_scores = {}
    for term in vocab:
        mi = calculate_mutual_information(X_train_docs, y_train, term, target_class)
        term_scores[term] = mi

    # MI مرتب‌سازی واژه‌ها بر اساس امتیاز
    sorted_terms = sorted(term_scores.items(), key=lambda x: x[1], reverse=True)
    all_terms = [term for term, score in sorted_terms]

    # ارزیابی برای تعداد مختلفی از ویژگی‌ها
    multinomial_scores = []
    bernoulli_scores = []

    for k in feature_counts:
        # ویژگی برتر  $k$  انتخاب
        selected_features = all_terms[:k]

        # ایجاد نمایش سند برای مدل‌ها
        X_train_multi, X_train_bern = create_document_representation(X_train_docs, selected_features)
        X_test_multi, X_test_bern = create_document_representation(X_test_docs, selected_features)

        # آموزش و ارزیابی مدل چندجمله‌ای
        clf_multi = MultinomialNB()
        clf_multi.fit(X_train_multi, y_train_binary)
        y_pred_multi = clf_multi.predict(X_test_multi)
        f1_multi = f1_score(y_test_binary, y_pred_multi)
```

```
multinomial_scores.append(f1_multi)

# آموزش و ارزیابی مدل برنولی
clf_bern = BernoulliNB()
clf_bern.fit(X_train_bern, y_train_binary)
y_pred_bern = clf_bern.predict(X_test_bern)
f1_bern = f1_score(y_test_binary, y_pred_bern)
bernoulli_scores.append(f1_bern)

return multinomial_scores, bernoulli_scores

# ایجاد مجموعه داده نمونه (شبيه به مثال کتاب)
np.random.seed(42)

# کلاس‌ها
classes = ['poultry', 'coffee', 'sports', 'china', 'uk', 'elections']

# واژگان مرتبط با هر کلاس
class_vocab = {
    'poultry': ['poultry', 'chicken', 'meat', 'avian', 'bird', 'farm', 'egg', 'broiler', 'inspection', 'disease'],
    'coffee': ['coffee', 'bean', 'roast', 'brew', 'colombia', 'brazil', 'export', 'crop', 'harvest', 'arabica'],
    'sports': ['soccer', 'game', 'match', 'team', 'player', 'score', 'win', 'league', 'cup', 'championship'],
    'china': ['china', 'chinese', 'beijing', 'yuan', 'shanghai', 'hong', 'kong', 'xinhua', 'province', 'taiwan'],
    'uk': ['london', 'uk', 'british', 'britain', 'england', 'pound', 'pence', 'stg', 'plc', 'english'],
    'elections': ['election', 'vote', 'candidate', 'campaign', 'poll', 'party', 'democratic', 'republican', 'president', 'ballot']
}

# واژگان عمومی
general_vocab = ['report', 'said', 'year', 'today', 'new', 'government', 'market', 'company', 'people', 'world']

# ایجاد اسناد
documents = []
labels = []
num_docs_per_class = 100

for cls in classes:
    for i in range(num_docs_per_class):
        # هر سند شامل 5-15 واژه است
        doc_length = np.random.randint(5, 16)
        doc = []

        # اضافه کردن 2-4 واژه از کلاس
        num_class_terms = np.random.randint(2, 5)
        class_terms = np.random.choice(class_vocab[cls], num_class_terms, replace=False)
        doc.extend(class_terms)

        # اضافه کردن واژگان عمومی
        num_general_terms = doc_length - num_class_terms
        general_terms = np.random.choice(general_vocab, num_general_terms, replace=True)
        doc.extend(general_terms)

        # اضافه کردن برخی واژگان نویز (با احتمال کم)
        if np.random.random() < 0.1:
            noise_term = f"noise_term_{np.random.randint(100)}"
            doc.append(noise_term)

        documents.append(doc)
        labels.append(cls)

# export و واژه poultry محاسبه اطلاعات متقابل برای کلاس
mi_export = calculate_mutual_information(documents, labels, "export", "poultry")
print(f"export و کلاس 'poultry': {mi_export:.6f}")

# poultry انتخاب 10 ویژگی برتر برای کلاس
selected_features, scores = select_features_mi(documents, labels, 10, "poultry")
print("poultry ویژگی‌های منتخب برای کلاس:")
for feature, score in zip(selected_features, scores):
    print(f"{feature}: {score:.6f}")

# ارزیابی تأثیر انتخاب ویژگی بر دقت طبقه‌بندی
feature_counts = [1, 5, 10, 20, 50, 100]
multinomial_scores, bernoulli_scores = evaluate_feature_selection(documents, labels, "poultry", feature_counts)

# نمایش نتایج
plt.figure(figsize=(10, 6))
plt.plot(feature_counts, multinomial_scores, 'o-', label='Multinomial NB')
plt.plot(feature_counts, bernoulli_scores, 'x-', label='Bernoulli NB')
plt.xscale('log')
plt.xlabel('Number of Features')
plt.ylabel('F1 Score')
plt.title('Effect of Feature Selection on Classification Accuracy')
plt.legend()
plt.grid(True)
plt.show()

# نمایش نتایج عددی
print("\nنتایج ارزیابی:")
print(f"(برنولی) \tF1 (چندجمله‌ای) \t\tتعداد ویژگی")
for i, k in enumerate(feature_counts):
    print(f"{k} \t\t\t{multinomial_scores[i]:.4f} \t\t\t{bernoulli_scores[i]:.4f}")
```

poultry و کلاس 'export' اطلاعات متقابل برای واژه

poultry ویژگی‌های منتخب برای کلاس:

avian: 0.170331

chicken: 0.165115

inspection: 0.149661

bird: 0.149661

meat: 0.144573

egg: 0.144573

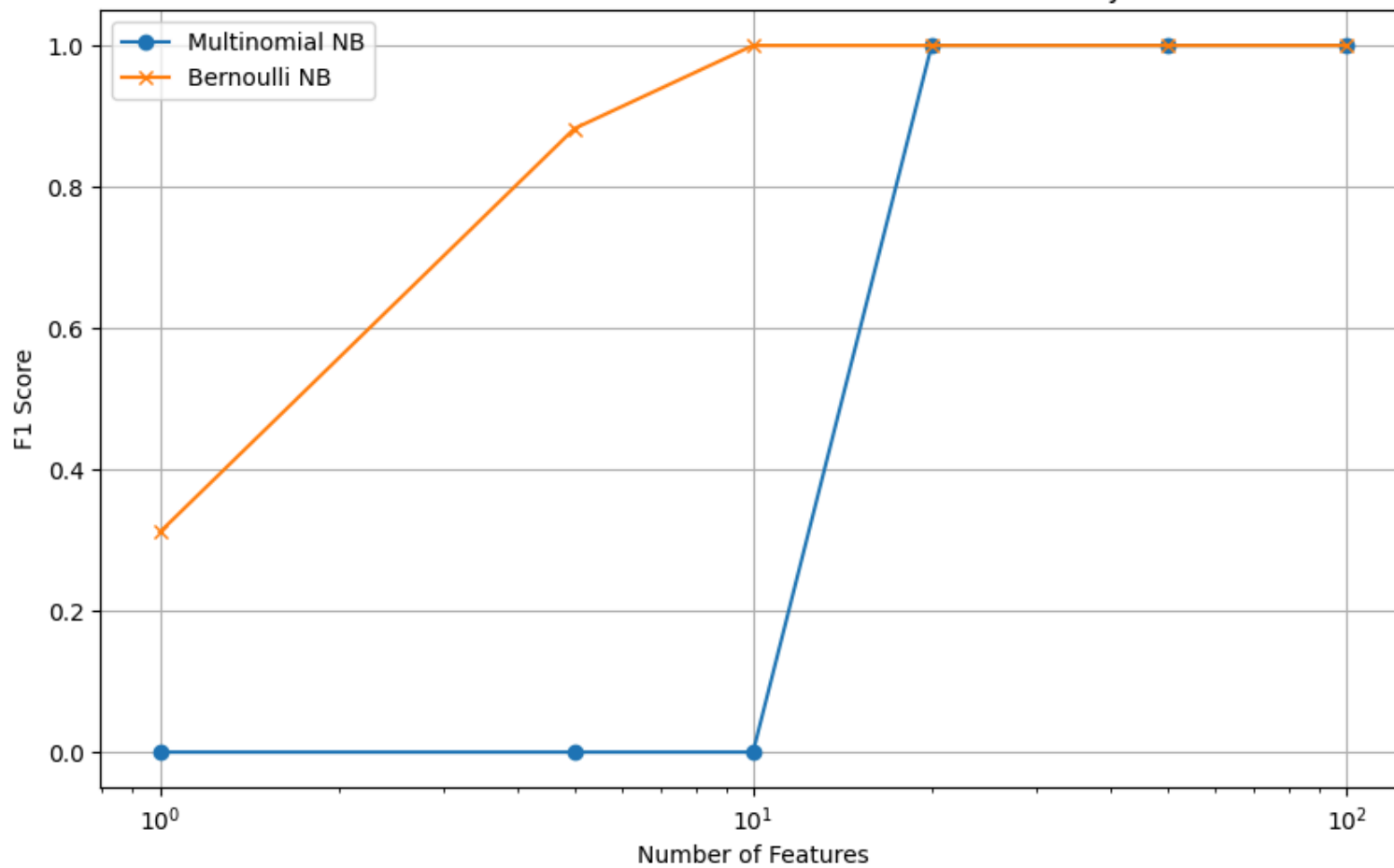
poultry: 0.129490

disease: 0.129490

farm: 0.129490

broiler: 0.119581

Effect of Feature Selection on Classification Accuracy



نتایج ارزیابی:

تعداد ویژگی

F1 (چندجمله‌ای)

F1 (برنولی)

1

0.0000

0.3111

5

0.0000

0.8824

10

0.0000

1.0000

20

1.0000

1.0000

50

1.0000

1.0000

100

1.0000

1.0000

13.5.2 آزمون χ^2 : چگونه کلمات مهم را از کلمات تصادفی تشخیص دهیم؟

🛒 آمازون و تشخیص محصولات محبوب: آیا "خرید" واقعاً نشان‌دهنده محبوبیت است؟

تصور کنید در آمازون کار می‌کنید و باید سیستم طراحی کنید که محصولات را به دسته‌بندی‌های مختلفی مانند "محبوب"، "متوسط" یا "نامحسوب" تقسیم کند. کلمه "خرید" در نظرات محصولات بسیار زیاد تکرار می‌شود، اما آیا این کلمه واقعاً به ما کمک می‌کند تا تشخیص دهیم یک محصول محبوب است یا خیر؟

آزمون χ^2 به ما کمک می‌کند تا بفهمیم آیا یک کلمه واقعاً با یک کلاس مرتبط است یا فقط به صورت تصادفی در اسناد ظاهر شده است. این آزمون فراوانی مشاهده شده یک کلمه در یک کلاس را با فراوانی مورد انتظار تحت فرض استقلال مقایسه می‌کند.

روش دیگر محبوب برای انتخاب ویژگی، استفاده از آزمون χ^2 است. در آمار، آزمون χ^2 برای بررسی استقلال دو رویداد به کار می‌رود. در انتخاب ویژگی، دو رویداد عبارتند از وقوع واژه و وقوع کلاس. ما واژه‌ها را بر اساس مقدار زیر رتبه‌بندی می‌کنیم:

$$\chi^2(D, t, c) = \sum_{e_t \in \{0,1\}} \sum_{e_c \in \{0,1\}} \frac{(N_{e_t e_c} - E_{e_t e_c})^2}{E_{e_t e_c}}$$

که در آن N فراوانی مشاهده شده و E فراوانی مورد انتظار است. برای مثال، E_{11} فراوانی مورد انتظار وقوع همزمان واژه t و کلاس c است با فرض استقلال آن‌ها.

12 از تئوری تا عمل: چگونه χ^2 را محاسبه کنیم؟

مثال 13.4: بیایید به مثال اخبار کشاورزی برگردیم و χ^2 را برای کلاس poultry (طیور) و واژه export (صادرات) محاسبه کنیم. ابتدا E_{11} را برای داده‌های مثال 13.3 محاسبه می‌کنیم:

$$E_{11} = N \times P(t) \times P(c) = N \times (49+141)/N \times (49+27652)/N \approx 6.6$$

با محاسبه سایر مقادیر E و جایگذاری در معادله، به دست می‌آوریم: $\chi^2(D, t, c) \approx 284$

معنای آماری: چگونه χ^2 به ما کمک می‌کند؟

χ^2 معیاری از انحراف فراوانی‌های مورد انتظار (E) و مشاهده شده (N) از یکدیگر است. مقدار بالای χ^2 نشان می‌دهد که فرض استقلال، که به معنای شباهت فراوانی‌های مورد انتظار و مشاهده شده است، نادرست است.

در مثال ما، $\chi^2 \approx 284 > 10.83$ ، بنابراین می‌توانیم فرض استقلال poultry و export را با احتمال خطای 0.001 رد کنیم. این به ما می‌گوید که واژه "export" و کلاس "poultry" به هم وابسته هستند و این وابستگی تصادفی نیست.

کاربرد واقعی: در سیستم طبقه‌بندی نظرات مشتریان فیسبوک، کلمه "عالی" ممکن است در نظرات مثبت و منفی ظاهر شود. آزمون χ^2 به ما کمک می‌کند تا بفهمیم آیا این کلمه واقعاً با نظرات مثبت مرتبط است یا فقط به صورت تصادفی در هر دو نوع نظر ظاهر می‌شود.

یک روش ساده‌تر برای محاسبه χ^2 به صورت زیر است:

$$\chi^2(D, t, c) = \frac{(N_{11} + N_{10} + N_{01} + N_{00}) \times (N_{11}N_{00} - N_{10}N_{01})^2}{(N_{11} + N_{01}) \times (N_{11} + N_{10}) \times (N_{10} + N_{00}) \times (N_{01} + N_{00})}$$

جدول 13.6 مقادیر بحرانی توزیع χ^2 با یک درجه آزادی را نشان می‌دهد:

χ^2 مقدار بحرانی	p
2.71	0.1
3.84	0.05
6.63	0.01
10.83	0.001

⚠ محدودیت‌های آماری: آیا χ^2 همیشه دقیق است؟

از دیدگاه آماری، انتخاب ویژگی با χ^2 مشکل‌ساز است. برای یک آزمون با یک درجه آزادی، باید از اصلاح Yates استفاده شود که رسیدن به اهمیت آماری را دشوارتر می‌کند. همچنین، هرگاه یک آزمون آماری چندین بار استفاده شود، احتمال خطا افزایش می‌یابد.

با این حال، در طبقه‌بندی متن، به ندرت اهمیت دارد که چند واژه اضافی به مجموعه ویژگی اضافه یا از آن حذف شوند. مهم‌تر، اهمیت نسبی ویژگی‌ها است. تا زمانی که انتخاب ویژگی با χ^2 فقط ویژگی‌ها را بر اساس کاربردی رتبه‌بندی کند و برای بیان بیانیه‌هایی درباره وابستگی آماری متغیرها استفاده نشود، نباید بیش از حد نگران عدم پیروی دقیق از نظریه آماری باشیم.

مثال واقعی: در سیستم طبقه‌بندی اخبار گوگل، حتی اگر χ^2 برای برخی کلمات به دلیل محدودیت‌های آماری دقیق نباشد، سیستم همچنان می‌تواند با استفاده از رتبه‌بندی نسبی کلمات، اخبار را با دقت بالایی دسته‌بندی کند. در عمل، این کافی است که کلمات مرتبط‌تر بالاترین رتبه را داشته باشند، حتی اگر مقادیر دقیق χ^2 از نظر آماری کامل نباشند.

```
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import f1_score

def calculate_chi2(documents, labels, term, target_class):
    """
    برای یک واژه و یک کلاس  $\chi^2$  محاسبه مقدار

    توضیح داده شد MD cell را پیاده‌سازی می‌کند که در  $\chi^2$  این تابع فرمول
    میزان انحراف فراوانی مشاهده شده از فراوانی مورد انتظار تحت فرض استقلال را اندازه‌گیری می‌کند  $\chi^2$ .

    پارامترها:
    documents: لیستی از اسناد، هر سند یک لیست از کلمات است
    labels: لیستی از برچسب‌های مربوط به هر سند
    term: واژه مورد نظر
    target_class: کلاس مورد نظر
    """
    N = len(documents) # تعداد کل اسناد

    # شمارش مقادیر N11, N10, N01, N00
    N11 = N10 = N01 = N00 = 0

    for doc, label in zip(documents, labels):
        term_in_doc = 1 if term in doc else 0
        is_target_class = 1 if label == target_class else 0

        if term_in_doc == 1 and is_target_class == 1:
            N11 += 1
        elif term_in_doc == 1 and is_target_class == 0:
            N10 += 1
        elif term_in_doc == 0 and is_target_class == 1:
            N01 += 1
        else: # term_in_doc == 0 and is_target_class == 0
            N00 += 1

    # با استفاده از فرمول ساده‌شده (معادل فرمول اصلی)  $\chi^2$  محاسبه
    numerator = (N11 + N10 + N01 + N00) * ((N11 * N00) - (N10 * N01))**2
    denominator = (N11 + N01) * (N11 + N10) * (N10 + N00) * (N01 + N00)

    if denominator == 0:
        return 0

    chi2 = numerator / denominator
    return chi2

def calculate_mutual_information(documents, labels, term, target_class):
    """
     $\chi^2$  محاسبه اطلاعات متقابل بین یک واژه و یک کلاس برای مقایسه با
    """
    N = len(documents)

    # شمارش مقادیر N11, N10, N01, N00
    N11 = N10 = N01 = N00 = 0

    for doc, label in zip(documents, labels):
        term_in_doc = 1 if term in doc else 0
        is_target_class = 1 if label == target_class else 0

        if term_in_doc == 1 and is_target_class == 1:
            N11 += 1
        elif term_in_doc == 1 and is_target_class == 0:
            N10 += 1
        elif term_in_doc == 0 and is_target_class == 1:
            N01 += 1
        else: # term_in_doc == 0 and is_target_class == 0
            N00 += 1

    # محاسبه مقادیر جانبی
    N1_ = N10 + N11 # تعداد اسناد شامل واژه
    N0_ = N00 + N01 # تعداد اسناد بدون واژه
    N_1 = N01 + N11 # تعداد اسناد در کلاس هدف
    N_0 = N00 + N10 # تعداد اسناد در کلاس غیرهدف

    # محاسبه اطلاعات متقابل
    mi = 0.0
    if N11 > 0:
        mi += (N11 / N) * np.log2((N * N11) / (N1_ * N_1))
    if N01 > 0:
        mi += (N01 / N) * np.log2((N * N01) / (N0_ * N_1))
    if N10 > 0:
        mi += (N10 / N) * np.log2((N * N10) / (N1_ * N_0))
    if N00 > 0:
        mi += (N00 / N) * np.log2((N * N00) / (N0_ * N_0))

    return mi

def select_features_chi2(documents, labels, k, target_class):
    """
     $\chi^2$  ویژگی برتر با استفاده از آزمون k انتخاب

    واژه با بالاترین امتیاز را انتخاب می‌کند k را برای همه واژه‌ها محاسبه کرده و  $\chi^2$  این تابع
    """
    # استخراج واژگان
    vocab = set()
    for doc in documents:
        vocab.update(doc)

    # برای هر واژه  $\chi^2$  محاسبه
    term_scores = {}
    for term in vocab:
```

```
chi2 = calculate_chi2(documents, labels, term, target_class)
term_scores[term] = chi2

# واژه با بالاترین امتیاز k انتخاب
selected_features = sorted(term_scores.items(), key=lambda x: x[1], reverse=True)[:k]
return [term for term, score in selected_features], [score for term, score in selected_features]

def select_features_mi(documents, labels, k, target_class):
    """
    ویژگی برتر با استفاده از اطلاعات متقابل برای مقایسه k انتخاب
    """
    # استخراج واژگان
    vocab = set()
    for doc in documents:
        vocab.update(doc)

    # محاسبه اطلاعات متقابل برای هر واژه
    term_scores = {}
    for term in vocab:
        mi = calculate_mutual_information(documents, labels, term, target_class)
        term_scores[term] = mi

    # واژه با بالاترین امتیاز k انتخاب
    selected_features = sorted(term_scores.items(), key=lambda x: x[1], reverse=True)[:k]
    return [term for term, score in selected_features], [score for term, score in selected_features]

def create_document_representation(documents, selected_features):
    """
    ایجاد نمایش سند بر اساس ویژگی‌های منتخب
    برای مدل چندجمله‌ای: تعداد تکرار هر ویژگی
    برای مدل برنولی: وجود یا عدم وجود هر ویژگی
    """
    # نمایش برای مدل چندجمله‌ای
    multinomial_rep = []
    for doc in documents:
        doc_rep = [doc.count(term) for term in selected_features]
        multinomial_rep.append(doc_rep)

    # نمایش برای مدل برنولی
    bernoulli_rep = []
    for doc in documents:
        doc_rep = [1 if term in doc else 0 for term in selected_features]
        bernoulli_rep.append(doc_rep)

    return np.array(multinomial_rep), np.array(bernoulli_rep)

def compare_feature_selection_methods(documents, labels, target_class, k=10):
    """
    MI و  $\chi^2$  مقایسه روش‌های انتخاب ویژگی

    این تابع نشان می‌دهد که چگونه دو روش مختلف می‌توانند ویژگی‌های متفاوتی را انتخاب کنند.
    """
    #  $\chi^2$  انتخاب ویژگی با
    chi2_features, chi2_scores = select_features_chi2(documents, labels, k, target_class)

    # MI انتخاب ویژگی با
    mi_features, mi_scores = select_features_mi(documents, labels, k, target_class)

    # نمایش نتایج
    print(f"Top {k} features selected by  $\chi^2$  for class '{target_class}':")
    for feature, score in zip(chi2_features, chi2_scores):
        print(f"{feature}: {score:.2f}")

    print(f"\nTop {k} features selected by MI for class '{target_class}':")
    for feature, score in zip(mi_features, mi_scores):
        print(f"{feature}: {score:.6f}")

    # مقایسه ویژگی‌های مشترک
    common_features = set(chi2_features) & set(mi_features)
    print(f"\nCommon features selected by both methods: {list(common_features)}")

    return chi2_features, mi_features

def evaluate_feature_selection_methods(documents, labels, target_class, feature_counts):
    """
    ارزیابی تأثیر روش‌های مختلف انتخاب ویژگی بر دقت طبقه‌بندی

    این تابع دقت مدل‌های چندجمله‌ای و برنولی را برای تعداد مختلفی از ویژگی‌ها محاسبه می‌کند.
    نتایج نشان می‌دهد چگونه روش‌های مختلف انتخاب ویژگی می‌توانند دقت را بهبود بخشند.
    """
    # تقسیم داده‌ها به آموزش و آزمون
    X_train_docs, X_test_docs, y_train, y_test = train_test_split(
        documents, labels, test_size=0.3, random_state=42
    )

    # تبدیل برجسب‌ها به باینری (کلاس هدف در مقابل بقیه)
    y_train_binary = np.array([1 if label == target_class else 0 for label in y_train])
    y_test_binary = np.array([1 if label == target_class else 0 for label in y_test])

    # انتخاب ویژگی‌ها بر اساس داده‌های آموزشی
    vocab = set()
    for doc in X_train_docs:
        vocab.update(doc)

    # برای همه واژه‌ها  $\chi^2$  محاسبه امتیاز
    chi2_scores = {}
    for term in vocab:
        chi2_scores[term] = calculate_chi2(X_train_docs, y_train, term, target_class)

    # برای همه واژه‌ها MI محاسبه امتیاز
    mi_scores = {}
    for term in vocab:
        mi_scores[term] = calculate_mutual_information(X_train_docs, y_train, term, target_class)
```

```
# مرتب‌سازی واژه‌ها بر اساس امتیازها
chi2_sorted_terms = sorted(chi2_scores.items(), key=lambda x: x[1], reverse=True)
mi_sorted_terms = sorted(mi_scores.items(), key=lambda x: x[1], reverse=True)

chi2_terms = [term for term, score in chi2_sorted_terms]
mi_terms = [term for term, score in mi_sorted_terms]

# ارزیابی برای تعداد مختلفی از ویژگی‌ها
chi2_multinomial_scores = []
chi2_bernoulli_scores = []
mi_multinomial_scores = []
mi_bernoulli_scores = []

for k in feature_counts:
    # ویژگی برتر با k انتخاب
    chi2_selected_features = chi2_terms[:k]

    # ایجاد نمایش سند برای مدل‌ها
    X_train_multi, X_train_bern = create_document_representation(X_train_docs, chi2_selected_features)
    X_test_multi, X_test_bern = create_document_representation(X_test_docs, chi2_selected_features)

    # آموزش و ارزیابی مدل چندجمله‌ای با  $\chi^2$ 
    clf_multi = MultinomialNB()
    clf_multi.fit(X_train_multi, y_train_binary)
    y_pred_multi = clf_multi.predict(X_test_multi)
    f1_multi = f1_score(y_test_binary, y_pred_multi)
    chi2_multinomial_scores.append(f1_multi)

    # آموزش و ارزیابی مدل برنولی با  $\chi^2$ 
    clf_bern = BernoulliNB()
    clf_bern.fit(X_train_bern, y_train_binary)
    y_pred_bern = clf_bern.predict(X_test_bern)
    f1_bern = f1_score(y_test_binary, y_pred_bern)
    chi2_bernoulli_scores.append(f1_bern)

    # MI ویژگی برتر با k انتخاب
    mi_selected_features = mi_terms[:k]

    # ایجاد نمایش سند برای مدل‌ها
    X_train_multi, X_train_bern = create_document_representation(X_train_docs, mi_selected_features)
    X_test_multi, X_test_bern = create_document_representation(X_test_docs, mi_selected_features)

    # MI آموزش و ارزیابی مدل چندجمله‌ای با
    clf_multi = MultinomialNB()
    clf_multi.fit(X_train_multi, y_train_binary)
    y_pred_multi = clf_multi.predict(X_test_multi)
    f1_multi = f1_score(y_test_binary, y_pred_multi)
    mi_multinomial_scores.append(f1_multi)

    # MI آموزش و ارزیابی مدل برنولی با
    clf_bern = BernoulliNB()
    clf_bern.fit(X_train_bern, y_train_binary)
    y_pred_bern = clf_bern.predict(X_test_bern)
    f1_bern = f1_score(y_test_binary, y_pred_bern)
    mi_bernoulli_scores.append(f1_bern)

    return (chi2_multinomial_scores, chi2_bernoulli_scores,
            mi_multinomial_scores, mi_bernoulli_scores)

# ایجاد مجموعه داده نمونه (شبیه به مثال کتاب)
np.random.seed(42)

# کلاس‌ها
classes = ['poultry', 'coffee', 'sports', 'china', 'uk', 'elections']

# واژگان مرتبط با هر کلاس
class_vocab = {
    'poultry': ['poultry', 'chicken', 'meat', 'avian', 'bird', 'farm', 'egg', 'broiler', 'inspection', 'disease'],
    'coffee': ['coffee', 'bean', 'roast', 'brew', 'colombia', 'brazil', 'export', 'crop', 'harvest', 'arabica'],
    'sports': ['soccer', 'game', 'match', 'team', 'player', 'score', 'win', 'league', 'cup', 'championship'],
    'china': ['china', 'chinese', 'beijing', 'yuan', 'shanghai', 'hong', 'kong', 'xinhua', 'province', 'taiwan'],
    'uk': ['london', 'uk', 'british', 'britain', 'england', 'pound', 'pence', 'stg', 'plc', 'english'],
    'elections': ['election', 'vote', 'candidate', 'campaign', 'poll', 'party', 'democratic', 'republican', 'president', 'ballot']
}

# واژگان عمومی
general_vocab = ['report', 'said', 'year', 'today', 'new', 'government', 'market', 'company', 'people', 'world']

# ایجاد اسناد
documents = []
labels = []
num_docs_per_class = 100

for cls in classes:
    for i in range(num_docs_per_class):
        # هر سند شامل 5-15 واژه است
        doc_length = np.random.randint(5, 16)
        doc = []

        # اضافه کردن 2-4 واژه از کلاس
        num_class_terms = np.random.randint(2, 5)
        class_terms = np.random.choice(class_vocab[cls], num_class_terms, replace=False)
        doc.extend(class_terms)

        # اضافه کردن واژگان عمومی
        num_general_terms = doc_length - num_class_terms
        general_terms = np.random.choice(general_vocab, num_general_terms, replace=True)
        doc.extend(general_terms)

        # اضافه کردن برخی واژگان نویز (با احتمال کم)
        if np.random.random() < 0.1:
            noise_term = f"noise_term_{np.random.randint(100)}"
```

```
doc.append(noise_term)

documents.append(doc)
labels.append(cls)

# export و واژه poultry برای کلاس  $\chi^2$  مثال از کتاب: محاسبه
chi2_export = calculate_chi2(documents, labels, "export", "poultry")
print(f"مقدار  $\chi^2$  برای واژه 'export' و کلاس 'poultry': {chi2_export:.2f}")

# MI و  $\chi^2$  مقایسه روش‌های انتخاب ویژگی
chi2_features, mi_features = compare_feature_selection_methods(documents, labels, "poultry", k=10)

# ارزیابی تأثیر روش‌های مختلف انتخاب ویژگی بر دقت طبقه‌بندی
feature_counts = [1, 5, 10, 20, 50, 100]
(chi2_multi_scores, chi2_bernoulli_scores,
 mi_multi_scores, mi_bernoulli_scores) = evaluate_feature_selection_methods(
    documents, labels, "poultry", feature_counts)

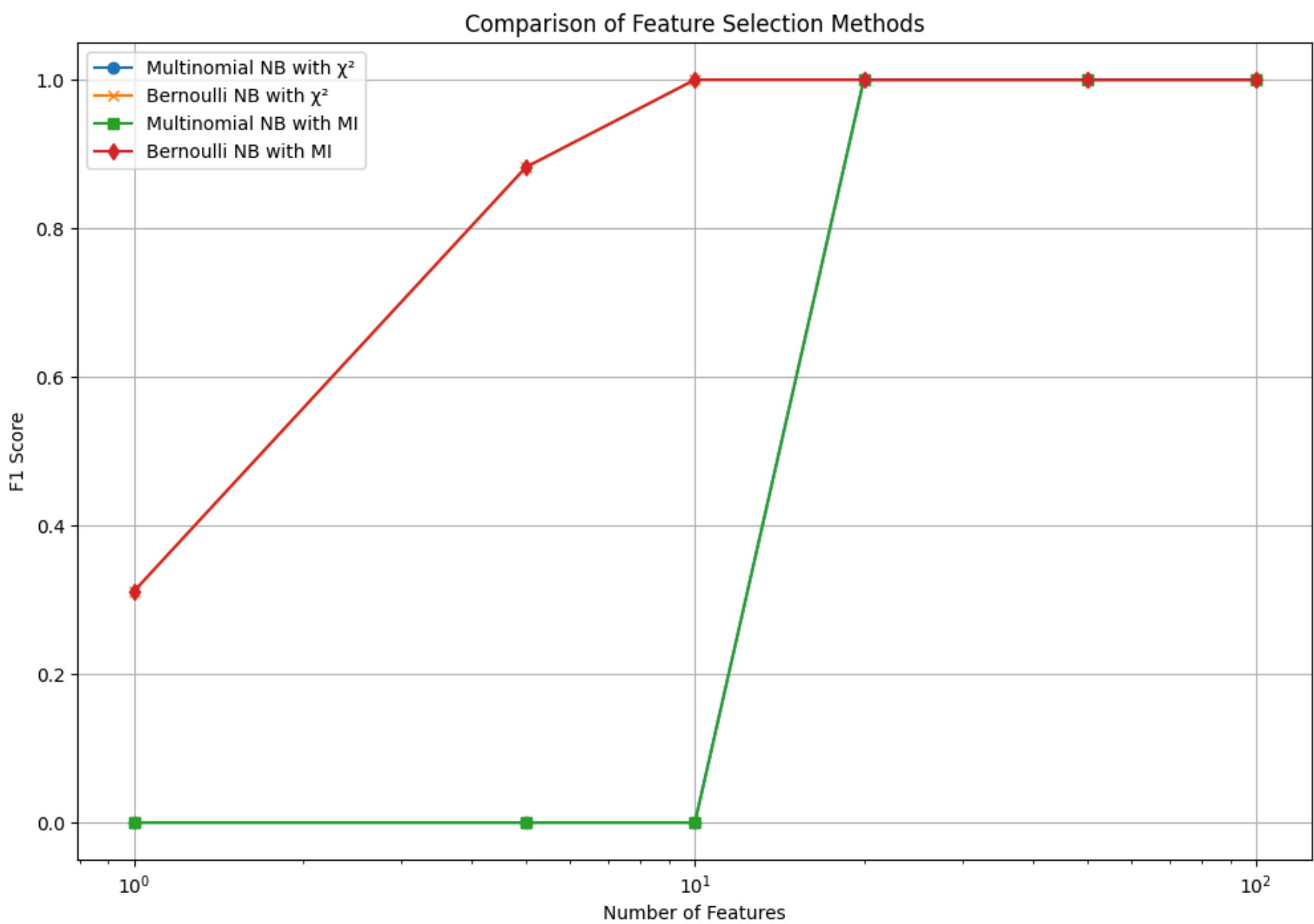
# نمایش نتایج
plt.figure(figsize=(12, 8))
plt.plot(feature_counts, chi2_multi_scores, 'o-', label='Multinomial NB with  $\chi^2$ ')
plt.plot(feature_counts, chi2_bernoulli_scores, 'x-', label='Bernoulli NB with  $\chi^2$ ')
plt.plot(feature_counts, mi_multi_scores, 's-', label='Multinomial NB with MI')
plt.plot(feature_counts, mi_bernoulli_scores, 'd-', label='Bernoulli NB with MI')
plt.xscale('log')
plt.xlabel('Number of Features')
plt.ylabel('F1 Score')
plt.title('Comparison of Feature Selection Methods')
plt.legend()
plt.grid(True)
plt.ylim(-0.05, 1.05)
plt.show()

# نمایش نتایج عددی
print("\nنتایج ارزیابی:")
print("MI (برنولی) \tF1 (چندجمله‌ای) \tF1 ( $\chi^2$  برنولی) \tF1 ( $\chi^2$  چندجمله‌ای) \tF1 (تعداد ویژگی)")
for i, k in enumerate(feature_counts):
    print(f"{k} \t\t{chi2_multi_scores[i]:.4f} \t\t{chi2_bernoulli_scores[i]:.4f} \t\t{mi_multi_scores[i]:.4f} \t\t{mi_bernoulli_scores[i]:.4f}")
```

مقدار χ^2 برای واژه 'export' و کلاس 'poultry': 5.44
Top 10 features selected by χ^2 for class 'poultry':
avian: 191.49
chicken: 185.84
bird: 169.01
inspection: 169.01
meat: 163.44
egg: 163.44
farm: 146.85
disease: 146.85
poultry: 146.85
broiler: 135.89

Top 10 features selected by MI for class 'poultry':
avian: 0.170331
chicken: 0.165115
bird: 0.149661
inspection: 0.149661
meat: 0.144573
egg: 0.144573
farm: 0.129490
disease: 0.129490
poultry: 0.129490
broiler: 0.119581

Common features selected by both methods: [np.str_('avian'), np.str_('farm'), np.str_('egg'), np.str_('meat'), np.str_('bird'), np.str_('chicken'), np.str_('disease'), np.str_('inspection'), np.str_('poultry'), np.str_('broiler')]



نتایج ارزیابی:

تعداد ویژگی	F1 (χ^2 چندجمله‌ای)	F1 (χ^2 برنولی)	F1 (MI چندجمله‌ای)	F1 (MI برنولی)
1	0.0000	0.3111	0.0000	0.3111
5	0.0000	0.8824	0.0000	0.8824
10	0.0000	1.0000	0.0000	1.0000
20	1.0000	1.0000	1.0000	1.0000
50	1.0000	1.0000	1.0000	1.0000
100	1.0000	1.0000	1.0000	1.0000

13.5.3 انتخاب ویژگی مبتنی بر فراوانی: ساده‌ترین روش، مؤثرترین؟ 📊

🎬 نتفلیکس و تحلیل نظرات کاربران: چگونه فیلم‌ها را دسته‌بندی کنیم؟

تصور کنید در تیم تحلیل داده‌های نتفلیکس کار می‌کنید و باید سیستم طراحی کنید که نظرات کاربران را به ژانرهای مختلف فیلم (اکشن، کمدی، درام، علمی-تخیلی) دسته‌بندی کند. یک رویکرد ساده این است که کلماتی را پیدا کنید که بیشترین تکرار را در نظرات هر ژانر دارند.

برای مثال، در نظرات فیلم‌های اکشن، کلماتی مانند "اکشن"، "هیجان‌انگیز"، "تعقیب" و "آتش" ممکن است بیشتر تکرار شوند. این روش ساده، انتخاب ویژگی مبتنی بر فراوانی نام دارد.

روش سوم برای انتخاب ویژگی، **انتخاب ویژگی مبتنی بر فراوانی** است، یعنی انتخاب واژه‌هایی که در یک کلاس بیشترین فراوانی را دارند. این ساده‌ترین روش انتخاب ویژگی است، اما همیشه مؤثرترین نیست. فراوانی می‌تواند به دو صورت تعریف شود:

- **فراوانی سندی** (document frequency): تعداد اسناد در کلاس c که شامل واژه t هستند (مناسب برای مدل برنولی)
- **فراوانی مجموعه** (collection frequency): تعداد توکن‌های t که در اسناد کلاس c رخ می‌دهند (مناسب برای مدل چندجمله‌ای)

⚠️ مزایا و معایب: چرا ساده‌ترین همیشه بهترین نیست؟

انتخاب ویژگی مبتنی بر فراوانی ممکن است برخی واژه‌های پرتکرار را انتخاب کند که اطلاعات خاصی درباره کلاس ندارند، مانند روزهای هفته (Monday, Tuesday, ...) که در متن‌های خبری در همه کلاس‌ها پرتکرار هستند.

مثال واقعی: در سیستم تحلیل نظرات آمازون، کلماتی مانند "محصول"، "خرید" و "ارسال" در نظرات همه دسته‌ها (کتاب، الکترونیک، لباس) پرتکرار هستند، اما هیچ اطلاعات خاصی درباره دسته محصول به ما نمی‌دهند. این کلمات "نویز" می‌توانند دقت طبقه‌بندی را کاهش دهند.

🏢 چه زمانی از این روش استفاده کنیم؟

وقتی هزاران ویژگی انتخاب می‌شوند، انتخاب ویژگی مبتنی بر فراوانی اغلب خوب عمل می‌کند. بنابراین، اگر دقت کمی زیربهبینه قابل قبول باشد، انتخاب ویژگی مبتنی بر فراوانی می‌تواند جایگزین خوبی برای روش‌های پیچیده‌تر باشد.

کاربرد واقعی: در سیستم تحلیل توییت‌ر، وقتی میلیون‌ها توئیت باید به سرعت دسته‌بندی شوند، استفاده از روش مبتنی بر فراوانی می‌تواند یک راه حل سریع و کارآمد باشد، حتی اگر دقت کمی کمتر از روش‌های پیچیده‌تر باشد.

🔍 مقایسه با روش‌های دیگر

با این حال، شکل 13.8 موردی است که انتخاب ویژگی مبتنی بر فراوانی بسیار بدتر از MI و χ^2 عمل می‌کند و نباید استفاده شود. این نشان می‌دهد که سادگی همیشه بهترین راه حل نیست.

مثال واقعی: در سیستم طبقه‌بندی مقالات علمی گوگل اسکالر، روش مبتنی بر فراوانی ممکن است کلمات عمومی مانند "تحقیق"، "نتایج" و "تحلیل" را انتخاب کند که در همه رشته‌ها پرتکرار هستند، در حالی که روش‌های MI و χ^2 می‌توانند کلمات تخصصی‌تری را شناسایی کنند که تمایز بهتری بین رشته‌ها ایجاد می‌کنند.

In [6]: *Frequency-based Feature Selection (پایه‌سازی انتخاب ویژگی مبتنی بر فراوانی)* #

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import f1_score

def calculate_document_frequency(documents, labels, target_class):
    """
    محاسبه فراوانی سندی برای هر واژه در کلاس هدف

    هستند t که شامل واژه c فراوانی سندی: تعداد اسناد در کلاس
    .این روش برای مدل برنولی مناسب است

    پارامترها:
    documents: (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    target_class: کلاسی که می‌خواهیم فراوانی واژه‌ها را برای آن محاسبه کنیم

    خروجی: دیکشنری که کلیدها واژه‌ها و مقادیر فراوانی سندی آن‌هاست
    """
    # فیلتر کردن اسناد مربوط به کلاس هدف
    target_docs = [doc for doc, label in zip(documents, labels) if label == target_class]

    # شمارش تعداد اسنادی که هر واژه در آن‌ها وجود دارد
    doc_freq = {}
    for doc in target_docs:
        unique_words = set(doc) # هر سند شمرش شود
        for word in unique_words:
            doc_freq[word] = doc_freq.get(word, 0) + 1

    return doc_freq

def calculate_collection_frequency(documents, labels, target_class):
    """
    محاسبه فراوانی مجموعه برای هر واژه در کلاس هدف

    رخ می‌دهند c که در اسناد کلاس t فراوانی مجموعه: تعداد کل توکن‌های
    .این روش برای مدل چندجمله‌ای مناسب است

    پارامترها:
```

```
documents: لیستی از اسناد (هر سند لیستی از کلمات است)
labels: لیستی از برچسب‌های مربوط به اسناد
target_class: کلاسی که می‌خواهیم فراوانی واژه‌ها را برای آن محاسبه کنیم

خروجی: دیکشنری که کلیدها واژه‌ها و مقادیر فراوانی مجموعه آن‌هاست
"""

# فیلتر کردن اسناد مربوط به کلاس هدف
target_docs = [doc for doc, label in zip(documents, labels) if label == target_class]

# شمارش تعداد کل تکرار هر واژه در تمام اسناد کلاس هدف
all_words = [word for doc in target_docs for word in doc]
coll_freq = Counter(all_words)

return dict(coll_freq)

def select_features_by_frequency(documents, labels, k, target_class, frequency_type='document'):
    """
    ویژگی برتر بر اساس فراوانی k انتخاب

    پارامترها:
    documents: لیستی از اسناد (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    k: تعداد ویژگی‌هایی که می‌خواهیم انتخاب کنیم
    target_class: کلاس هدف برای انتخاب ویژگی
    frequency_type: (برای فراوانی مجموعه 'collection'، برای فراوانی سندی 'document') نوع فراوانی

    واژه با بالاترین فراوانی k خروجی: لیستی از
    """
    # محاسبه فراوانی بر اساس نوع درخواستی
    if frequency_type == 'document':
        term_frequencies = calculate_document_frequency(documents, labels, target_class)
    elif frequency_type == 'collection':
        term_frequencies = calculate_collection_frequency(documents, labels, target_class)
    else:
        raise ValueError("باید 'collection' یا 'document' نوع فراوانی باشد")

    # واژه برتر k مرتب‌سازی واژه‌ها بر اساس فراوانی و انتخاب
    sorted_terms = sorted(term_frequencies.items(), key=lambda x: x[1], reverse=True)
    selected_features = [term for term, freq in sorted_terms[:k]]

    return selected_features, [freq for term, freq in sorted_terms[:k]]

def compare_feature_selection_methods(documents, labels, target_class, k=10):
    """
    مقایسه روش‌های انتخاب ویژگی مبتنی بر فراوانی سندی و مجموعه

    این تابع نشان می‌دهد که چگونه دو نوع فراوانی می‌توانند ویژگی‌های متفاوتی را انتخاب کنند
    """
    # انتخاب ویژگی با فراوانی سندی
    doc_features, doc_scores = select_features_by_frequency(documents, labels, k, target_class, 'document')

    # انتخاب ویژگی با فراوانی مجموعه
    coll_features, coll_scores = select_features_by_frequency(documents, labels, k, target_class, 'collection')

    # نمایش نتایج
    print(f"Top {k} features selected by document frequency for class '{target_class}':")
    for feature, score in zip(doc_features, doc_scores):
        print(f"{feature}: {score}")

    print(f"\nTop {k} features selected by collection frequency for class '{target_class}':")
    for feature, score in zip(coll_features, coll_scores):
        print(f"{feature}: {score}")

    # مقایسه ویژگی‌های مشترک
    common_features = set(doc_features) & set(coll_features)
    print(f"\nCommon features selected by both methods: {list(common_features)}")

    return doc_features, coll_features

def create_document_representation(documents, selected_features):
    """
    ایجاد نمایش سند بر اساس ویژگی‌های منتخب
    برای مدل چندجمله‌ای: تعداد تکرار هر ویژگی
    برای مدل برنولی: وجود یا عدم وجود هر ویژگی
    """
    # نمایش برای مدل چندجمله‌ای
    multinomial_rep = []
    for doc in documents:
        doc_rep = [doc.count(term) for term in selected_features]
        multinomial_rep.append(doc_rep)

    # نمایش برای مدل برنولی
    bernoulli_rep = []
    for doc in documents:
        doc_rep = [1 if term in doc else 0 for term in selected_features]
        bernoulli_rep.append(doc_rep)

    return np.array(multinomial_rep), np.array(bernoulli_rep)

def evaluate_frequency_feature_selection(documents, labels, target_class, feature_counts):
    """
    ارزیابی تأثیر انتخاب ویژگی مبتنی بر فراوانی بر دقت طبقه‌بندی

    این تابع دقت مدل‌های چندجمله‌ای و برنولی را برای تعداد مختلفی از ویژگی‌ها محاسبه می‌کند.
    نتایج نشان می‌دهد چگونه انتخاب ویژگی مبتنی بر فراوانی می‌تواند دقت را بهبود بخشد
    """
    # تقسیم داده‌ها به آموزش و آزمون
    X_train_docs, X_test_docs, y_train, y_test = train_test_split(
        documents, labels, test_size=0.3, random_state=42
    )

    # تبدیل برچسب‌ها به باینری (کلاس هدف در مقابل بقیه)
    y_train_binary = np.array([1 if label == target_class else 0 for label in y_train])
```

```
y_test_binary = np.array([1 if label == target_class else 0 for label in y_test])

# استخراج واژگان از داده‌های آموزشی
vocab = set()
for doc in X_train_docs:
    vocab.update(doc)

# محاسبه فراوانی سندی برای همه واژه‌ها
doc_freq_scores = {}
for term in vocab:
    doc_freq_scores[term] = calculate_document_frequency(X_train_docs, y_train, target_class).get(term, 0)

# محاسبه فراوانی مجموعه برای همه واژه‌ها
coll_freq_scores = {}
for term in vocab:
    coll_freq_scores[term] = calculate_collection_frequency(X_train_docs, y_train, target_class).get(term, 0)

# مرتب‌سازی واژه‌ها بر اساس فراوانی‌ها
doc_sorted_terms = sorted(doc_freq_scores.items(), key=lambda x: x[1], reverse=True)
coll_sorted_terms = sorted(coll_freq_scores.items(), key=lambda x: x[1], reverse=True)

doc_terms = [term for term, score in doc_sorted_terms]
coll_terms = [term for term, score in coll_sorted_terms]

# ارزیابی برای تعداد مختلفی از ویژگی‌ها
doc_multinomial_scores = []
doc_bernoulli_scores = []
coll_multinomial_scores = []
coll_bernoulli_scores = []

for k in feature_counts:
    # ویژگی برتر با فراوانی سندی k انتخاب
    doc_selected_features = doc_terms[:k]

    # ایجاد نمایش سند برای مدل‌ها
    X_train_multi, X_train_bern = create_document_representation(X_train_docs, doc_selected_features)
    X_test_multi, X_test_bern = create_document_representation(X_test_docs, doc_selected_features)

    # آموزش و ارزیابی مدل چندجمله‌ای با فراوانی سندی
    clf_multi = MultinomialNB()
    clf_multi.fit(X_train_multi, y_train_binary)
    y_pred_multi = clf_multi.predict(X_test_multi)
    f1_multi = f1_score(y_test_binary, y_pred_multi)
    doc_multinomial_scores.append(f1_multi)

    # آموزش و ارزیابی مدل برنولی با فراوانی سندی
    clf_bern = BernoulliNB()
    clf_bern.fit(X_train_bern, y_train_binary)
    y_pred_bern = clf_bern.predict(X_test_bern)
    f1_bern = f1_score(y_test_binary, y_pred_bern)
    doc_bernoulli_scores.append(f1_bern)

    # ویژگی برتر با فراوانی مجموعه k انتخاب
    coll_selected_features = coll_terms[:k]

    # ایجاد نمایش سند برای مدل‌ها
    X_train_multi, X_train_bern = create_document_representation(X_train_docs, coll_selected_features)
    X_test_multi, X_test_bern = create_document_representation(X_test_docs, coll_selected_features)

    # آموزش و ارزیابی مدل چندجمله‌ای با فراوانی مجموعه
    clf_multi = MultinomialNB()
    clf_multi.fit(X_train_multi, y_train_binary)
    y_pred_multi = clf_multi.predict(X_test_multi)
    f1_multi = f1_score(y_test_binary, y_pred_multi)
    coll_multinomial_scores.append(f1_multi)

    # آموزش و ارزیابی مدل برنولی با فراوانی مجموعه
    clf_bern = BernoulliNB()
    clf_bern.fit(X_train_bern, y_train_binary)
    y_pred_bern = clf_bern.predict(X_test_bern)
    f1_bern = f1_score(y_test_binary, y_pred_bern)
    coll_bernoulli_scores.append(f1_bern)

    return (doc_multinomial_scores, doc_bernoulli_scores,
            coll_multinomial_scores, coll_bernoulli_scores)

# ایجاد مجموعه داده نمونه
np.random.seed(42)

# کلاس‌ها
classes = ['poultry', 'coffee', 'sports', 'china', 'uk', 'elections']

# واژگان مرتبط با هر کلاس
class_vocab = {
    'poultry': ['poultry', 'chicken', 'meat', 'avian', 'bird', 'farm', 'egg', 'broiler', 'inspection', 'disease'],
    'coffee': ['coffee', 'bean', 'roast', 'brew', 'colombia', 'brazil', 'export', 'crop', 'harvest', 'arabica'],
    'sports': ['soccer', 'game', 'match', 'team', 'player', 'score', 'win', 'league', 'cup', 'championship'],
    'china': ['china', 'chinese', 'beijing', 'yuan', 'shanghai', 'hong', 'kong', 'xinhua', 'province', 'taiwan'],
    'uk': ['london', 'uk', 'british', 'britain', 'england', 'pound', 'pence', 'stg', 'plc', 'english'],
    'elections': ['election', 'vote', 'candidate', 'campaign', 'poll', 'party', 'democratic', 'republican', 'president', 'ballot']
}

# واژگان عمومی (که در همه کلاس‌ها پرتکرار هستند)
general_vocab = ['report', 'said', 'year', 'today', 'new', 'government', 'market', 'company', 'people', 'world',
                 'monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday', 'sunday']

# ایجاد اسناد
documents = []
labels = []
num_docs_per_class = 100

for cls in classes:
    for i in range(num_docs_per_class):
```

```

هر سند شامل 5-15 واژه است
doc_length = np.random.randint(5, 16)
doc = []

# اضافه کردن 2-4 واژه از کلاس
num_class_terms = np.random.randint(2, 5)
class_terms = np.random.choice(class_vocab[cls], num_class_terms, replace=False)
doc.extend(class_terms)

# اضافه کردن واژگان عمومی
num_general_terms = doc_length - num_class_terms
general_terms = np.random.choice(general_vocab, num_general_terms, replace=True)
doc.extend(general_terms)

documents.append(doc)
labels.append(cls)

# مقایسه روش‌های انتخاب ویژگی مبتنی بر فراوانی سندی و مجموعه
doc_features, coll_features = compare_feature_selection_methods(documents, labels, "poultry", k=10)

# ارزیابی تأثیر انتخاب ویژگی مبتنی بر فراوانی بر دقت طبقه‌بندی
feature_counts = [1, 5, 10, 20, 50, 100]
(doc_multi_scores, doc_bernoulli_scores,
 coll_multi_scores, coll_bernoulli_scores) = evaluate_frequency_feature_selection(
    documents, labels, "poultry", feature_counts)

# نمایش نتایج
plt.figure(figsize=(12, 8))
plt.plot(feature_counts, doc_multi_scores, 'o-', label='Multinomial NB with Document Frequency')
plt.plot(feature_counts, doc_bernoulli_scores, 'x-', label='Bernoulli NB with Document Frequency')
plt.plot(feature_counts, coll_multi_scores, 's-', label='Multinomial NB with Collection Frequency')
plt.plot(feature_counts, coll_bernoulli_scores, 'd-', label='Bernoulli NB with Collection Frequency')
plt.xscale('log')
plt.xlabel('Number of Features')
plt.ylabel('F1 Score')
plt.title('Effect of Frequency-based Feature Selection on Classification Accuracy')
plt.legend()
plt.grid(True)
plt.show()

# نمایش نتایج عددی
print("\nنتایج ارزیابی:")
print("(برنولی با فراوانی مجموعه) \tF1 (چندجمله‌ای با فراوانی مجموعه) \tF1 (برنولی با فراوانی سندی) \tF1 (چندجمله‌ای با فراوانی سندی) \tتعداد ویژگی")
for i, k in enumerate(feature_counts):
    print(f"{k}\t\t{doc_multi_scores[i]:.4f}\t\t\t{doc_bernoulli_scores[i]:.4f}\t\t\t{coll_multi_scores[i]:.4f}\t\t\t{coll_bernoulli_scores[i]:.4f}")

# نمایش ویژگی‌های منتخب با فراوانی سندی و مجموعه
print("\nپoultry: ویژگی‌های منتخب با فراوانی سندی برای کلاس")
for i, feature in enumerate(doc_features[:10]):
    print(f"{i+1}. {feature}")

print("\nپoultry: ویژگی‌های منتخب با فراوانی مجموعه برای کلاس")
for i, feature in enumerate(coll_features[:10]):
    print(f"{i+1}. {feature}")

```

Top 10 features selected by document frequency for class 'poultry':

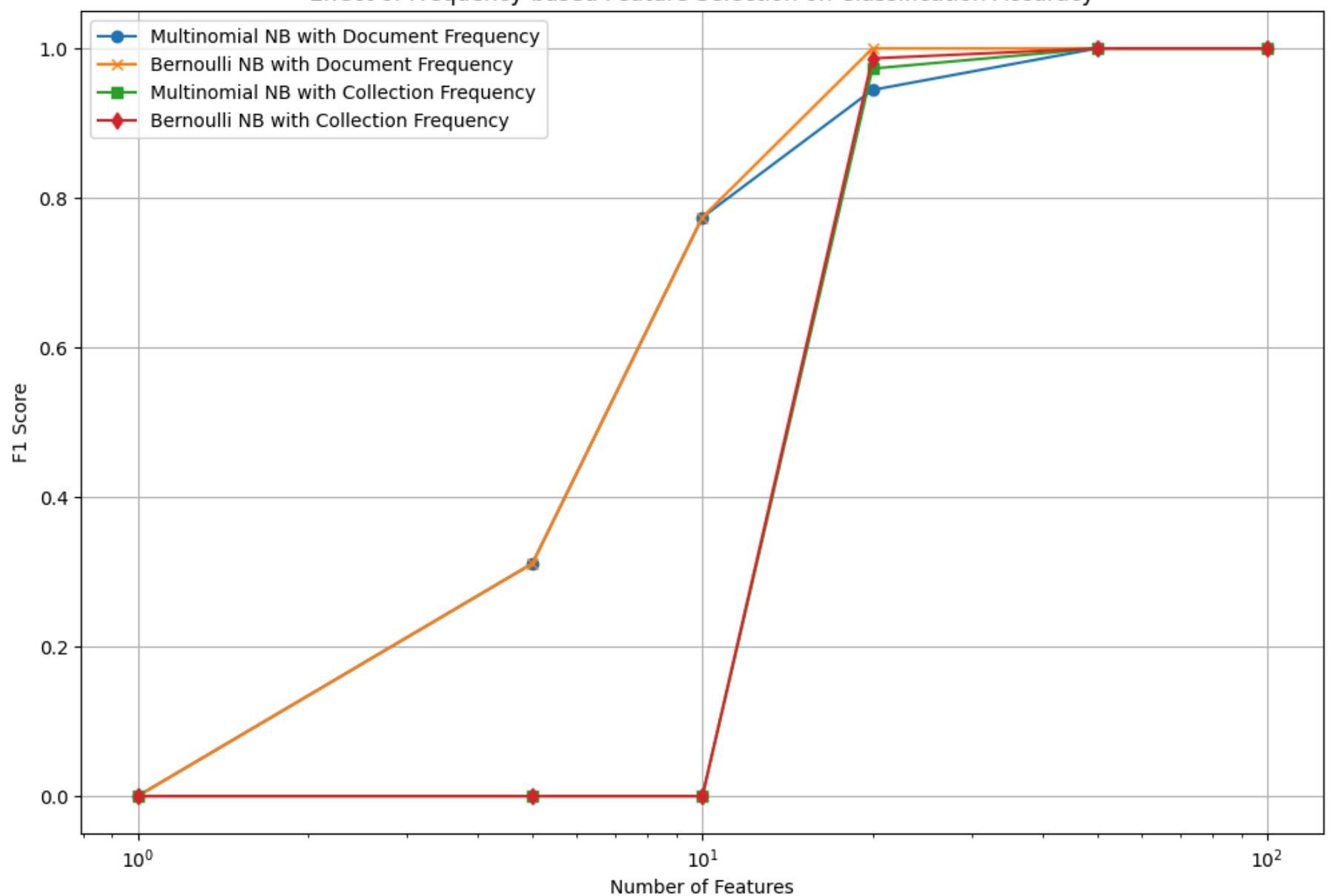
report: 42
disease: 40
company: 39
people: 39
new: 37
year: 36
wednesday: 35
sunday: 34
tuesday: 32
bird: 32

Top 10 features selected by collection frequency for class 'poultry':

report: 54
people: 49
company: 48
wednesday: 48
year: 45
tuesday: 45
new: 45
sunday: 45
disease: 40
market: 38

Common features selected by both methods: [np.str_('new'), np.str_('disease'), np.str_('tuesday'), np.str_('report'), np.str_('sunday'), np.str_('wednesday'), np.str_('year'), np.str_('people'), np.str_('company')]

Effect of Frequency-based Feature Selection on Classification Accuracy



نتایج ارزیابی:

تعداد ویژگی	F1 (چندجمله‌ای با فراوانی سندی)	F1 (برنولی با فراوانی سندی)	F1 (چندجمله‌ای با فراوانی مجموعه)	F1 (برنولی با فراوانی مجموعه)
1	0.0000	0.0000	0.0000	0.0000
5	0.3111	0.3111	0.0000	0.0000
10	0.7742	0.7742	0.0000	0.0000
20	0.9444	1.0000	0.9730	0.9867
50	1.0000	1.0000	1.0000	1.0000
100	1.0000	1.0000	1.0000	1.0000

poultry: ویژگی‌های منتخب با فراوانی سندی برای کلاس

- report
- disease
- company
- people
- new
- year
- wednesday
- sunday
- tuesday
- bird

poultry: ویژگی‌های منتخب با فراوانی مجموعه برای کلاس

- report
- people
- company
- wednesday
- year
- tuesday
- new
- sunday
- disease
- market

13.5.4 انتخاب ویژگی برای چندین طبقه‌بند: چگونه بین دقت و کارایی تعادل برقرار کنیم؟

🌐 گوگل و مدیریت میلیاردها سند: چگونه همه چیز را با هم طبقه‌بندی کنیم؟

تصور کنید در گوگل کار می‌کنید و مسئول طراحی سیستمی هستید که باید روزانه میلیاردها سند (ایمیل‌ها، اسناد گوگل داک، عکس‌ها، ویدیوها و صفحات وب) را به دسته‌های مختلفی اختصاص دهد. هر نوع محتوا نیاز به طبقه‌بند خاص خود دارد، اما آیا باید برای هر کدام ویژگی‌های متفاوتی انتخاب کنید؟

این چالشی است که تیم‌های مهندسی گوگل با آن روبرو هستند: چگونه می‌توان مجموعه‌ای از ویژگی‌ها را انتخاب کرد که برای همه طبقه‌بندها کارآمد باشد، حتی اگر برای هر کدام به صورت جداگانه کمی دقت کمتری داشته باشد؟

در یک سیستم عملیاتی با تعداد زیادی طبقه‌بند، مطلوب است که یک مجموعه واحد از ویژگی‌ها انتخاب شود به جای مجموعه‌های متفاوت برای هر طبقه‌بند. این رویکرد به دلایل زیر مهم است:

- کاهش پیچیدگی محاسباتی و نیاز به حافظه
- تسهیل نگهداری و به‌روزرسانی سیستم
- امکان استفاده از نمایش سند مشترک برای چندین وظیفه

دو رویکرد اصلی برای انتخاب ویژگی‌های مشترک برای چندین طبقه‌بند وجود دارد:

🇮🇷 رویکرد اول: آماره χ^2 برای جدول $n \times 2$

یک روش برای انجام این کار، محاسبه آماره χ^2 برای یک جدول $n \times 2$ است که در آن ستون‌ها وقوع و عدم وقوع یک واژه هستند و هر ردیف مربوط به یکی از کلاس‌ها است. سپس می‌توانیم k واژه با بالاترین آماره χ^2 را انتخاب کنیم.

مثال واقعی: در سیستم طبقه‌بندی اسناد مایکروسافت، این رویکرد به تیم اجازه می‌دهد تا مجموعه‌ای از ویژگی‌ها را برای طبقه‌بندی انواع مختلف اسناد (Word، Excel، PowerPoint) انتخاب کند. این کار باعث می‌شود که نمایش سند مشترکی برای همه انواع اسناد وجود داشته باشد و جستجو و تحلیل آسان‌تر شود.

🇮🇷 رویکرد دوم: ترکیب آماره‌های جداگانه

روش متداول‌تر این است که آماره‌های انتخاب ویژگی ابتدا به صورت جداگانه برای هر کلاس در وظایف طبقه‌بندی دو کلاسی (c در مقابل $\bar{c}$) محاسبه شده و سپس ترکیب شوند. دو روش ترکیب متداول وجود دارد:

- روش میانگین:** یک شاخص واحد برای هر ویژگی محاسبه می‌شود (مثلاً با میانگین‌گیری از مقادیر $A(t, c)$ و k ویژگی با بالاترین شاخص‌ها انتخاب می‌شوند.
- روش ترکیبی:** k/n ویژگی برتر برای هر یک از n طبقه‌بند انتخاب شده و سپس این n مجموعه در یک مجموعه ویژگی جهانی ترکیب می‌شوند.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان آمازون، تیم تحلیل داده‌ها ابتدا ویژگی‌های برتر را برای هر دسته (کتاب، الکترونیک، لباس) جداگانه محاسبه می‌کند. سپس با استفاده از روش ترکیبی، 20٪ از برترین ویژگی‌های هر دسته را انتخاب کرده و در یک مجموعه جهانی ترکیب می‌کند. این کار باعث می‌شود که سیستم بتواند به سرعت نظرات جدید را دسته‌بندی کند، حتی اگر برای هر دسته کمی دقت کمتری نسبت به استفاده از ویژگی‌های اختصاصی داشته باشد.

🇮🇷 معامله بین دقت و کارایی

دقت طبقه‌بندی اغلب کاهش می‌یابد هنگام انتخاب k ویژگی مشترک برای یک سیستم با n طبقه‌بند در مقابل n مجموعه متفاوت به اندازه k . اما حتی اگر دقت کاهش یابد، افزایش کارایی به دلیل نمایش سند مشترک ممکن است ارزش کاهش دقت را داشته باشد.

مثال واقعی: در سیستم فیلتر محتوای فیسبوک، که باید میلیاردها پست را در ثانیه طبقه‌بندی کند، استفاده از یک مجموعه ویژگی مشترک برای همه طبقه‌بندها (خشونت، هرزنامه، اخبار جعلی) ضروری است. حتی اگر این کار باعث

کاهش 1-2 درصدی دقت شود، افزایش سرعت پردازش (تا 10 برابر) این کاهش دقت را جبران می‌کند و به فیسبوک اجازه می‌دهد تا محتوای مضر را به سرعت شناسایی و حذف کند.

In [7]: پیاده‌سازی انتخاب ویژگی برای چندین طبقه‌بند #

```
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
import math
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import f1_score

def calculate_chi2_for_multiple_classes(documents, labels, term):
    """
    محاسبه  $\chi^2$  برای یک واژه در جدول  $\chi^2$  محاسبه آماره

    محاسبه می‌کند که در آن  $n \times 2$  را برای یک واژه در جدول  $\chi^2$  این تابع آماره
    ستون‌ها وقوع و عدم وقوع واژه هستند و هر ردیف مربوط به یکی از کلاس‌هاست.

    پارامترها:
    documents: (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    term: واژه مورد نظر

    برای واژه مورد نظر  $\chi^2$  خروجی: مقدار
    """
    N = len(documents)
    classes = list(set(labels))
    n_classes = len(classes)

    # برای واژه مورد نظر  $n \times 2$  ایجاد جدول
    # [که شامل واژه نیستند i که شامل واژه هستند, تعداد اسناد در کلاس i, تعداد اسناد در کلاس]: هر ردیف
    table = []

    for c in classes:
        N11 = N10 = N01 = N00 = 0

        for doc, label in zip(documents, labels):
            term_in_doc = 1 if term in doc else 0
            is_target_class = 1 if label == c else 0

            if term_in_doc == 1 and is_target_class == 1:
                N11 += 1
            elif term_in_doc == 1 and is_target_class == 0:
                N10 += 1
            elif term_in_doc == 0 and is_target_class == 1:
                N01 += 1
            else: # term_in_doc == 0 and is_target_class == 0
                N00 += 1

        table.append([N11, N01])

    # محاسبه  $\chi^2$  برای جدول  $\chi^2$  محاسبه
    chi2 = 0.0

    # محاسبه مجموع فراوانی‌های مشاهده شده در هر ستون
    col1_sum = sum(row[0] for row in table) # همه کلاس‌ها
    col2_sum = sum(row[1] for row in table) # همه کلاس‌ها

    # محاسبه مجموع فراوانی‌های مشاهده شده در هر ردیف
    row_sums = [sum(row) for row in table] # مجموع اسناد در هر کلاس

    # محاسبه فراوانی‌های مورد انتظار
    for i, row in enumerate(table):
        row_sum = row_sums[i]
        for j, cell in enumerate(row):
            col_sum = col1_sum if j == 0 else col2_sum
            expected = (row_sum * col_sum) / N

            if expected > 0:
                chi2 += ((cell - expected) ** 2) / expected

    return chi2

def calculate_feature_scores_per_class(documents, labels, score_func):
    """
    محاسبه امتیاز ویژگی‌ها برای هر کلاس به صورت جداگانه

    این تابع امتیاز ویژگی‌ها را برای هر کلاس به صورت جداگانه محاسبه می‌کند.
    این رویکرد برای روش ترکیبی مناسب است.

    پارامترها:
    documents: (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    score_func:  $\chi^2$  مانند اطلاعات متقابل یا) تابعی برای محاسبه امتیاز ویژگی

    خروجی: دیکشنری که کلیدها کلاس‌ها و مقادیر دیکشنری امتیاز ویژگی‌ها برای آن کلاس است
    """
    classes = list(set(labels))
    feature_scores = defaultdict(dict)

    # استخراج واژگان
    vocab = set()
    for doc in documents:
```

```

vocab.update(doc)

# محاسبه امتیاز برای هر واژه و هر کلاس
for term in vocab:
    for c in classes:
        score = score_func(documents, labels, term, c)
        feature_scores[c][term] = score

return feature_scores

def select_features_average_method(documents, labels, k, score_func):
    """
    انتخاب ویژگی با روش میانگین

    A(t,c) با میانگین‌گیری از مقادیر) این روش یک شاخص واحد برای هر ویژگی محاسبه می‌کند
    .ویژگی با بالاترین شاخص‌ها را انتخاب می‌کند k و

    پارامترها:
    documents: (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    k: تعداد ویژگی‌هایی که می‌خواهیم انتخاب کنیم
    score_func: (مانند اطلاعات متقابل یا) تابعی برای محاسبه امتیاز ویژگی

    واژه با بالاترین امتیاز میانگین k خروجی: لیستی از
    """
    # محاسبه امتیاز ویژگی‌ها برای هر کلاس
    feature_scores = calculate_feature_scores_per_class(documents, labels, score_func)
    classes = list(set(labels))

    # محاسبه میانگین امتیاز برای هر ویژگی
    avg_scores = {}
    for term in feature_scores[classes[0]].keys(): # همه واژه‌ها در همه کلاس‌ها وجود دارند
        avg_score = sum(feature_scores[c][term] for c in classes) / len(classes)
        avg_scores[term] = avg_score

    # واژه با بالاترین امتیاز میانگین k انتخاب
    selected_features = sorted(avg_scores.items(), key=lambda x: x[1], reverse=True)[:k]
    return [term for term, score in selected_features]

def select_features_union_method(documents, labels, k_per_class, score_func):
    """
    انتخاب ویژگی با روش ترکیبی

    طبقه‌بند انتخاب کرده n ویژگی برتر برای هر یک از k/n این روش
    .مجموعه را در یک مجموعه ویژگی جهانی ترکیب می‌کند n و سپس این

    پارامترها:
    documents: (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    k_per_class: تعداد ویژگی‌هایی که برای هر کلاس انتخاب می‌شوند
    score_func: (مانند اطلاعات متقابل یا) تابعی برای محاسبه امتیاز ویژگی

    خروجی: لیستی از ویژگی‌های منتخب از ترکیب همه کلاس‌ها
    """
    # محاسبه امتیاز ویژگی‌ها برای هر کلاس
    feature_scores = calculate_feature_scores_per_class(documents, labels, score_func)
    classes = list(set(labels))

    # ویژگی برتر برای هر کلاس k انتخاب
    selected_per_class = {}
    for c in classes:
        class_scores = feature_scores[c]
        top_features = sorted(class_scores.items(), key=lambda x: x[1], reverse=True)[:k_per_class]
        selected_per_class[c] = [term for term, score in top_features]

    # ترکیب مجموعه‌های ویژگی
    combined_features = set()
    for features in selected_per_class.values():
        combined_features.update(features)

    return list(combined_features)

def mutual_information(documents, labels, term, target_class):
    """
    محاسبه اطلاعات متقابل بین یک واژه و یک کلاس
    """
    N = len(documents)

    # N11, N10, N01, N00 شمارش مقادیر
    N11 = N10 = N01 = N00 = 0

    for doc, label in zip(documents, labels):
        term_in_doc = 1 if term in doc else 0
        is_target_class = 1 if label == target_class else 0

        if term_in_doc == 1 and is_target_class == 1:
            N11 += 1
        elif term_in_doc == 1 and is_target_class == 0:
            N10 += 1
        elif term_in_doc == 0 and is_target_class == 1:
            N01 += 1
        else: # term_in_doc == 0 and is_target_class == 0
            N00 += 1

    # محاسبه مقادیر جانبی
    N1_ = N10 + N11 # تعداد اسناد شامل واژه
    N0_ = N00 + N01 # تعداد اسناد بدون واژه
    N_1 = N01 + N11 # تعداد اسناد در کلاس هدف
    N_0 = N00 + N10 # تعداد اسناد در کلاس غیرهدف

    # محاسبه اطلاعات متقابل
    mi = 0.0
    if N11 > 0:

```

```

mi += (N11 / N) * math.log2((N * N11) / (N1_ * N_1))
if N01 > 0:
    mi += (N01 / N) * math.log2((N * N01) / (N0_ * N_1))
if N10 > 0:
    mi += (N10 / N) * math.log2((N * N10) / (N1_ * N_0))
if N00 > 0:
    mi += (N00 / N) * math.log2((N * N00) / (N0_ * N_0))

return mi

def chi2(documents, labels, term, target_class):
    """
    برای یک واژه و یک کلاس  $\chi^2$  محاسبه آماره
    """
    N = len(documents)

    # شمارش مقادیر N11, N10, N01, N00
    N11 = N10 = N01 = N00 = 0

    for doc, label in zip(documents, labels):
        term_in_doc = 1 if term in doc else 0
        is_target_class = 1 if label == target_class else 0

        if term_in_doc == 1 and is_target_class == 1:
            N11 += 1
        elif term_in_doc == 1 and is_target_class == 0:
            N10 += 1
        elif term_in_doc == 0 and is_target_class == 1:
            N01 += 1
        else: # term_in_doc == 0 and is_target_class == 0
            N00 += 1

    # با استفاده از فرمول ساده‌شده  $\chi^2$  محاسبه
    numerator = (N11 + N10 + N01 + N00) * ((N11 * N00) - (N10 * N01))**2
    denominator = (N11 + N01) * (N11 + N10) * (N10 + N00) * (N01 + N00)

    if denominator == 0:
        return 0

    chi2_value = numerator / denominator
    return chi2_value

def evaluate_feature_selection_methods(documents, labels, k_values):
    """
    ارزیابی روش‌های مختلف انتخاب ویژگی برای چندین طبقه‌بند
    این تابع دقت طبقه‌بندی را برای روش‌های مختلف انتخاب ویژگی محاسبه می‌کند.
    نتایج نشان می‌دهد چگونه روش‌های مختلف می‌توانند دقت را تحت تأثیر قرار دهند.
    """
    # تقسیم داده‌ها به آموزش و آزمون
    X_train_docs, X_test_docs, y_train, y_test = train_test_split(
        documents, labels, test_size=0.3, random_state=42
    )

    # استخراج واژگان
    vocab = set()
    for doc in X_train_docs:
        vocab.update(doc)

    # ارزیابی برای تعداد مختلفی از ویژگی‌ها
    chi2_n2_scores = []
    mi_avg_scores = []
    mi_union_scores = []

    for k in k_values:
        # روش اول:  $\chi^2$ 
        chi2_scores = {}
        for term in vocab:
            chi2_scores[term] = calculate_chi2_for_multiple_classes(X_train_docs, y_train, term)

        # ویژگی برتر k انتخاب
        selected_chi2 = sorted(chi2_scores.items(), key=lambda x: x[1], reverse=True)[:k]
        chi2_features = [term for term, score in selected_chi2]

        # ایجاد نمایش سند برای مدل چندجمله‌ای
        X_train = []
        X_test = []
        for doc in X_train_docs:
            doc_rep = [doc.count(term) for term in chi2_features]
            X_train.append(doc_rep)

        for doc in X_test_docs:
            doc_rep = [doc.count(term) for term in chi2_features]
            X_test.append(doc_rep)

        # آموزش و ارزیابی مدل
        clf = MultinomialNB()
        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)
        f1 = f1_score(y_test, y_pred, average='macro')
        chi2_n2_scores.append(f1)

    # روش دوم: میانگین اطلاعات متقابل
    selected_mi_avg = select_features_average_method(X_train_docs, y_train, k, mutual_information)

    # ایجاد نمایش سند برای مدل چندجمله‌ای
    X_train = []
    X_test = []
    for doc in X_train_docs:
        doc_rep = [doc.count(term) for term in selected_mi_avg]
        X_train.append(doc_rep)

    for doc in X_test_docs:

```

```
doc_rep = [doc.count(term) for term in selected_mi_avg]
X_test.append(doc_rep)

# آموزش و ارزیابی مدل
clf = MultinomialNB()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
f1 = f1_score(y_test, y_pred, average='macro')
mi_avg_scores.append(f1)

# روش سوم: ترکیب اطلاعات متقابل
# بر تعداد کلاس‌ها k تقسیم
k_per_class = max(1, k // len(set(y_train)))
selected_mi_union = select_features_union_method(X_train_docs, y_train, k_per_class, mutual_information)

# ایجاد نمایش سند برای مدل چندجمله‌ای
X_train = []
X_test = []
for doc in X_train_docs:
    doc_rep = [doc.count(term) for term in selected_mi_union]
    X_train.append(doc_rep)

for doc in X_test_docs:
    doc_rep = [doc.count(term) for term in selected_mi_union]
    X_test.append(doc_rep)

# آموزش و ارزیابی مدل
clf = MultinomialNB()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
f1 = f1_score(y_test, y_pred, average='macro')
mi_union_scores.append(f1)

return chi2_n2_scores, mi_avg_scores, mi_union_scores

# ایجاد مجموعه داده نمونه
np.random.seed(42)

# کلاس‌ها
classes = ['poultry', 'coffee', 'sports', 'china', 'uk', 'elections']

# واژگان مرتبط با هر کلاس
class_vocab = {
    'poultry': ['poultry', 'chicken', 'meat', 'avian', 'bird', 'farm', 'egg', 'broiler', 'inspection', 'disease'],
    'coffee': ['coffee', 'bean', 'roast', 'brew', 'colombia', 'brazil', 'export', 'crop', 'harvest', 'arabica'],
    'sports': ['soccer', 'game', 'match', 'team', 'player', 'score', 'win', 'league', 'cup', 'championship'],
    'china': ['china', 'chinese', 'beijing', 'yuan', 'shanghai', 'hong', 'kong', 'xinhua', 'province', 'taiwan'],
    'uk': ['london', 'uk', 'british', 'britain', 'england', 'pound', 'pence', 'stg', 'plc', 'english'],
    'elections': ['election', 'vote', 'candidate', 'campaign', 'poll', 'party', 'democratic', 'republican', 'president', 'ballot']
}

# واژگان عمومی
general_vocab = ['report', 'said', 'year', 'today', 'new', 'government', 'market', 'company', 'people', 'world']

# ایجاد اسناد
documents = []
labels = []
num_docs_per_class = 100

for cls in classes:
    for i in range(num_docs_per_class):
        # هر سند شامل 5-15 واژه است
        doc_length = np.random.randint(5, 16)
        doc = []

        # اضافه کردن 2-4 واژه از کلاس
        num_class_terms = np.random.randint(2, 5)
        class_terms = np.random.choice(class_vocab[cls], num_class_terms, replace=False)
        doc.extend(class_terms)

        # اضافه کردن واژگان عمومی
        num_general_terms = doc_length - num_class_terms
        general_terms = np.random.choice(general_vocab, num_general_terms, replace=True)
        doc.extend(general_terms)

    documents.append(doc)
    labels.append(cls)

# مقایسه روش‌های انتخاب ویژگی برای چندین طبقه‌بند
print("مقایسه روش‌های انتخاب ویژگی برای چندین طبقه‌بند")

# n×2 برای جدول  $\chi^2$ : روش اول
chi2_scores = {}
for term in set(word for doc in documents for word in doc):
    chi2_scores[term] = calculate_chi2_for_multiple_classes(documents, labels, term)

# انتخاب 10 ویژگی برتر
selected_chi2 = sorted(chi2_scores.items(), key=lambda x: x[1], reverse=True)[:10]
chi2_features = [term for term, score in selected_chi2]

print("\nn10 برای جدول  $\chi^2$  ویژگی برتر با استفاده از روش n×2:")
for i, (term, score) in enumerate(selected_chi2):
    print(f"{i+1}. {term}: {score:.4f}")

# روش دوم: میانگین اطلاعات متقابل
selected_mi_avg = select_features_average_method(documents, labels, 10, mutual_information)

print("\nn10 روش میانگین اطلاعات متقابل")
for i, term in enumerate(selected_mi_avg):
    print(f"{i+1}. {term}")

# روش سوم: ترکیب اطلاعات متقابل
# ویژگی برای هر کلاس 2
k_per_class = 2
selected_mi_union = select_features_union_method(documents, labels, k_per_class, mutual_information)
```

```
print(f"\n(2 ویژگی برای هر کلاس) استفاده از روش ترکیب اطلاعات متقابل")
print(f"تعداد کل ویژگی‌ها: {len(selected_mi_union)}")
print(f"ویژگی‌ها: {selected_mi_union}")

# ارزیابی روش‌های مختلف انتخاب ویژگی
k_values = [10, 20, 50, 100]
chi2_n2_scores, mi_avg_scores, mi_union_scores = evaluate_feature_selection_methods(documents, labels, k_values)

# نمایش نتایج
plt.figure(figsize=(10, 6))
plt.plot(k_values, chi2_n2_scores, 'o-', label='χ² for n×2 table')
plt.plot(k_values, mi_avg_scores, 's-', label='Average MI')
plt.plot(k_values, mi_union_scores, 'd-', label='Union MI')
plt.xlabel('Number of Features')
plt.ylabel('Macro F1 Score')
plt.title('Comparison of Feature Selection Methods for Multiple Classifiers')
plt.legend()
plt.grid(True)
plt.show()

# نمایش نتایج عددی
print("\nنتایج ارزیابی:")
print("MI ترکیب MI میانگین (جدول n×2) χ² تعداد ویژگی")
for i, k in enumerate(k_values):
    print(f"{k}\t\t{chi2_n2_scores[i]:.4f}\t\t{mi_avg_scores[i]:.4f}\t\t{mi_union_scores[i]:.4f}")
```

مقایسه روش‌های انتخاب ویژگی برای چندین طبقه‌بند

nx2: برای جدول χ^2 ویژگی برتر با استفاده از روش 10

- 1. republican: 284.6715
- 2. beijing: 214.2857
- 3. ballot: 208.5561
- 4. party: 202.8470
- 5. brazil: 197.1581
- 6. bean: 191.4894
- 7. disease: 191.4894
- 8. british: 191.4894
- 9. chicken: 185.8407
- 10. chinese: 180.2120

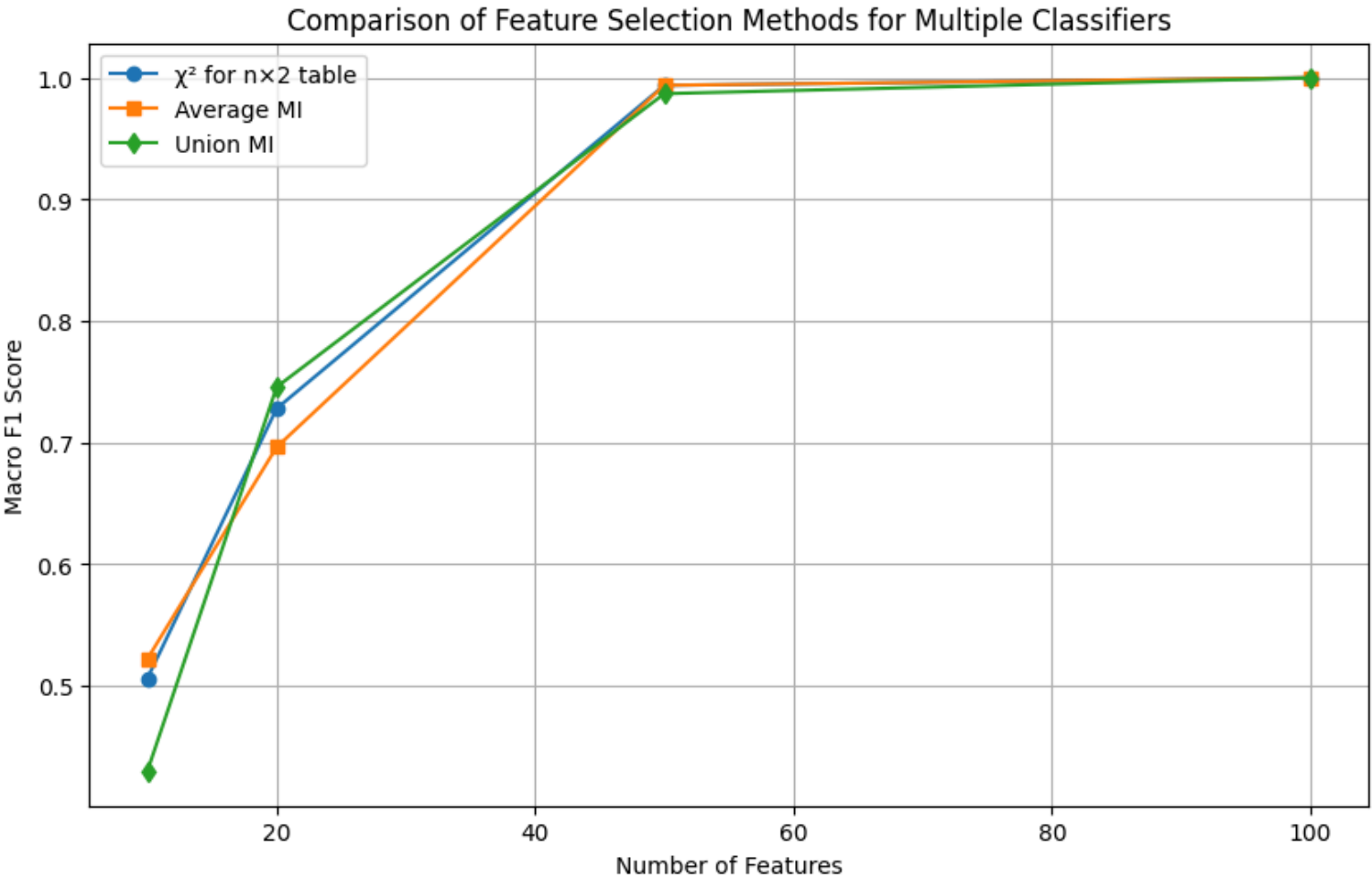
ویژگی برتر با استفاده از روش میانگین اطلاعات متقابل

- 1. republican
- 2. beijing
- 3. ballot
- 4. party
- 5. brazil
- 6. bean
- 7. disease
- 8. british
- 9. chicken
- 10. chinese

ویژگی‌های منتخب با استفاده از روش ترکیب اطلاعات متقابل (2 ویژگی برای هر کلاس)

تعداد کل ویژگی‌ها: 12

ویژگی‌ها: [np.str_('stg'), np.str_('disease'), np.str_('ballot'), np.str_('beijing'), np.str_('cup'), np.str_('british'), np.str_('chicken'), np.str_('bean'), np.str_('brazil'), np.str_('republican'), np.str_('game'), np.str_('chinese')]



نتایج ارزیابی:

تعداد ویژگی	χ^2 (جدول n×2)	MI ترکیب MI میانگین	
10	0.5063	0.5223	0.4300
20	0.7281	0.6964	0.7458
50	0.9939	0.9939	0.9870
100	1.0000	1.0000	1.0000



13.5.5 مقایسه روش‌های انتخاب ویژگی: کدام روش برای کدام مشکل؟

🔍 گوگل نیوز و چالش واژه‌های نادر: چگونه اخبار مهم را از اخبار عادی تشخیص دهیم؟

تیم گوگل نیوز با چالشی جالب روبرو است: واژه‌ای مانند "پاندا" ممکن است فقط یک بار در اخبار مربوط به چین ظاهر شود، اما از نظر آماری بسیار معنادار باشد. آیا این واژه باید به عنوان یک ویژگی مهم برای شناسایی اخبار چین انتخاب شود؟

این دقیقاً همان تفاوت بین اطلاعات متقابل (MI) و χ^2 است: χ^2 به دنبال واژه‌های معنادار از نظر آماری است، حتی اگر به ندرت ظاهر شوند، در حالی که MI به دنبال واژه‌هایی است که اطلاعات بیشتری را به همراه دارند، صرف نظر از تعداد تکرار آن‌ها.

اطلاعات متقابل (MI) و χ^2 روش‌های نسبتاً متفاوتی برای انتخاب ویژگی هستند. تفاوت اصلی آن‌ها در نحوه برخورد با واژه‌های نادر است:

📊 معناداری در مقابل اطلاعاتی بودن: تفاوت کلیدی χ^2 و MI

استقلال واژه t و کلاس c گاهی می‌تواند با اطمینان بالایی رد شود حتی اگر t اطلاعات کمی درباره عضویت سند در c داشته باشد. این موضوع به ویژه برای واژه‌های نادر صادق است.

اگر واژه‌ای یک بار در یک مجموعه بزرگ رخ دهد و آن یک وقوع در کلاس poultry باشد، این از نظر آماری معنادار است. اما یک وقوع واحد بر اساس تعریف نظریه اطلاعات، بسیار اطلاعاتی نیست. چون معیار χ^2 معناداری است، این روش واژه‌های نادر بیشتری را نسبت به اطلاعات متقابل انتخاب می‌کند (که اغلب شاخص‌های قابل اعتماد کمتری هستند).

مثال واقعی: در سیستم طبقه‌بندی مقالات علمی آرکایو (ArXiv)، واژه‌های بسیار تخصصی مانند "supersymmetry" ممکن است فقط در چند مقاله ظاهر شوند، اما از نظر χ^2 بسیار معنادار باشند. با این حال، این واژه‌ها ممکن است اطلاعات کمی برای طبقه‌بندی عمومی ارائه دهند، در حالی که MI به واژه‌های رایج‌تری مانند "quantum" یا "experiment" اهمیت بیشتری می‌دهد.

🌀 عملکرد در عمل: آیا تفاوت‌ها واقعاً مهم هستند؟

با وجود تفاوت‌ها، دقت طبقه‌بندی مجموعه‌های ویژگی منتخب با χ^2 و MI تفاوت سیستماتیک نشان نمی‌دهد. در اکثر مسائل طبقه‌بندی متن، چند شاخص قوی و بسیاری شاخص‌های ضعیف وجود دارد. تا زمانی که تمام شاخص‌های قوی و تعداد زیادی از شاخص‌های ضعیف انتخاب شوند، دقت انتظار می‌رود که خوب باشد. هر دو روش این کار را انجام می‌دهند.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان ایبی (eBay)، هر دو روش MI و χ^2 واژه‌هایی مانند "عالی"، "بد"، "ارسال سریع" و "بسته‌بندی" را به عنوان ویژگی‌های برتر شناسایی می‌کنند، حتی اگر اولویت‌بندی آن‌ها کمی متفاوت باشد. در نهایت، هر دو روش به دقت مشابهی در طبقه‌بندی نظرات مثبت و منفی منجر می‌شوند.

📈 شکل 13.8: مقایسه عملکرد MI و χ^2

شکل 13.8 انتخاب ویژگی MI و χ^2 را برای مدل چندجمله‌ای مقایسه می‌کند. اوج کارایی تقریباً برای هر دو روش یکسان است. χ^2 این اوج را دیرتر، در 300 ویژگی، به دست می‌آورد، احتمالاً به این دلیل که ویژگی‌های نادر اما بسیار معناداری را

که در ابتدا انتخاب می‌کند، تمام اسناد در کلاس را پوشش نمی‌دهند. با این حال، ویژگی‌های انتخابی بعدی (در محدوده 100-300) از کیفیت بهتری نسبت به ویژگی‌های انتخابی با MI برخوردارند.

تفسیر عملی: در سیستم طبقه‌بندی اخبار توییت، χ^2 ممکن است ابتدا واژه‌های نادر اما بسیار خاصی مانند "POTUS" (برای اخبار ریاست جمهوری) را انتخاب کند که فقط در چند توئیت ظاهر شده‌اند. این واژه‌ها به تنهایی پوشش کمی دارند، اما با افزایش تعداد ویژگی‌ها، χ^2 به واژه‌های رایج‌تر و کارآمدتر مانند "انتخابات" یا "رأی‌گیری" می‌رسد که پوشش بهتری ایجاد می‌کنند.

 محدودیت روش‌های حریصانه: مشکل ویژگی‌های زائد

هر سه روش - χ^2 ، MI، و مبتنی بر فراوانی - روش‌های حریصانه هستند. ممکن است ویژگی‌هایی را انتخاب کنند که اطلاعات افزایشی نسبت به ویژگی‌های قبلی ندارند. در شکل 13.7، kog به عنوان هفتمین واژه انتخاب شده است با وجود اینکه با ویژگی قبلی hong همبستگی بالایی دارد و بنابراین زائد است.

مثال واقعی: در سیستم طبقه‌بندی اخبار مربوط به هنگ کنگ، واژه‌های "hong" و "kong" تقریباً همیشه با هم ظاهر می‌شوند. اگر "hong" به عنوان یک ویژگی مهم انتخاب شده باشد، افزودن "kong" اطلاعات جدیدی به سیستم نمی‌دهد و فقط پیچیدگی را افزایش می‌دهد. این مشکل در شرکت‌هایی مانند بلومبرگ (Bloomberg) که باید اخبار مالی را به سرعت طبقه‌بندی کنند، اهمیت ویژه‌ای دارد.

اگرچه چنین زائدی می‌تواند تأثیر منفی بر دقت داشته باشد، روش‌های غیرحریصانه (به بخش 13.7 مراجعه کنید) به دلیل هزینه محاسباتی به ندرست در طبقه‌بندی متن استفاده می‌شوند. در عمل، سادگی و سرعت روش‌های حریصانه اغلب بر مزایای بالقوه روش‌های پیچیده‌تر ارجحیت دارد.

 تمرین 13.5: انتخاب ویژگی برای کلاس قهوه

فراوانی‌های زیر را برای کلاس coffee و چهار واژه در 100,000 سند اول مجموعه Reuters-RCV1 در نظر بگیرید:

واژه	N_{00}	N_{01}	N_{10}	N_{11}
brazil	98,012	102	1835	51
council	96,322	133	3525	20
producers	98,524	119	1118	34
roasted	99,824	143	23	10

دو واژه از این چهار واژه را بر اساس موارد زیر انتخاب کنید:

1. χ^2

2. اطلاعات متقابل

3. فراوانی

راهنمایی عملی: این تمرین به شما کمک می‌کند تا تفاوت‌های عملی بین این سه روش را درک کنید. برای مثال، "brazil" ممکن است بر اساس χ^2 امتیاز بالایی بگیرد به دلیل همبستگی قوی با کلاس قهوه، در حالی که "roasted" ممکن است بر اساس فراوانی امتیاز بالایی بگیرد به دلیل تکرار زیاد در اسناد مربوط به قهوه.

In [8]: # مقایسه روش‌های انتخاب ویژگی بر اساس تمرین 13.5

```
import math
import matplotlib.pyplot as plt
import numpy as np

def calculate_chi2_from_counts(N11, N10, N01, N00):
    """
    Nij بر اساس شمارش‌های  $\chi^2$  محاسبه امتیاز

    را بر اساس فرمول ساده‌شده محاسبه می‌کند  $\chi^2$  این تابع آماره
    .معیاری از انحراف فراوانی مشاهده شده از فراوانی مورد انتظار تحت فرض استقلال است  $\chi^2$ 

    پارامترها:
    N11: تعداد اسناد که شامل واژه و در کلاس هدف هستند
    N10: تعداد اسناد که شامل واژه اما در کلاس هدف نیستند
    N01: تعداد اسناد که در کلاس هدف هستند اما شامل واژه نیستند
    N00: تعداد اسناد که نه شامل واژه هستند و نه در کلاس هدف

     $\chi^2$  خروجی: مقدار
    """
    N = N11 + N10 + N01 + N00
    numerator = N * ((N11 * N00) - (N10 * N01))**2
    denominator = (N11 + N01) * (N11 + N10) * (N10 + N00) * (N01 + N00)

    if denominator == 0:
        return 0 # تغییر یافت 0 به return0 :اصلاح شده

    chi2_value = numerator / denominator
    return chi2_value

def calculate_mi_from_counts(N11, N10, N01, N00):
    """
    Nij بر اساس شمارش‌های (MI) محاسبه امتیاز اطلاعات متقابل

    این تابع اطلاعات متقابل را بر اساس تعریف نظریه اطلاعات محاسبه می‌کند
    .معیاری از اطلاعات مشترک بین دو متغیر تصادفی است MI

    پارامترها:
    N11: تعداد اسناد که شامل واژه و در کلاس هدف هستند
    N10: تعداد اسناد که شامل واژه اما در کلاس هدف نیستند
    N01: تعداد اسناد که در کلاس هدف هستند اما شامل واژه نیستند
    N00: تعداد اسناد که نه شامل واژه هستند و نه در کلاس هدف

    خروجی: مقدار اطلاعات متقابل
    """
    N = N11 + N10 + N01 + N00

    # محاسبه مقادیر جانبی
    N1_ = N10 + N11 # تعداد اسناد شامل واژه
    N0_ = N00 + N01 # تعداد اسناد بدون واژه
    N_1 = N01 + N11 # تعداد اسناد در کلاس هدف
    N_0 = N00 + N10 # تعداد اسناد در کلاس غیرهدف

    # محاسبه اطلاعات متقابل
    mi = 0.0
    if N11 > 0:
        mi += (N11 / N) * math.log2((N * N11) / (N1_ * N_1))
    if N01 > 0:
        mi += (N01 / N) * math.log2((N * N01) / (N0_ * N_1))
    if N10 > 0:
        mi += (N10 / N) * math.log2((N * N10) / (N1_ * N_0))
    if N00 > 0:
        mi += (N00 / N) * math.log2((N * N00) / (N0_ * N_0))

    return mi

def calculate_document_frequency(N11, N10):
    """
    (Document Frequency) محاسبه فراوانی سندی

    این تابع تعداد اسنادی که واژه در آن‌ها وجود دارد را محاسبه می‌کند
    .این معیار برای مدل برنولی مناسب است

    پارامترها:
    N11: تعداد اسناد که شامل واژه و در کلاس هدف هستند
    N10: تعداد اسناد که شامل واژه اما در کلاس هدف نیستند

    خروجی: فراوانی سندی واژه
    """
    return N11 + N10

def compare_feature_selection_methods(term_counts):
    """
    مقایسه روش‌های مختلف انتخاب ویژگی

    این تابع سه روش انتخاب ویژگی را مقایسه می‌کند:
    1.  $\chi^2$ 
    2. (MI) اطلاعات متقابل
    3. فراوانی سندی

    پارامترها:
    term_counts: برای هر واژه Nij دیکشنری شامل شمارش‌های

    خروجی: دیکشنری شامل امتیاز هر واژه برای هر روش
    """
    results = {}

    for term, counts in term_counts.items():
        N11 = counts['N11']
        N10 = counts['N10']
        N01 = counts['N01']
        N00 = counts['N00']
```

```
# محاسبه امتیازها برای هر روش
chi2_score = calculate_chi2_from_counts(N11, N10, N01, N00)
mi_score = calculate_mi_from_counts(N11, N10, N01, N00)
doc_freq = calculate_document_frequency(N11, N10)

results[term] = {
    'chi2': chi2_score,
    'mi': mi_score,
    'doc_freq': doc_freq,
    'N11': N11,
    'N10': N10,
    'N01': N01,
    'N00': N00
}

return results

def visualize_comparison(results):
    """
    مصورسازی مقایسه روش‌های انتخاب ویژگی

    این تابع نموداری برای مقایسه امتیاز واژه‌ها بر اساس روش‌های مختلف ایجاد می‌کند.
    """
    terms = list(results.keys())
    chi2_scores = [results[term]['chi2'] for term in terms]
    mi_scores = [results[term]['mi'] for term in terms]
    doc_freq_scores = [results[term]['doc_freq'] for term in terms]

    # ایجاد نمودار
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # MI و  $\chi^2$  نمودار مقایسه
    x = np.arange(len(terms))
    width = 0.35

    ax1.bar(x - width/2, chi2_scores, width, label='χ²')
    ax1.bar(x + width/2, mi_scores, width, label='Mutual Information')

    ax1.set_xlabel('Terms')
    ax1.set_ylabel('Score')
    ax1.set_title('Comparison of χ² and Mutual Information')
    ax1.set_xticks(x)
    ax1.set_xticklabels(terms)
    ax1.legend()

    # نمودار مقایسه فراوانی سندی
    ax2.bar(terms, doc_freq_scores, color='green')
    ax2.set_xlabel('Terms')
    ax2.set_ylabel('Document Frequency')
    ax2.set_title('Document Frequency Comparison')

    plt.tight_layout()
    plt.show()

# coffee داده‌های تمرین 13.5 برای کلاس
term_counts = {
    'brazil': {'N00': 98012, 'N01': 102, 'N10': 1835, 'N11': 51},
    'council': {'N00': 96322, 'N01': 133, 'N10': 3525, 'N11': 20},
    'producers': {'N00': 98524, 'N01': 119, 'N10': 1118, 'N11': 34},
    'roasted': {'N00': 99824, 'N01': 143, 'N10': 23, 'N11': 10}
}

# محاسبه و مقایسه امتیازها
results = compare_feature_selection_methods(term_counts)

# نمایش نتایج در جدول
print("="*90)
print("(تمرین 13.5) 'coffee' مقایسه امتیاز ویژگی‌ها برای کلاس")
print("="*90)
print(f"{'MI امتیاز':>15} | {'χ² امتیاز':>15} | {'فراوانی سندی':>15} | {'واژه':>15}")
print("-"*90)

for term, scores in results.items():
    print(f"term:<15 | {scores['doc_freq']:>15} | {scores['chi2']:>15.4f} | {scores['mi']:>15.6f}")

# انتخاب دو واژه برتر برای هر روش
print("\n" + "="*90)
print("دو واژه برتر انتخاب شده برای هر روش")
print("="*90)

# (i) بر اساس  $\chi^2$ 
sorted_chi2 = sorted(results.items(), key=lambda item: item[1]['chi2'], reverse=True)
selected_chi2 = [term for term, scores in sorted_chi2[:2]]
print(f"(i) χ²: {'', '.join(selected_chi2)}")

# (ii) بر اساس اطلاعات متقابل
sorted_mi = sorted(results.items(), key=lambda item: item[1]['mi'], reverse=True)
selected_mi = [term for term, scores in sorted_mi[:2]]
print(f"(ii) اطلاعات متقابل: {'', '.join(selected_mi)}")

# (iii) بر اساس فراوانی
sorted_freq = sorted(results.items(), key=lambda item: item[1]['doc_freq'], reverse=True)
selected_freq = [term for term, scores in sorted_freq[:2]]
print(f"(iii) فراوانی: {'', '.join(selected_freq)}")

# مصورسازی نتایج
visualize_comparison(results)

# تحلیل نتایج
print("\n" + "="*90)
print("تحلیل نتایج")
print("="*90)
```

```
# تحلیل چرا روش‌های مختلف واژه‌های متفاوتی را انتخاب می‌کنند
print("1. واژه‌های متفاوتی را انتخاب می‌کنند؟ MI و  $\chi^2$  چرا")
print("    - به واژه‌هایی که از نظر آماری معنادار هستند اهمیت می‌دهد، حتی اگر نادر باشند  $\chi^2$ ")
print("    - به واژه‌هایی که اطلاعات بیشتری ارائه می‌دهند اهمیت می‌دهد، صرف نظر از معناداری آماری MI")
print("    - دارد coffee دارد چون همبستگی قوی با کلاس  $\chi^2$  امتیاز بالایی در 'brazil'، برای مثال")
print("    - حتی اگر تعداد وقوع آن کم باشد")
print("    - «امتیاز بالایی در فراوانی سندی دارد چون در اسناد زیادی ظاهر می‌شود 'council'")
print("    - ضعیف باشد coffee حتی اگر همبستگی آن با کلاس")

print("\n2. تفاوت‌های عملی در انتخاب ویژگی")
print("    - برای تشخیص واژه‌های نادر اما بسیار خاص مناسب است  $\chi^2$ ")
print("    - برای تشخیص واژه‌هایی که اطلاعات عمومی و کاربردی ارائه می‌دهند مناسب است MI")
print("    - فراوانی سندی ساده‌ترین روش است اما ممکن است واژه‌های عمومی و غیرخاص را انتخاب کند")

print("\n3. کاربردهای عملی")
print("    - می‌تواند واژه‌های خاصی را شناسایی کند که در اسپم‌ها  $\chi^2$ ، در سیستم‌های فیلتر اسپم")
print("    - به ندرت ظاهر می‌شوند اما بسیار مشخص‌کننده هستند")
print("    - می‌تواند واژه‌های رایجی را انتخاب کند که در MI، در سیستم‌های طبقه‌بندی اخبار")
print("    - انواع مختلف اخبار ظاهر می‌شوند و اطلاعات عمومی ارائه می‌دهند")
print("    - در سیستم‌های طبقه‌بندی نظرات مشتریان، فراوانی سندی می‌تواند واژه‌هایی را")
print("    - انتخاب کند که در نظرات زیاد تکرار می‌شوند، حتی اگر لزوماً مشخص‌کننده نباشند")
```

=====

تمرین 13.5) 'coffee' مقایسه امتیاز ویژگی‌ها برای کلاس

=====

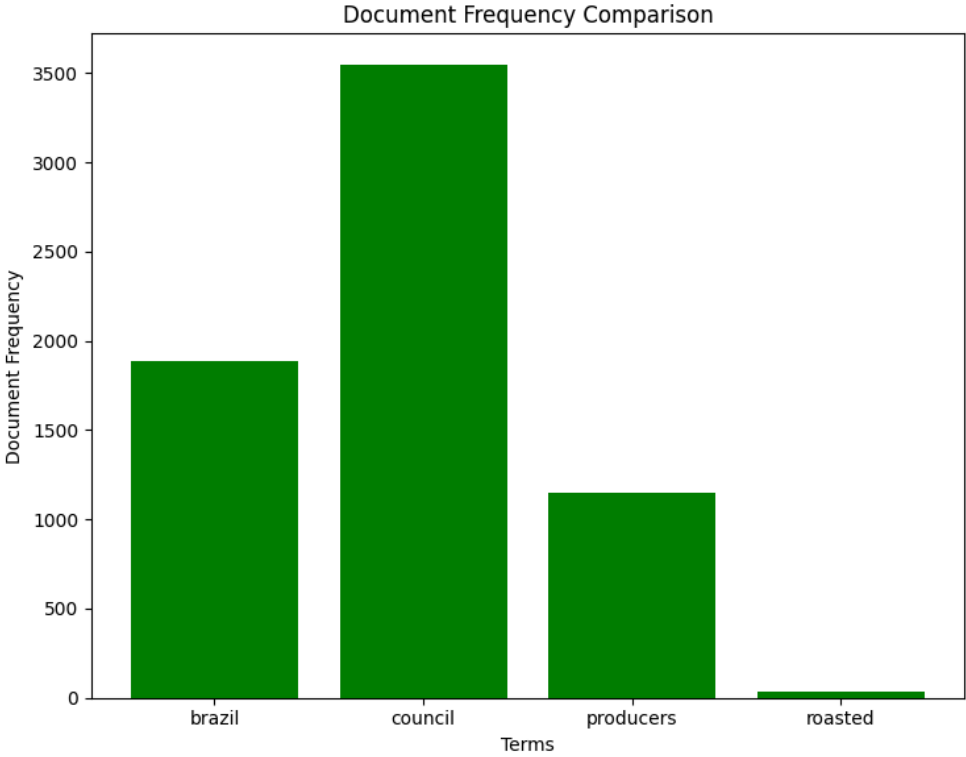
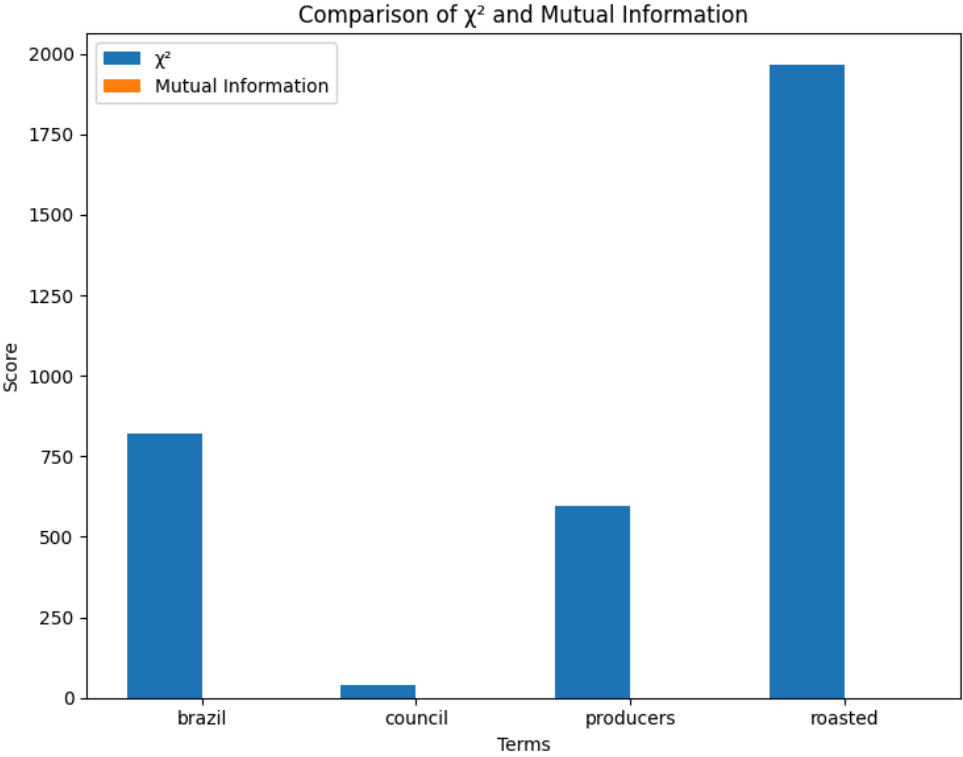
امتیاز	MI	χ^2 واژه	فراوانی سندی	امتیاز
brazil	0.001554	818.9387	1886	
council	0.000177	40.6741	3545	
producers	0.001048	596.0700	1152	
roasted	0.000648	1964.2933	33	

=====

دو واژه برتر انتخاب شده برای هر روش:

=====

(i) χ^2 : roasted, brazil
(ii) اطلاعات متقابل: brazil, producers
(iii) فراوانی: council, brazil



- =====
- تحلیل نتایج:
- =====
- واژه‌های متفاوتی را انتخاب می‌کنند؟ MI و χ^2 چرا
 - به واژه‌هایی که از نظر آماری معنادار هستند اهمیت می‌دهد، حتی اگر نادر باشند χ^2 .
 - به واژه‌هایی که اطلاعات بیشتری ارائه می‌دهند اهمیت می‌دهد، صرف نظر از معناداری آماری MI.
 - درد coffee دارد چون همبستگی قوی با کلاس χ^2 امتیاز بالایی در 'brazil'، برای مثال، حتی اگر تعداد وقوع آن کم باشد.
 - «امتیاز بالایی در فراوانی سندی دارد چون در اسناد زیادی ظاهر می‌شود 'council'، ضعیف باشد coffee حتی اگر همبستگی آن با کلاس.
 - تفاوت‌های عملی در انتخاب ویژگی:
 - برای تشخیص واژه‌های نادر اما بسیار خاص مناسب است χ^2 .
 - برای تشخیص واژه‌هایی که اطلاعات عمومی و کاربردی ارائه می‌دهند مناسب است MI.
 - فراوانی سندی ساده‌ترین روش است اما ممکن است واژه‌های عمومی و غیرخاص را انتخاب کند.
 - کاربردهای عملی:
 - می‌تواند واژه‌های خاصی را شناسایی کند که در اسپم‌ها χ^2 ، در سیستم‌های فیلتر اسپم، به ندرت ظاهر می‌شوند اما بسیار مشخص‌کننده هستند.
 - می‌تواند واژه‌های رایجی را انتخاب کند که در MI، در سیستم‌های طبقه‌بندی اخبار، انواع مختلف اخبار ظاهر می‌شوند و اطلاعات عمومی ارائه می‌دهند.
 - در سیستم‌های طبقه‌بندی نظرات مشتریان، فراوانی سندی می‌تواند واژه‌هایی را، انتخاب کند که در نظرات زیاد تکرار می‌شوند، حتی اگر لزوماً مشخص‌کننده نباشند.

13.6 ارزیابی طبقه‌بندی متن: چگونه بفهمیم الگوریتم ما چقدر خوب کار می‌کند؟



رویترز و چالش طبقه‌بندی اخبار: چگونه سرعت را با دقت ترکیب کنیم؟

تصور کنید در خبرگزاری رویترز کار می‌کنید و باید سیستمی طراحی کنید که هزاران خبر روزانه را به دسته‌های مختلفی (سیاست، ورزشی، اقتصادی، فرهنگی) اختصاص دهد. تیم رقیب شما در خبرگزاری گاردین از الگوریتم پیچیده‌تری استفاده می‌کند، اما شما باید سیستمی بسازید که هم سریع و هم قابل اعتماد باشد.

این دقیقاً همان چالشی است که در ارزیابی طبقه‌بندی متن با آن روبرو هستیم: چگونه می‌توانیم بفهمیم که الگوریتم ما چقدر خوب کار می‌کند، به خصوص وقتی باید بین سرعت و دقت تعادل برقرار کنیم؟

در گذشته، مجموعه **Reuters-21578** به عنوان معیار اصلی برای ارزیابی طبقه‌بندی متن استفاده می‌شد. این مجموعه شامل 21,578 مقاله خبری است که توسط Carnegie Group و Reuters جمع‌آوری و برچسب‌گذاری شد. این مجموعه پایه و اساس بسیاری از تحقیقات اولیه در زمینه طبقه‌بندی متن بود.

چالش طبقه‌بندی چندکلاسی: یک در مقابل همه

در ارزیابی طبقه‌بندی، ما به جای دقت (accuracy)، از معیارهای دقیق‌تری مانند **دقت (precision)**، **بازیابی (recall)** و **F1** استفاده می‌کنیم، به خصوص برای کلاس‌های کوچک که فراوانی کمی دارند.

چرا دقت به تنهایی کافی نیست؟ تصور کنید سیستمی که 99% اسناد را به درستی طبقه‌بندی کند، اما این کار را با گفتن "هیچ سندی در کلاس X نیست" انجام دهد. اگر کلاس X فقط 1% از اسناد را تشکیل دهد، این سیستم 99% دقیق است اما کاملاً بی‌فایده است!

مثال واقعی: در سیستم فیلتر محتوای یوتیوب، اگر سیستمی تمام محتوای مربوط به گروه‌های تروریستی را به درستی شناسایی کند اما در تشخیص محتوای مربوط به گروه‌های کوچک و جدید شکست بخورد، دقت بالای آن گمراه‌کننده است. در چنین مواردی، معیارهای F1 و بازیابی بهتر عملکرد واقعی سیستم را نشان می‌دهند.

🏠 میانگین‌گیری کلان در مقابل ریز: چگونه عملکرد کلی را گزارش کنیم؟

وقتی با چندین طبقه‌بند دو کلاسی کار می‌کنیم (مانند Reuters-21578 با 118 کلاس)، اغلب می‌خواهیم یک معیار تجمعی محاسبه کنیم که نتایج طبقه‌بندهای فردی را ترکیب کند. دو روش اصلی برای این کار وجود دارد:

- **میانگین‌گیری کلان (Macroaveraging):** میانگین ساده روی کلاس‌ها که به هر کلاس وزن برابر می‌دهد.
- **میانگین‌گیری ریز (Microaveraging):** تصمیمات طبقه‌بندی را در تمام کلاس‌ها ادغام کرده و سپس معیار را روی جدول ترکیبی محاسبه می‌کند.

تفاوت بین این دو روش می‌تواند زیاد باشد. میانگین‌گیری کلان به هر کلاس وزن برابر می‌دهد، در حالی که میانگین‌گیری ریز به هر تصمیم طبقه‌بندی وزن برابر می‌دهد. از آنجایی که معیار F1 موارد منفی را نادیده می‌گیرد و عمدتاً توسط تعداد موارد مثبت تعیین می‌شود، کلاس‌های بزرگ در میانگین‌گیری ریز تسلط دارند.

📊 جدول 13.8: تفاوت میانگین‌گیری کلان و ریز در عمل

جدول 13.8 تفاوت بین میانگین‌گیری کلان و ریز را با یک مثال ساده نشان می‌دهد:

► **Table 13.8** Macro- and microaveraging. “Truth” is the true class and “call” the decision of the classifier. In this example, macroaveraged precision is $[10/(10+10) + 90/(10+90)]/2 = (0.5+0.9)/2 = 0.7$. Microaveraged precision is $100/(100+20) \approx 0.83$.

class 1			class 2			pooled table		
	truth: yes	truth: no		truth: yes	truth: no		truth: yes	truth: no
call: yes	10	10	call: yes	90	10	call: yes	100	20
call: no	10	970	call: no	10	890	call: no	20	1860

در این مثال، میانگین‌گیری کلان دقت $= 2/[(90+10)/90 + (10+10)/10] = 2/(0.9 + 0.5) = 0.7$ است، در حالی که میانگین‌گیری ریز دقت $= (20+100)/100 = 0.83 \approx$ است. نتایج ریز به عملکرد کلاس بزرگ (کلاس 2) نزدیک‌تر است، در حالی که نتایج کلان به هر دو کلاس وزن برابر می‌دهد.

► **Table 13.9** Text classification effectiveness numbers on Reuters-21578 for F₁ (in percent). Results from Li and Yang (2003) (a), Joachims (1998) (b: kNN) and Dumais et al. (1998) (b: NB, Rocchio, trees, SVM).

(a)	NB	Rocchio	kNN	SVM	
micro-avg-L (90 classes)	80	85	86	89	
macro-avg (90 classes)	47	59	60	60	

(b)	NB	Rocchio	kNN	trees	SVM
earn	96	93	97	98	98
acq	88	65	92	90	94
money-fx	57	47	78	66	75
grain	79	68	82	85	95
crude	80	70	86	85	89
trade	64	65	77	73	76
interest	65	63	74	67	78
ship	85	49	79	74	86
wheat	70	69	77	93	92
corn	65	48	78	92	90
micro-avg (top 10)	82	65	82	88	92
micro-avg-D (118 classes)	75	62	n/a	n/a	87

🏆 جدول 13.9: بیز ساده در مقابل رقبا

جدول 13.9 نتایج ارزیابی بیز ساده را در مقایسه با سایر طبقه‌بندها نشان می‌دهد. بیز ساده F1 میانگین‌گیری ریز 80% دارد که 9% کمتر از SVM (89%) است. با این حال، در کلاس‌های کوچک، عملکرد بیز ساده بسیار ضعیف‌تر است؛ F1 میانگین‌گیری کلان آن 13% کمتر از SVM است.

تفسیر عملی: این نتایج نشان می‌دهند که اگرچه بیز ساده به اندازه SVM مؤثر نیست، اما سادگی و کارایی آن را به یک گزینه قوی برای مسائل طبقه‌بندی متن تبدیل کرده است. در دنیای واقعی، عواملی دیگری مانند ماهیت داده‌ها، تغییر مفهوم در طول زمان، و خطا در داده‌ها می‌توانند بر عملکرد طبقه‌بندها تأثیر گذارند.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان آمازون، بیز ساده ممکن است عملکرد خوبی در دسته‌بندی نظرات مثبت و منفی داشته باشد، اما در تشخیص زیردسته‌های خاص (مانند "کیفیت بسته‌بندی" در مقابل "کیفیت محصول") ضعیف عمل کند. در چنین مواردی، یک سیستم ترکیبی که از بیز ساده برای طبقه‌بندی کلی و از یک طبقه‌بند تخصصی برای زیردسته‌های خاص استفاده می‌کند، ممکن است نتایج بهتری ارائه دهد.

⚠ قوانین ارزیابی: چگونه نتایج معتبر بگیریم؟

در ارزیابی طبقه‌بندی، جداسازی دقیق بین مجموعه آموزشی، توسعه و آزمون بسیار مهم است. هرگز نباید در حین توسعه سیستم به داده‌های آزمون دسترسی داشت یا روی آن‌ها برنامه اجرا کرد. در عوض، یک مجموعه توسعه برای آزمون در نظر گرفته شده و سیستم روی آن تنظیم می‌شود.

این مجموعه توسعه همچنین **داده‌های نگه‌داشته** (held-out data) نامیده می‌شود. این رویکرد تضمین می‌کند که نتایج گزارش‌شده نماینده عملکرد واقعی سیستم در دنیای واقعی باشند، نه نتیجه تنظیم بیش از حد روی داده‌های آزمون.

مثال واقعی: در مسابقات کاگل (Kaggle) که شرکت‌ها در آن الگوریتم‌های خود را برای مسائل داده‌کاوی آزمایش می‌کنند، این قوانین به شدت رعایت می‌شود. شرکت‌کنندگان فقط به مجموعه آموزشی دسترسی دارند و نتایج الگوریتم‌هایشان فقط روی مجموعه آزمون ارزیابی می‌شود که هرگز در حین توسعه دیده نشده است. این تضمین می‌کند که برنده مسابقه واقعاً بهترین الگوریتم را دارد، نه الگوریتمی که بیش از حد روی داده‌های آزمون تنظیم شده است.

In [11]: پیاده‌سازی معیارهای ارزیابی طبقه‌بندی متن #

```
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict

def calculate_precision(y_true, y_pred, class_label):
    """
    برای یک کلاس خاص (Precision) محاسبه دقت

    دقت نسبت اسنادی که به درستی به کلاس اختصاص داده شده‌اند به کل اسنادی که به آن کلاس اختصاص داده شده‌اند.

    پارامترها:
    y_true: لیست برچسب‌های واقعی
    y_pred: لیست برچسب‌های پیش‌بینی شده
    class_label: کلاسی که می‌خواهیم دقت را برای آن محاسبه کنیم

    خروجی: مقدار دقت برای کلاس مورد نظر
    """
    true_positives = sum(1 for true, pred in zip(y_true, y_pred)
                        if true == class_label and pred == class_label)
    predicted_positives = sum(1 for pred in y_pred if pred == class_label)

    if predicted_positives == 0:
        return 0.0

    return true_positives / predicted_positives

def calculate_recall(y_true, y_pred, class_label):
    """
    برای یک کلاس خاص (Recall) محاسبه بازیابی

    بازیابی نسبت اسنادی که به درستی به کلاس اختصاص داده شده‌اند به کل اسنادی که باید به آن کلاس اختصاص داده می‌شدند.

    پارامترها:
    y_true: لیست برچسب‌های واقعی
    y_pred: لیست برچسب‌های پیش‌بینی شده
    class_label: کلاسی که می‌خواهیم بازیابی را برای آن محاسبه کنیم

    خروجی: مقدار بازیابی برای کلاس مورد نظر
    """
    true_positives = sum(1 for true, pred in zip(y_true, y_pred)
                        if true == class_label and pred == class_label)
    actual_positives = sum(1 for true in y_true if true == class_label)

    if actual_positives == 0:
        return 0.0

    return true_positives / actual_positives

def calculate_f1(precision, recall):
    """
    F1 محاسبه معیار

    میانگین هارمونیک دقت و بازیابی است و معیاری متعادل برای ارزیابی عملکرد طبقه‌بند F1.

    پارامترها:
    precision: مقدار دقت
    recall: مقدار بازیابی

    خروجی: مقدار F1
    """
    if precision + recall == 0:
```

```
        return 0.0
    return 2 * (precision * recall) / (precision + recall)

def calculate_macro_avg(y_true, y_pred, classes):
    """
    برای معیارها (Macro-averaging) محاسبه میانگین کلان

    میانگین کلان به هر کلاس وزن برابر می‌دهد و برای ارزیابی عملکرد روی کلاس‌های کوچک مناسب است.

    پارامترها:
    y_true: لیست برچسب‌های واقعی
    y_pred: لیست برچسب‌های پیش‌بینی شده
    classes: لیست کلاس‌ها

    خروجی: دیکشنری شامل معیارهای میانگین کلان
    """
    precision_sum = 0.0
    recall_sum = 0.0
    f1_sum = 0.0

    for c in classes:
        precision = calculate_precision(y_true, y_pred, c)
        recall = calculate_recall(y_true, y_pred, c)
        f1 = calculate_f1(precision, recall)

        precision_sum += precision
        recall_sum += recall
        f1_sum += f1

    n_classes = len(classes)
    return {
        'precision': precision_sum / n_classes,
        'recall': recall_sum / n_classes,
        'f1': f1_sum / n_classes
    }

def calculate_micro_avg(y_true, y_pred, classes):
    """
    برای معیارها (Micro-averaging) محاسبه میانگین ریز

    میانگین ریز تصمیمات طبقه‌بندی را در تمام کلاس‌ها ادغام کرده و سپس معیار را محاسبه می‌کند.
    این معیار بیشتر تحت تأثیر کلاس‌های بزرگ قرار می‌گیرد.

    پارامترها:
    y_true: لیست برچسب‌های واقعی
    y_pred: لیست برچسب‌های پیش‌بینی شده
    classes: لیست کلاس‌ها

    خروجی: دیکشنری شامل معیارهای میانگین ریز
    """
    # ایجاد جدول ترکیبی برای تمام کلاس‌ها
    true_positives = 0
    predicted_positives = 0
    actual_positives = 0

    for c in classes:
        true_positives += sum(1 for true, pred in zip(y_true, y_pred)
                              if true == c and pred == c)
        predicted_positives += sum(1 for pred in y_pred if pred == c)
        actual_positives += sum(1 for true in y_true if true == c)

    # محاسبه معیارهای ریز
    precision = true_positives / predicted_positives if predicted_positives > 0 else 0.0
    recall = true_positives / actual_positives if actual_positives > 0 else 0.0
    f1 = calculate_f1(precision, recall)

    return {
        'precision': precision,
        'recall': recall,
        'f1': f1
    }

def visualize_confusion_matrix(y_true, y_pred, classes):
    """
    مصورسازی ماتریس درهم‌ریختگی برای نمایش عملکرد طبقه‌بند

    این تابع ماتریس درهم‌ریختگی را نمایش می‌دهد که نشان می‌دهد چگونه طبقه‌بند
    اسناد را به کلاس‌های مختلف اختصاص داده است.

    پارامترها:
    y_true: لیست برچسب‌های واقعی
    y_pred: لیست برچسب‌های پیش‌بینی شده
    classes: لیست کلاس‌ها

    خروجی: نمایش ماتریس درهم‌ریختگی
    """
    n_classes = len(classes)
    # ایجاد ماتریس درهم‌ریختگی
    confusion_matrix = np.zeros((n_classes, n_classes), dtype=int)

    # پر کردن ماتریس
    class_to_index = {c: i for i, c in enumerate(classes)}
    for true, pred in zip(y_true, y_pred):
        true_idx = class_to_index[true]
        pred_idx = class_to_index[pred]
        confusion_matrix[true_idx, pred_idx] += 1

    # نمایش ماتریس
    # خط: ax.figure.colorbar(im)
    # صحیح: fig.colorbar(im)
    fig, ax = plt.subplots(figsize=(8, 6))
    im = ax.imshow(confusion_matrix, interpolation='nearest', cmap=plt.cm.Blues)
    fig.colorbar(im)
```

```
# تنظیم محورها
ax.set_xticks(np.arange(n_classes))
ax.set_yticks(np.arange(n_classes))
ax.set_xticklabels(classes, rotation=45)
ax.set_yticklabels(classes)
ax.set_xlabel('Predicted Label')
ax.set_ylabel('True Label')
ax.set_title('Confusion Matrix')

# اضافه کردن اعداد به هر سلول
for i in range(n_classes):
    for j in range(n_classes):
        ax.text(j, i, str(confusion_matrix[i, j]),
                ha="center", va="center", color="red")

plt.tight_layout()
plt.show()

def compare_macro_micro(y_true, y_pred, classes):
    """
    مقایسه میانگین‌گیری کلان و ریز با مثال‌های واقعی

    این تابع تفاوت بین دو روش میانگین‌گیری را با مثال‌های واقعی نشان می‌دهد
    """
    # محاسبه معیارها
    macro_avg = calculate_macro_avg(y_true, y_pred, classes)
    micro_avg = calculate_micro_avg(y_true, y_pred, classes)

    # محاسبه دقت کلی
    accuracy = sum(1 for true, pred in zip(y_true, y_pred) if true == pred) / len(y_true)

    print("مقایسه روش‌های ارزیابی:")
    print("="*50)
    print(f"دقت کلی (Accuracy): {accuracy:.4f}")
    print(f"میانگین کلان دقت: {macro_avg['precision']:.4f}")
    print(f"میانگین ریز دقت: {micro_avg['precision']:.4f}")
    print()
    print(f"میانگین کلان بازیابی: {macro_avg['recall']:.4f}")
    print(f"میانگین ریز بازیابی: {micro_avg['recall']:.4f}")
    print()
    print(f"میانگین کلان F1: {macro_avg['f1']:.4f}")
    print(f"میانگین ریز F1: {micro_avg['f1']:.4f}")

    # نمایش ماتریس درهم‌ریختگی
    visualize_confusion_matrix(y_true, y_pred, classes)

def demonstrate_accuracy_problem():
    """
    برای مجموعه‌های نامتعادل (Accuracy) نمایش مشکل دقت

    این مثال نشان می‌دهد که چرا دقت ممکن است معیار گمراه‌کننده باشد
    """
    print("برای مجموعه‌های نامتعادل (Accuracy) نمایش مشکل دقت:")
    print("="*60)

    # B و 1% در کلاس A مجموعه نامتعادل: 99% اسناد در کلاس
    y_true = ['A'] * 99 + ['B'] # 100 سند با برچسب واقعی
    y_pred_perfect = ['A'] * 99 + ['B'] # طبقه‌بند کامل
    y_pred_naive = ['A'] * 100 # A طبقه‌بند ساده که همیشه

    # محاسبه دقت برای هر طبقه‌بند
    accuracy_perfect = sum(1 for true, pred in zip(y_true, y_pred_perfect) if true == pred) / len(y_true)
    accuracy_naive = sum(1 for true, pred in zip(y_true, y_pred_naive) if true == pred) / len(y_true)

    print(f"دقت طبقه‌بند کامل: {accuracy_perfect:.2f}")
    print(f"دقت طبقه‌بند ساده (A همیشه): {accuracy_naive:.2f}")
    print()
    print("اِ! را به درستی تشخیص نمی‌دهد، دقت آن 99% است B با وجود اینکه طبقه‌بند ساده هیچ سند")
    print("این مثال نشان می‌دهد که چرا دقت به تنهایی برای مجموعه‌های نامتعادل معیار مناسبی نیست.")
    print()

    # برای هر طبقه‌بند F1 محاسبه
    macro_perfect = calculate_macro_avg(y_true, y_pred_perfect, ['A', 'B'])
    macro_naive = calculate_macro_avg(y_true, y_pred_naive, ['A', 'B'])

    print("تفاوت را به خوبی نشان می‌دهد F1 اما معیار")
    print(f"F1 طبقه‌بند کامل: {macro_perfect['f1']:.4f}")
    print(f"F1 طبقه‌بند ساده: {macro_naive['f1']:.4f}")
    print()
    print("را به درستی تشخیص نمی‌دهد B صفر است، زیرا هیچ سند B طبقه‌بند ساده برای کلاس F1")

# مثال 1: مقایسه طبقه‌بندهای مختلف بر اساس معیارهای ارزیابی
print("مثال 1: مقایسه طبقه‌بندهای مختلف بر اساس معیارهای ارزیابی")
print("="*60)

# داده‌های شبیه‌سازی شده از جدول 13.9 کتاب
# فرض می‌کنیم 1000 سند با توزیع کلاس‌های زیر داریم
# سند earn: 300 کلاس
# سند acq: 200 کلاس
# سند money-fx: 100 کلاس
# سند grain: 150 کلاس
# سند crude: 120 کلاس
# سند corn: 80 کلاس
# سند trade: 50 کلاس

# ایجاد داده‌های واقعی و پیش‌بینی شده برای دو طبقه‌بند مختلف
np.random.seed(42)
classes = ['earn', 'acq', 'money-fx', 'grain', 'crude', 'corn', 'trade']

# ایجاد داده‌های واقعی
y_true = []
for c, count in [('earn', 300), ('acq', 200), ('money-fx', 100),
```

```
( 'grain', 150), ( 'crude', 120), ( 'corn', 80), ( 'trade', 50)]:
y_true.extend([c] * count)

# عملکرد بهتر روی کلاس‌های بزرگ - (Naive Bayes) ایجاد پیش‌بینی‌های طبقه‌بند 1
y_pred_nb = []
for c, count in [( 'earn', 300), ( 'acq', 200), ( 'money-fx', 100),
                  ( 'grain', 150), ( 'crude', 120), ( 'corn', 80), ( 'trade', 50)]:
    if c in [ 'earn', 'acq']: # عملکرد خوب روی کلاس‌های بزرگ
        y_pred_nb.extend([c] * int(count * 0.9)) # دقت 90%
    else: # عملکرد ضعیف‌تر روی کلاس‌های کوچک
        y_pred_nb.extend([c] * int(count * 0.6)) # دقت 60%

# عملکرد بهتر روی کلاس‌های کوچک - (SVM) ایجاد پیش‌بینی‌های طبقه‌بند 2
y_pred_svm = []
for c, count in [( 'earn', 300), ( 'acq', 200), ( 'money-fx', 100),
                  ( 'grain', 150), ( 'crude', 120), ( 'corn', 80), ( 'trade', 50)]:
    if c in [ 'earn', 'acq']: # عملکرد خوب روی کلاس‌های بزرگ
        y_pred_svm.extend([c] * int(count * 0.85)) # دقت 85%
    else: # عملکرد بهتر روی کلاس‌های کوچک
        y_pred_svm.extend([c] * int(count * 0.75)) # دقت 75%

# محاسبه و مقایسه معیارها
print("ارزیابی Naive Bayes:")
macro_nb = calculate_macro_avg(y_true, y_pred_nb, classes)
micro_nb = calculate_micro_avg(y_true, y_pred_nb, classes)
print(f"میانگین کلان F1: {macro_nb['f1']:.4f}")
print(f"میانگین ریز F1: {micro_nb['f1']:.4f}")
print()

print("ارزیابی SVM:")
macro_svm = calculate_macro_avg(y_true, y_pred_svm, classes)
micro_svm = calculate_micro_avg(y_true, y_pred_svm, classes)
print(f"میانگین کلان F1: {macro_svm['f1']:.4f}")
print(f"میانگین ریز F1: {micro_svm['f1']:.4f}")
print()

# مقایسه دو طبقه‌بند
print("مقایسه دو طبقه‌بند:")
print(f"تفاوت در میانگین کلان F1: {macro_svm['f1'] - macro_nb['f1']:.4f}")
print(f"تفاوت در میانگین ریز F1: {micro_svm['f1'] - micro_nb['f1']:.4f}")

# نمایش مشکل دقت برای مجموعه‌های نامتعادل
demonstrate_accuracy_problem()

# مثال 2: مقایسه میانگین‌گیری کلان و ریز
print("\nمثال 2: مقایسه میانگین‌گیری کلان و ریز")
print("=*60)

# ایجاد یک مثال ساده با دو کلاس برای نمایش تفاوت میانگین‌گیری کلان و ریز
# کلاس 1: 100 سند (کلاس بزرگ)
# کلاس 2: 20 سند (کلاس کوچک)
y_true = [ 'class1' ] * 100 + [ 'class2' ] * 20

# داده‌های واقعی
y_true_example = [ 'class1' ] * 100 + [ 'class2' ] * 20

# پیش‌بینی‌ها
# class1 درست برای 90
# class2 درست برای 5
# (پیش‌بینی class1 واقعی، class2 اشتباه 10)
# (پیش‌بینی class2 واقعی، class1 اشتباه 15)
y_pred_example = [ 'class1' ] * 90 + [ 'class2' ] * 5 + [ 'class1' ] * 10 + [ 'class1' ] * 15 # طول 120

compare_macro_micro(y_true_example, y_pred_example, [ 'class1', 'class2' ])

print("\nنتیجه‌گیری:")
print("=*60)
print("1. میانگین‌گیری کلان به هر کلاس وزن برابر می‌دهد و برای ارزیابی عملکرد روی کلاس‌های کوچک مناسب است.")
print("2. میانگین‌گیری ریز به هر تصمیم طبقه‌بندی وزن برابر می‌دهد و بیشتر تحت تأثیر کلاس‌های بزرگ قرار می‌گیرد.")
print("3. در مجموعه‌های نامتعادل، میانگین‌گیری کلان معیار قابل اعتمادتری برای ارزیابی عملکرد کلی است.")
```

مثال 1: مقایسه طبقه‌بندهای مختلف بر اساس معیارهای ارزیابی
=====

ارزیابی Naive Bayes:
F1 میانگین کلان: 0.2660
F1 میانگین ریز: 0.4914

ارزیابی SVM:
F1 میانگین کلان: 0.2409
F1 میانگین ریز: 0.4358

مقایسه دو طبقه‌بند:
F1 تفاوت در میانگین کلان: -0.0251
F1 تفاوت در میانگین ریز: -0.0556
برای مجموعه‌های نامتعادل (Accuracy) نمایش مشکل دقت
=====
دقت طبقه‌بند کامل: 1.00
0.99: (را پیش‌بینی می‌کند A همیشه) دقت طبقه‌بند ساده

!را به درستی تشخیص نمی‌دهد، دقت آن 99% است B با وجود اینکه طبقه‌بند ساده هیچ سند
.این مثال نشان می‌دهد که چرا دقت به تنهایی برای مجموعه‌های نامتعادل معیار مناسبی نیست

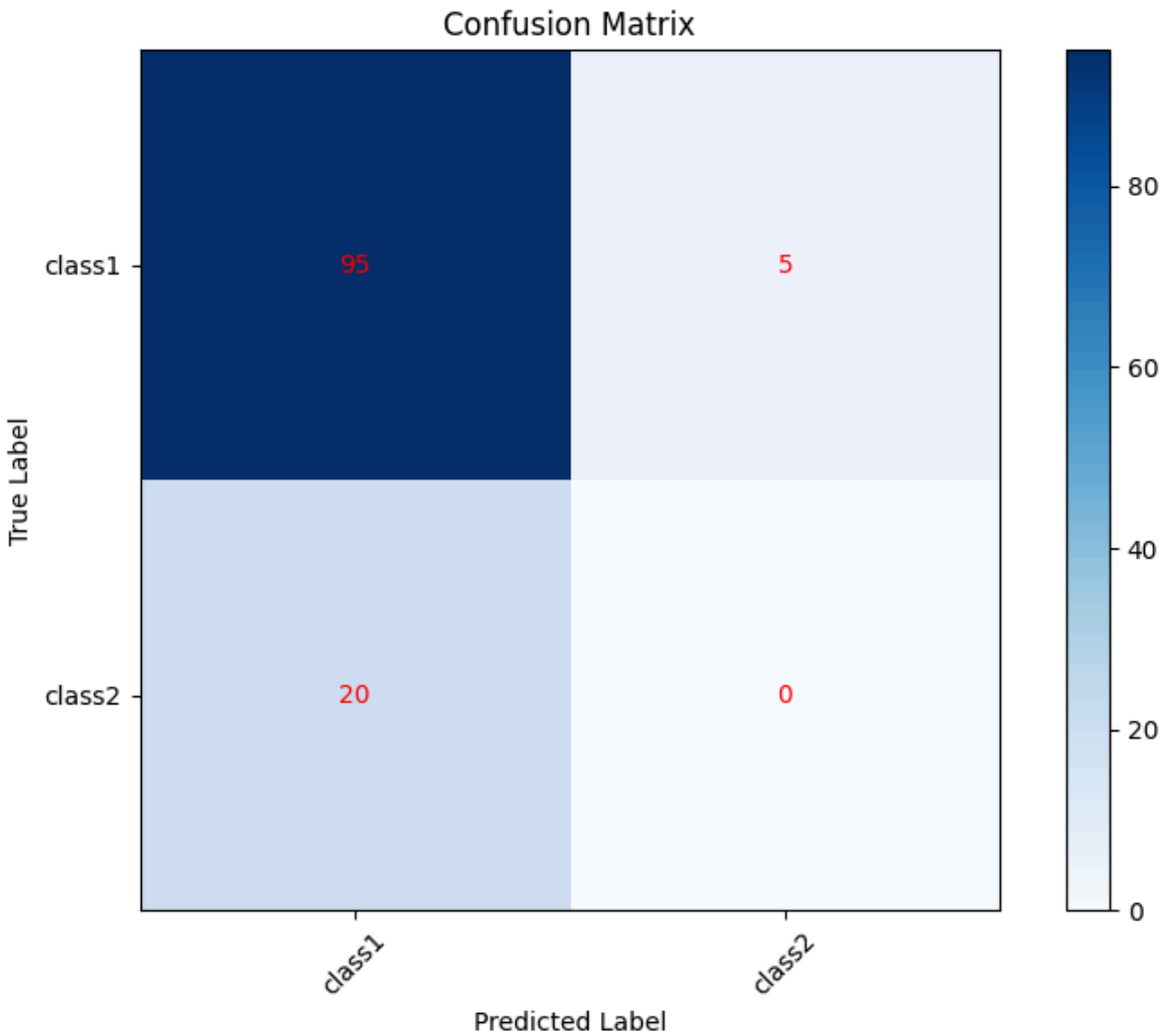
تفاوت را به خوبی نشان می‌دهد F1 اما معیار
طبقه‌بند کامل: 1.0000 F1
طبقه‌بند ساده: 0.4975 F1

.را به درستی تشخیص نمی‌دهد B صفر است، زیرا هیچ سند B طبقه‌بند ساده برای کلاس F1

مثال 2: مقایسه میانگین‌گیری کلان و ریز
=====
مقایسه روش‌های ارزیابی
=====
دقت کلی (Accuracy): 0.7917
میانگین کلان دقت: 0.4130
میانگین ریز دقت: 0.7917

میانگین کلان بازایی: 0.4750
میانگین ریز بازایی: 0.7917

F1 میانگین کلان: 0.4419
F1 میانگین ریز: 0.7917



نتیجه‌گیری:
=====
1. میانگین‌گیری کلان به هر کلاس وزن برابر می‌دهد و برای ارزیابی عملکرد روی کلاس‌های کوچک مناسب است.
2. میانگین‌گیری ریز به هر تصمیم طبقه‌بندی وزن برابر می‌دهد و بیشتر تحت تأثیر کلاس‌های بزرگ قرار می‌گیرد.
3. در مجموعه‌های نامتعادل، میانگین‌گیری کلان معیار قابل اعتمادتری برای ارزیابی عملکرد کلی است.

🧠 جمع‌بندی فصل ۱۳: طبقه‌بندی متن و مدل Naive Bayes

در این فصل:

- مفهوم طبقه‌بندی متن را معرفی کردیم و آن را به‌عنوان تعمیمی از پرسمان‌های ایستا (standing queries) تحلیل کردیم.
- کاربردهای متنوع طبقه‌بندی در IR را بررسی کردیم: از فیلتر کردن اسپم و محتوای حساس تا جست‌وجوی عمودی و تشخیص احساسات.

- مدل یادگیری نظارت‌شده را تعریف کردیم: تابع طبقه‌بندی $\gamma : X \rightarrow C$ با استفاده از مجموعه آموزش برچسب‌خورده.
 - مدل Naive Bayes را به‌عنوان یک روش آماری ساده و مؤثر معرفی کردیم و فرمول MAP را برای انتخاب کلاس بهینه استخراج کردیم.
 - دو نسخه مدل NB را بررسی کردیم: مدل چندجمله‌ای (multinomial) و مدل برنولی (Bernoulli) – با تفاوت در نحوه استفاده از فراوانی واژه و غیاب واژه‌ها.
 - مفهوم smoothing را برای مقابله با داده‌های پراکنده معرفی کردیم و از نرم‌سازی لاپلاس (add-one) استفاده کردیم.
 - ارتباط مدل NB با مدل زبانی چندجمله‌ای را نشان دادیم و پیچیدگی زمانی آموزش و تست را تحلیل کردیم.
- این فصل پایه‌های **یادگیری آماری برای طبقه‌بندی متن** را فراهم می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:
- طبقه‌بندی برداری و ماشین‌های بردار پشتیبان (فصل ۱۴ و ۱۵)
 - بازخورد مرتبط بودن و یادگیری تعاملی (فصل ۱۵)
- آشنا خواهیم شد – که همگی بر همین پایه مدل‌های ساده ولی قدرتمند آماری بنا شده‌اند.

End of Chapter Exercises

مسئله 13.1: مفاهیم اولیه طبقه‌بندی متن (The Text Classification Problem)

هدف اصلی در مسئله **طبقه‌بندی متون** (Text Classification) چیست؟ با ذکر دو مثال از کاربردهای روزمره در دنیای وب، ورودی و خروجی یک سیستم طبقه‌بند متن را مشخص کنید.

مسئله 13.2: فرض استقلال نایو بیز (Naive Bayes Independence Assumption)

چرا به الگوریتم بیز، کلمه «**ساده‌لوحانه**» (Naive) اطلاق می‌شود؟ فرض **استقلال شرطی** در این الگوریتم را توضیح داده و بیان کنید چرا در زبان طبیعی این فرض معمولاً نقض می‌شود.

مسئله 13.3: احتمالات پیشین (Prior Probabilities)

در فرمول قضیه بیز، احتمال پیشین (Prior Probability) یک کلاس چگونه محاسبه می‌شود؟ نقش آن در تصمیم‌گیری نهایی الگوریتم چیست؟

مسئله 13.4: چندجمله‌ای در برابر برنولی (Multinomial vs Bernoulli)

تفاوت اصلی در نحوه نمایش یک سند متنی بین مدل چندجمله‌ای (Multinomial) و مدل برنولی (Bernoulli) چیست؟ کدام مدل به تکرار کلمات اهمیت می‌دهد؟

مسئله 13.5: مشکل فرکانس صفر (Zero Frequency Problem)

اگر در هنگام طبقه‌بندی یک سند جدید، کلمه‌ای وجود داشته باشد که در مرحله آموزش (**Training**) در کلاس خاصی دیده نشده باشد، چه اتفاقی برای احتمال کل آن کلاس می‌افتد؟

مسئله 13.6: هموارسازی لاپلاس (Laplace Smoothing)

توضیح دهید که تکنیک هموارسازی لاپلاس (**Laplace Smoothing**) چگونه مشکل فرکانس صفر (**Zero Frequency**) را حل می‌کند؟ پارامتر آلفا (**Alpha**) در این فرمول چه نقشی دارد؟

مسئله 13.7: مدل زبانی مونوگرام (Unigram Language Model)

ارتباط بین مدل نایو بیز چندجمله‌ای و مدل زبانی مونوگرام (**Unigram**) چیست؟ چگونه می‌توان نایو بیز را به عنوان یک تولیدکننده متن بر اساس مونوگرام‌ها در نظر گرفت؟

مسئله 13.8: فضای ویژگی‌ها (Feature Space)

در کانتکست طبقه‌بندی متون، «دایره لغات» (**Vocabulary**) چه نقشی در تعیین ابعاد فضای ویژگی‌ها دارد؟

مسئله 13.9: محاسبه مدل برنولی (Bernoulli Model Calculation)

در مدل برنولی (**Bernoulli Model**) ، عدم حضور یک کلمه در سند چه تاثیری در محاسبه احتمال نهایی آن کلاس دارد؟ فرمول آن را بنویسید.

مسئله 13.10: پیچیدگی زمانی آموزش (Training Time Complexity)

نشان دهید که پیچیدگی زمانی مرحله آموزش (**Training**) در مدل نایو بیز چندجمله‌ای $O(|D| + |V|)$ است (که در آن $|D|$ مجموع طول تمام اسناد و $|V|$ اندازه دایره لغات است).

مسئله 13.11: پیچیدگی زمانی تست (Testing Time Complexity)

چرا پیچیدگی زمانی در مرحله تست (**Testing**) برای یک سند جدید با طول L_d در مدل نایو بیز معادل $O(|C| \times L_d)$ است؟ ($|C|$ تعداد کلاس‌ها است).

مسئله 13.12: لگاریتم احتمالات (Log Probabilities)

دلیل تبدیل ضرب احتمالات (**Multiplication of Probabilities**) به جمع لگاریتم‌ها (**Sum of Logarithms**) در پیاده‌سازی عملی الگوریتم **Naive Bayes** چیست؟

مسئله 13.13: تاثیر طول سند (Document Length Impact)

چرا در مدل نایو بیز چندجمله‌ای استاندارد، اسناد طولانی‌تر تمایل دارند مقادیر احتمال شرطی نهایی بسیار کوچکتری (نزدیک به صفر) تولید کنند؟

مسئله 13.14: انتخاب ویژگی مبتنی بر فرکانس (Frequency-based Feature Selection)

در روش انتخاب ویژگی مبتنی بر فرکانس (Frequency-based Feature Selection)، اگر کلماتی با بالاترین تکرار را انتخاب کنیم، با چه خطری مواجه می‌شویم و راه حل آن (استفاده از لیست توقف (Stop List)) چیست؟

مسئله 13.15: مفهوم اطلاعات متقابل (Mutual Information Concept)

معیار اطلاعات متقابل (Mutual Information) به زبان ساده چه چیزی را در مورد رابطه بین یک کلمه و یک کلاس اندازه‌گیری می‌کند؟

مسئله 13.16: مفهوم آزمون کای-دو (Chi-square Concept)

هدف از استفاده از آزمون χ^2 در انتخاب ویژگی (Feature Selection) چیست و فرضیه صفر (Null Hypothesis) در این آزمون چه چیزی را بیان می‌کند؟

مسئله 13.17: تحلیل ماتریس درهم‌ریختگی (Confusion Matrix Analysis)

شما یک سیستم فیلتر اسپم ساخته‌اید. در ماتریس درهم‌ریختگی به دست آمده، تعداد False Positive ها بالا است. این موضوع چه تاثیری روی تجربه کاربری دارد؟ آیا در این سناریو، بهینه‌سازی Precision مهم‌تر است یا Recall؟

مسئله 13.18: محاسبه دقیق اطلاعات متقابل (Scenario: Exact MI Calculation)

با استفاده از یک جدول توافقی 2×2 برای کلمه «تخفیف» و کلاس «اسپم»، نشان دهید چگونه مقدار Mutual Information محاسبه می‌شود. فرمول $I(U;C)$ را بسط دهید.

مسئله 13.19: چالش کلمات توقف در مدل برنولی (Scenario: Stop Words Challenge)

فرض کنید در پیش‌پردازش یک مجموعه داده برای مدل برنولی، کلمات توقف (Stop Words) حذف نشده‌اند. این کلمات در اسناد مربوط به هر دو کلاس حضور دارند. این موضوع چه تاثیری بر تفکیک‌پذیری کلاس‌ها در مدل برنولی (Bernoulli Model) خواهد داشت؟

مسئله 13.20: سناریو: داده‌های نامتوازن (Scenario: Imbalanced Data)

مجموعه آموزشی شما شامل 9000 سند ورزشی و 1000 سند اقتصادی است. اگر سیستم شما همه اسناد جدید را صرفاً «ورزشی» برچسب‌گذاری کند، مقدار Accuracy سیستم چقدر خواهد بود؟ چرا این معیار در این سناریو گمراه‌کننده است؟

مسئله 13.21: میکرو و ماکرو F1 (Scenario: Micro vs Macro F1)

در یک مسئله طبقه‌بندی سه کلاسه با داده‌های به شدت نامتوازن، سیستم شما در کلاس‌های اقلیت عملکرد بسیار ضعیفی دارد اما در کلاس اکثریت عالی کار می‌کند. کدام معیار (Micro-F1 یا Macro-F1) ضعف سیستم شما را بهتر آشکار می‌کند؟ دلیل خود را اثبات کنید.

مسئله 13.22: مقایسه روش‌های انتخاب ویژگی (Scenario: Feature Selection Comparison)

در چه سناریویی ممکن است روش χ^2 کلمه‌ای را به عنوان یک ویژگی مهم انتخاب کند، اما روش **Frequency-based** آن را نادیده بگیرد؟ یک مثال ملموس بیاورید.

مسئله 13.23: متغیرهای مدل چندجمله‌ای (Scenario: Multinomial Variant)

با توجه به بخش 13.4.1، چه تغییری در فرمول پایه مدل **Multinomial** می‌توان ایجاد کرد تا تاثیر تکرار یک کلمه خاص در سند (**Burstiness**) متعادل‌تر شود؟

مسئله 13.24: ویژگی‌های کمیاب (Scenario: Rare Features)

اگر معیار **MI** را برای تمام کلمات محاسبه کنیم، کلماتی که فقط یک بار در کل مجموعه داده ظاهر شده‌اند ممکن است امتیاز **MI** بالایی بگیرند. چرا این اتفاق می‌افتد و چگونه در عمل این مشکل را رفع می‌کنند؟

مسئله 13.25: انتخاب ویژگی برای طبقه‌بندهای چندگانه (Scenario: Feature Selection for Multiple Classifiers) ★

در یک سیستم طبقه‌بندی وب‌سایت‌ها با 100 دسته مختلف، شما مقادیر **Mutual Information** را برای کلمه «پایتون» نسبت به هر 100 کلاس محاسبه کرده‌اید. بر اساس سرفصل 13.5.4، با چه فرمول‌ها و استراتژی‌هایی می‌توانید این 100 مقدار را به یک امتیاز واحد (**Global Score**) تبدیل کنید تا در مورد حفظ یا حذف این کلمه تصمیم بگیرید؟

مسئله 13.26: اثبات مرز تصمیم خطی (Scenario: Linear Decision Boundary Proof) ★

به صورت ریاضیاتی اثبات کنید که الگوریتم **Naive Bayes** در واقع یک طبقه‌بند خطی (**Linear Classifier**) است. نشان دهید که تابع تصمیم‌گیری آن می‌تواند به شکل $w^T x + b > 0$ نوشته شود. متغیرهای **w** و **b** در اینجا دقیقاً معادل چه احتمالاتی در نایو بیز هستند؟

مسئله 13.27: تحلیل خطای Underflow در مقیاس بالا (Scenario: High-scale Underflow) ★

هنگام اجرای **Naive Bayes** بر روی متون حجیم حقوقی، محاسبه ضرب مکرر احتمالات شرطی باعث صفر شدن خروجی در زبان‌های برنامه‌نویسی می‌شود. با استفاده از قوانین پایه ریاضی، فرمول **نایو بیز چندجمله‌ای** را به فرمتی بازنویسی کنید که منحصراً از عملگر جمع برای غلبه بر این خطا استفاده کند.

مسئله 13.28: بروزرسانی آنلاین مدل نایو بیز (Scenario: Online Updating of Naive Bayes) ★

سیستم طبقه‌بند شما در یک شبکه اجتماعی دیپلوی شده و هر ثانیه هزاران پست جدید دریافت می‌کند. فرض کنید برچسب صحیح برخی از این پست‌ها توسط کاربران مشخص می‌شود. چگونه معماری مدل **Naive Bayes** خود را طراحی می‌کنید که نیازی به آموزش مجدد روی کل داده‌های قبلی نباشد و مدل بتواند به صورت بلادرنگ (**Online**) احتمالات را آپدیت کند؟

مسئله 13.29: طراحی طبقه‌بند سلسله‌مراتبی (Scenario: Hierarchical Classifier Design) ★

به جای یک طبقه‌بند مسطح (Flat)، می‌خواهیم مقالات را ابتدا به دو دسته کلی «علوم» و «هنر»، و سپس دسته «علوم» را به «فیزیک» و «ریاضی» طبقه‌بندی کنیم. طراحی یک مدل **نایو بیز سلسله‌مراتبی** چگونه خواهد بود و چه تاثیری بر **انتخاب ویژگی (Feature Selection)** در هر گره از این درخت تصمیم دارد؟

مسئله 13.30: پیاده‌سازی اسکرپت مدل چندجمله‌ای (Scenario: Multinomial Model Script Implementation) ★

شما باید اسکرپت اصلی مدل **Multinomial Naive Bayes** را در پایتون بنویسید. تابعی بنویسید که دیکشنری‌های مربوط به احتمالات پیشین و شرطی را خروجی دهد. کد شما باید قابلیت مدیریت **هموارسازی لاپلاس** را داشته باشد.

مسئله 13.31: پیاده‌سازی ارزیابگر خودکار (Scenario: Autograder Implementation) ★

یک کلاس پایتون طراحی کنید که دو لیست دریافت کند: مقادیر پیش‌بینی شده (**y_pred**) و مقادیر واقعی (**y_true**). این کلاس باید متدهایی برای محاسبه مقادیر **Macro-F1** و **Micro-F1** بدون استفاده از کتابخانه‌های آماده (مانند Scikit-learn) داشته باشد.

مسئله 13.32: مهندسی معکوس طبقه‌بند (Scenario: Classifier Reverse Engineering) ★

یک طبقه‌بند **Naive Bayes** به شما داده شده است (به عنوان یک جعبه سیاه) که مقادیر **Log-probabilities** خروجی آن برای هر کلاس قابل مشاهده است. چگونه می‌توانید با ارسال اسناد ساختگی (مثلاً اسنادی شامل فقط یک کلمه خاص) مقادیر احتمال شرطی $P(w_i | c_j)$ و احتمال پیشین $P(c_j)$ که درون سیستم ذخیره شده است را مهندسی معکوس کرده و به دست آورید؟ روابط ریاضی آن را مدل‌سازی کنید.

پاسخنامه مسئله 13.1: مفاهیم اولیه طبقه‌بندی متن

بخش 1) هدف اصلی در مسئله طبقه‌بندی متون

هدف اصلی در **طبقه‌بندی متن (Text Classification)**، تخصیص خودکار و نظام‌مند یک یا چند دسته (**Category**) یا برچسب (**Label**) از پیش‌تعیین‌شده به یک سند متنی است.

در دنیای وب و سیستم‌های اطلاعاتی، ما با حجم عظیمی از داده‌های بدون ساختار (**Unstructured Data**) مانند مقالات، ایمیل‌ها و نظرات مواجه هستیم. هدف ماشین در این مسئله، این است که با کشف الگوهای پنهان در کلمات، این متون خام را سازماندهی کرده و آن‌ها را به داده‌های ساختاریافته و قابل مدیریت تبدیل کند.

بخش 2) ساختار سیستم: مشخص کردن ورودی و خروجی

از دیدگاه ریاضی و معماری سیستم، یک طبقه‌بند متن (**Text Classifier**) را می‌توان به عنوان تابعی در نظر گرفت که یک ورودی را دریافت کرده و به یک خروجی نگاشت می‌کند:

ورودی (Input): شامل دو مؤلفه اصلی است:

یک فضای اسناد متنی مانند D . سیستم در هر لحظه یک سند خام مانند d (که $d \in D$) را دریافت می‌کند.

مجموعه‌ای ثابت و از پیش‌تعیین‌شده از دسته‌ها یا کلاس‌ها به فرم $C = \{c_1, c_2, \dots, c_n\}$.

خروجی (Output): سیستم باید تصمیم بگیرد که سند d به کدام کلاس تعلق دارد. بنابراین خروجی، یک برچسب نهایی مانند c_i (که $c_i \in C$) است. در سیستم‌های پیشرفته‌تر، خروجی می‌تواند یک توزیع احتمالاتی باشد که میزان تعلق سند به هر کلاس را به صورت درصدی نشان می‌دهد.

بخش 3) مثال‌های کاربردی در دنیای وب

برای درک بهتر معماری ورودی و خروجی، دو نمونه از پرکاربردترین سیستم‌های طبقه‌بندی وب در جدول زیر کالبدشکافی شده‌اند:

نام کاربرد	شرح مختصر سیستم	ورودی سیستم	خروجی سیستم (کلاس‌ها)
فیلترینگ هرزنامه (Spam Filtering)	سیستمی در سرویس‌دهنده‌های ایمیل که پیام‌های مزاحم را از پیام‌های واقعی جدا می‌کند.	متن کامل، عنوان ایمیل و فراداده‌های فرستنده	برچسب Spam (هرزنامه) یا برچسب Ham (پیام سالم)
تحلیل احساسات (Sentiment Analysis)	سیستمی در فروشگاه‌های اینترنتی که رضایت مشتریان را از روی متن نظرات آن‌ها ارزیابی می‌کند.	متن نقد و بررسی نوشته‌شده توسط کاربر درباره یک محصول	یکی از سه کلاس: مثبت ، منفی یا خنثی

نکته آموزشی:

کامپیوترها متون خام را متوجه نمی‌شوند. پیش از آنکه سند متنی (مانند ایمیل) به الگوریتم طبقه‌بند داده شود، باید طی مرحله‌ای به نام «استخراج ویژگی» (**Feature Extraction**) به بردارهای عددی قابل محاسبه تبدیل شود.

پاسخنامه مسئله 13.2: فرض استقلال در الگوریتم نایو بیز

بخش 1) مفهوم استقلال شرطی و دلیل نام‌گذاری «ساده‌لوحانه»

برای دسته‌بندی یک متن، الگوریتم بیز باید محاسبه کند که احتمال تعلق یک سند d (شامل مجموعه‌ای از کلمات) به یک کلاس خاص c چقدر است. چالش اصلی زمانی آغاز می‌شود که بخواهیم احتمال رخداد توأمان تمامی کلمات این سند را در آن کلاس محاسبه کنیم:

$$P(t_1, t_2, \dots, t_n | c)$$

محاسبه این احتمال مشترک برای ترکیب‌های مختلف کلمات، از نظر محاسباتی و ذخیره‌سازی در دنیای واقعی غیرممکن است؛ زیرا فضای حالت آن به شدت بزرگ است. در این نقطه، الگوریتم یک فرض بسیار قوی و ساده‌کننده را مطرح می‌کند که به **فرض استقلال شرطی (Conditional Independence Assumption)** معروف است.

بر اساس این فرض، الگوریتم در نظر می‌گیرد که حضور یک کلمه در متن، کاملاً مستقل از حضور کلمات دیگر در همان سند است (به شرط مشخص بودن کلاس). به بیان ریاضی، احتمال مشترک کلمات به حاصل ضرب احتمالات تک‌تک آن‌ها تجزیه می‌شود:

$$P(t_1, t_2, \dots, t_n | c) = \prod_{i=1}^n P(t_i | c)$$

دلیل اطلاق واژه «**ساده‌لوحانه**» (Naive) دقیقاً همین جاست. الگوریتم با ساده‌اندیشی تمام، ساختار پیچیده و درهم‌تنیده متن را نادیده گرفته و با کلمات یک جمله، مانند مهره‌هایی کاملاً مستقل و بی‌ارتباط با یکدیگر در یک کیسه برخورد می‌کند (مدل Bag of Words).

بخش 2) چرا این فرض در زبان طبیعی نقض می‌شود؟

در زبان طبیعی (Natural Language)، کلمات هرگز به صورت تصادفی و مستقل از هم کنار یکدیگر قرار نمی‌گیرند. زبان بر پایه قواعد نحوی (Grammar)، ساختار معنایی و هم‌آیندی‌ها (Collocations) بنا شده است. بنابراین، حضور یک کلمه، اطلاعات آماری بسیار مهمی درباره کلمات قبل و بعد از خود به همراه دارد.

مثال‌های نقض فرض در دنیای واقعی:

وابستگی معنایی: فرض کنید در یک متن به کلمه «هوش» برخورده‌ایم. در متون علوم کامپیوتر، با دیدن این کلمه، احتمال حضور کلمه «مصنوعی» به شدت افزایش می‌یابد. این دو کلمه مستقل نیستند، اما نایو بیز احتمال حضور هر یک را بدون توجه به دیگری محاسبه می‌کند.

نام‌های خاص و عبارات چندکلمه‌ای: در زبان انگلیسی، اگر کلمه "Hong" در متنی بیاید، با احتمال بسیار بالایی کلمه بعدی "Kong" خواهد بود. از نظر ریاضی در دنیای واقعی $P(Kong | Hong) \neq P(Kong)$ ، اما الگوریتم نایو بیز این وابستگی شدید را صفر در نظر می‌گیرد.

نکته مهم:

با وجود اینکه فرض استقلال در زبان طبیعی کاملاً اشتباه است و همواره نقض می‌شود، الگوریتم نایو بیز در عمل عملکرد بسیار خوبی در طبقه‌بندی متون (مانند تشخیص اسپم) دارد. دلیل این تضاد ظاهری این است که در دسته‌بندی متون، ما به دنبال یافتن **مقدار دقیق احتمال** نیستیم؛ بلکه فقط می‌خواهیم بدانیم احتمال کدام کلاس از بقیه **بیشتر** است (Maximum A Posteriori). نقض فرض استقلال، معمولاً مقادیر احتمالات را جابه‌جا می‌کند، اما رتبه‌بندی نهایی کلاس‌ها را تغییر نمی‌دهد.

پاسخنامه مسئله 13.3: احتمالات پیشین (Prior Probabilities) در نایو بیز

بخش 1) نحوه محاسبه احتمال پیشین

در قضیه بیز، **احتمال پیشین (Prior Probability)** که به صورت $P(c)$ نمایش داده می‌شود، بیانگر احتمال تعلق یک سند تصادفی به کلاس c است، پیش از آنکه هیچ اطلاعاتی درباره محتوای آن سند (کلمات درون متن) داشته باشیم.

در پیاده‌سازی الگوریتم نایو بیز، این احتمال به صورت تجربی و مستقیماً از روی توزیع داده‌ها در مجموعه آموزشی (**Training Set**) محاسبه می‌گردد. فرمول محاسبه آن بسیار ساده و به شرح زیر است:

$$P(c) = \frac{N_c}{N}$$

که در این رابطه:

متغیر **N_c** : نشان‌دهنده تعداد اسناد متعلق به کلاس **c** در مجموعه آموزشی است.

متغیر **N**: نشان‌دهنده کل تعداد اسناد موجود در مجموعه آموزشی (مجموع اسناد تمامی کلاس‌ها) است.

بخش 2) نقش احتمال پیشین در تصمیم‌گیری نهایی

بر اساس پایه ریاضی الگوریتم بیز، تصمیم‌گیری نهایی سیستم برای تخصیص برچسب به یک سند جدید، حاصل ضرب دو عامل است: احتمال پیشین (**Prior**) و درست‌نمایی کلمات (**Likelihood**).

نقش احتمال پیشین در این مکانیزم، ایجاد یک سوگیری (**Bias**) اولیه و منطقی بر اساس مشاهدات گذشته سیستم است. این نقش را می‌توان در دو سناریوی زیر بررسی کرد:

داده‌های نامتوازن (Imbalanced Data): فرض کنید در سیستم فیلترینگ ایمیل، 95 درصد داده‌های آموزشی هرزنامه (**Spam**) و تنها 5 درصد پیام سالم (**Ham**) باشند. در اینجا احتمال پیشین کلاس هرزنامه بسیار بالا است. این یعنی الگوریتم پیش از خواندن متن، به طور پیش‌فرض تمایل دارد ایمیل جدید را هرزنامه بداند. در نتیجه، برای اینکه یک ایمیل برچسب «سالم» دریافت کند، شواهد داخل متن (کلمات ایمیل) باید آن‌قدر قوی باشند که بتوانند بر این سوگیری اولیه غلبه کنند.

داده‌های متوازن (Balanced Data): اگر توزیع کلاس‌ها در فاز آموزش کاملاً مساوی باشد (مثلاً 50 درصد هرزنامه و 50 درصد سالم)، مقدار **P(c)** برای تمامی کلاس‌ها برابر خواهد بود. در این وضعیت یکتا، احتمال پیشین در فرآیند مقایسه ریاضی خنثی شده و تصمیم‌گیری الگوریتم منحصراً بر پایه محتوای متنی سند (درست‌نمایی کلمات) انجام می‌شود.

نکته آموزشی:

اهمیت تئوریک احتمال پیشین در این است که الگوریتم را به واقعیت توزیع داده‌ها در دنیای خارج متصل می‌کند. نادیده گرفتن این پارامتر باعث می‌شود سیستم، رخداد کلاس‌های بسیار نادر را با کلاس‌های روزمره و پرتکرار به یک اندازه محتمل فرض کند که به افت شدید دقت (**Accuracy**) در محیط‌های عملیاتی منجر می‌شود.

پاسخنامه مسئله 13.4: مدل چندجمله‌ای در برابر مدل برنولی

در الگوریتم‌های طبقه‌بندی متن مبتنی بر بیز (مانند **Naive Bayes**)، نحوه نگاه سیستم به یک سند متنی و شیوه تبدیل آن به داده‌های قابل محاسبه، اهمیت بالایی دارد. دو رویکرد استاندارد برای این کار وجود دارد که تفاوت بنیادین آن‌ها در **نحوه نمایش سند (Document Representation)** است.

بخش 1) مدل برنولی (Bernoulli Model)

در این مدل، فضای سند صرفاً بر اساس **حضور یا عدم حضور** کلمات ارزیابی می‌شود.

نحوه نمایش: سند به عنوان یک بردار باینری (دودویی) به طول کل واژگان سیستم (**Vocabulary**) در نظر گرفته می‌شود.

منطق محاسباتی: اگر یک کلمه در سند وجود داشته باشد، مقدار آن در بردار برابر با 1، و اگر وجود نداشته باشد، مقدار آن برابر با 0 قرار می‌گیرد.

این مدل با سند به صورت مجموعه‌ای از رویدادهای مستقل برخورد می‌کند و تنها بررسی می‌کند که آیا یک ویژگی (کلمه) رخ داده است یا خیر.

بخش 2) مدل چندجمله‌ای (Multinomial Model)

در این مدل، سند به صورت یک «کیسه کلمات» (**Bag of Words**) در نظر گرفته می‌شود که در آن، فراوانی و تعداد رخداد هر کلمه دارای اهمیت است.

نحوه نمایش: سند به عنوان برداری از فراوانی کلمات (**Term Frequencies**) نمایش داده می‌شود.

منطق محاسباتی: در این بردار، مقدار متناظر با هر کلمه برابر با تعداد دفعاتی است که آن کلمه در متن سند تکرار شده است (مثلاً عدد 3 به جای عدد باینری 1).

طول این بردار در محاسبات احتمالات، برابر با طول خود سند (مجموع تعداد کلمات متن) در نظر گرفته می‌شود.

بخش 3) کدام مدل به تکرار کلمات اهمیت می‌دهد؟

با توجه به تعاریف بالا، **مدل چندجمله‌ای (Multinomial)** به تکرار کلمات اهمیت می‌دهد، در حالی که مدل برنولی نسبت به دفعات تکرار یک کلمه کاملاً نابینا است.

مثال شفاف‌ساز:

فرض کنید کلمه «رایگان» در یک ایمیل تبلیغاتی 5 بار تکرار شده باشد.

از دیدگاه مدل برنولی: سیستم فقط ثبت می‌کند که کلمه «رایگان» در متن وجود دارد (مقدار = 1). تفاوتی نمی‌کند این کلمه 1 بار آمده باشد یا 100 بار.

از دیدگاه مدل چندجمله‌ای: سیستم تعداد تکرار را می‌شمارد (مقدار = 5) و در محاسبه احتمال نهایی، وزن و تأثیر این کلمه را به نسبت تکرار آن در نظر می‌گیرد. به همین دلیل، مدل چندجمله‌ای برای متون طولانی‌تر که تکرار کلمات در آن‌ها حاوی اطلاعات مهمی است، عملکرد بسیار بهتری نسبت به برنولی دارد.

مقایسه ساختاری در یک نگاه

ویژگی سیستم	مدل برنولی (Bernoulli)	مدل چندجمله‌ای (Multinomial)
نوع بردار نمایش سند	برداری باینری (شامل 0 و 1)	برداری فراوانی (شامل اعداد صحیح ≥ 0)
اهمیت به تکرار کلمه	خیر (فقط حضور / عدم حضور مهم است)	بله (تعداد دفعات تکرار شمارش می‌شود)

ویژگی سیستم	مدل برنولی (Bernoulli)	مدل چندجمله‌ای (Multinomial)
نوع کلمات لحاظ شده در محاسبه	هم کلمات موجود در سند و هم کلمات غایب	تنها کلماتی که در سند وجود دارند
کاربرد بهینه	متون بسیار کوتاه (مانند توییت‌ها)	متون طولانی‌تر (مانند مقالات و اخبار)

نکته آموزشی:

در مباحث پیشرفته‌تر بازایی اطلاعات (**IR**)، معمولاً مدل چندجمله‌ای به عنوان استاندارد پیش‌فرض برای طبقه‌بندی اسناد حجیم استفاده می‌شود؛ زیرا حذف اطلاعات مربوط به توزیع کلمات (کاری که مدل برنولی می‌کند) باعث از دست رفتن سیگنال‌های مهمی برای تفکیک دقیق کلاس‌ها خواهد شد.

پاسخنامه مسئله 13.5: مشکل فرکانس صفر (Zero Frequency Problem)

بخش 1) مکانیزم بروز مشکل در محاسبات

در الگوریتم نایو بیز، بر اساس فرض استقلال شرطی، احتمال تعلق یک سند جدید (**d**) به یک کلاس مشخص (**c**) از طریق ضرب احتمالات شرطی تک‌تک کلمات آن سند محاسبه می‌شود:

$$P(d|c) = \prod_{i=1}^n P(t_i|c)$$

حال اگر در سند جدید، کلمه‌ای مانند **t_k** وجود داشته باشد که در مرحله آموزش (**Training Phase**) هرگز در اسناد متعلق به کلاس **c** مشاهده نشده باشد، فرکانس (فراوانی) آن کلمه در کلاس مذکور صفر خواهد بود. در نتیجه، از نظر ریاضی الگوریتم احتمال رخداد این کلمه در آن کلاس را صفر در نظر می‌گیرد:

$$P(t_k|c) = 0$$

بخش 2) پیامد و اثر نهایی بر تصمیم‌گیری سیستم

به دلیل ماهیت ضربی فرمول بالا، وقوع **حتی یک احتمال صفر** باعث می‌شود که کل حاصل‌ضرب از بین برود و احتمال نهایی تعلق سند به آن کلاس **مطلقاً صفر** شود.

این اتفاق یک فاجعه محاسباتی در سیستم ایجاد می‌کند؛ زیرا فارغ از اینکه سایر کلمات موجود در سند چقدر گواه و دلیل قدرتمندی بر تعلق سند به کلاس **c** باشند، الگوریتم به دلیل حضور تنها یک کلمه ناشناخته، آن کلاس را به طور کامل از گزینه‌های تصمیم‌گیری حذف می‌کند.

مثال ملموس:

فرض کنید یک ایمیل کاملاً تبلیغاتی شامل 100 کلمه است. 99 کلمه آن دقیقاً مشابه هرزنامه‌هایی (**Spam**) است که سیستم قبلاً یاد گرفته (کلماتی مانند «رایگان»، «جایزه»، «کلیک»). اما یک کلمه جدید (مثلاً نام یک برند جدید) در متن

وجود دارد که در مجموعه آموزشی هرزنامه‌ها نبوده است. سیستم به دلیل وجود همین یک کلمه جدید، احتمال **Spam** بودن کل ایمیل را **صفر** محاسبه می‌کند و به اشتباه آن را به عنوان پیام سالم (**Ham**) دسته‌بندی می‌کند.

راهکار:

برای جلوگیری از پدیده فرکانس صفر، در پیاده‌سازی‌های عملیاتی از تکنیک‌های جبران‌سازی به نام **هموارسازی (Smoothing)** استفاده می‌شود. معروف‌ترین آن‌ها **هموارسازی لاپلاس (Laplace Smoothing)** یا **Add-One** است. در این تکنیک، در فرمول محاسبه احتمال، به صورت مجازی به فراوانی تمام کلمات در همه کلاس‌ها 1 واحد اضافه می‌شود (و به مخرج کسر نیز به اندازه، اندازه واژگان سیستم افزوده می‌شود) تا هیچ‌گاه فرکانس کلمه‌ای صفر نشود.

پاسخنامه مسئله 13.6: هموارسازی لاپلاس (Laplace Smoothing)

بخش 1) مکانیزم حل مشکل فرکانس صفر

همان‌طور که در مسئله قبل بررسی شد، مشکل فرکانس صفر زمانی رخ می‌دهد که احتمال رخداد یک کلمه ناشناخته در یک کلاس، 0 محاسبه شود و کل فرآیند ضرب احتمالات را از بین ببرد.

تکنیک هموارسازی (**Smoothing**) بر یک ایده بسیار ساده اما کارآمد ریاضی استوار است: **تخصیص یک شانس حداقلی به رویدادهای دیده‌نشده**. در این روش، ما به جای آنکه فرض کنیم فراوانی یک کلمه کاملاً 0 است، به صورت مجازی به تعداد دفعات رخداد تمام کلمات، یک مقدار ثابت اضافه می‌کنیم.

فرمول محاسبه احتمال شرطی در مدل چندجمله‌ای (**Multinomial**) با اعمال هموارسازی به شکل زیر تغییر می‌کند:

$$P(t|c) = \frac{T_{ct} + \alpha}{\sum_{t' \in V} T_{ct'} + \alpha|V|}$$

تشریح متغیرهای فرمول:

T_{ct} : تعداد واقعی دفعات تکرار کلمه **t** در مجموعه اسناد متعلق به کلاس **c**.

$\sum T_{ct'}$: مجموع تعداد کل کلمات (طول تمامی متون) در کلاس **c**.

V: واژگان (**Vocabulary**) سیستم؛ یعنی مجموعه تمام کلمات یکتایی که در کل فاز آموزش (فارغ از نوع کلاس) دیده شده‌اند.

|V|: اندازه واژگان (تعداد کلمات یکتا در کل دادگان آموزشی).

α : پارامتر هموارسازی.

با اضافه شدن **α** به صورت کسر، حتی اگر کلمه‌ای در یک کلاس هرگز دیده نشده باشد (**$T_{ct} = 0$**)، صورت کسر برابر با **α** خواهد بود و دیگر احتمال 0 تولید نمی‌شود.

بخش 2) نقش و تحلیل پارامتر آلفا (α)

پارامتر **α** در واقع یک عامل کنترلی (**Hyperparameter**) است که شدت و قدرت هموارسازی را تعیین می‌کند. تغییرات

این پارامتر رفتارهای متفاوتی در الگوریتم ایجاد می‌کند:

حالت $\alpha = 1$ (هموارسازی استاندارد لاپلاس): به این حالت **Add-One Smoothing** نیز گفته می‌شود. فرض

می‌شود که هر کلمه حداقل 1 بار بیشتر از مقدار واقعی‌اش دیده شده است. این رایج‌ترین حالت برای حل مشکل 0 شدن احتمالات است.

حالت $\alpha = 0$: سیستم به حالت پایه (بدون هموارسازی) بازمی‌گردد و مشکل فرکانس 0 مجدداً بروز می‌کند.

حالت $0 < \alpha < 1$ (هموارسازی لیدستون - **Lidstone**): در متون بسیار طولانی و واژگان بزرگ، افزودن عدد 1 به همه کلمات ممکن است توزیع واقعی احتمالات را بیش از حد تغییر دهد. در این شرایط از مقادیر کسری (مانند 0.1 یا 0.5) استفاده می‌شود تا تغییرات با ظرافت بیشتری اعمال شود.

اثر مقادیر بسیار بزرگ α : اگر مقدار آلفا بسیار بزرگ انتخاب شود، تأثیر فراوانی واقعی کلمات در داده‌های

آموزشی (T_{ct}) به شدت کمرنگ می‌شود و توزیع احتمالات سیستم به سمت یک «توزیع یکنواخت»

(**Uniform Distribution**) میل می‌کند. در این حالت، الگوریتم عملاً قدرت تفکیک و یادگیری خود را از دست

می‌دهد.

نکته:

چرا در مخرج کسر به جای α ، عبارت $\alpha |V|$ اضافه می‌شود؟

در علم آمار، مجموع احتمالات شرطی تمام کلمات ممکن برای یک کلاس خاص باید دقیقاً برابر با عدد 1 باشد ($\sum P(t|c) = 1$). از آنجایی که ما در صورت کسر، به فراوانی هر کلمه مقدار α را اضافه کرده‌ایم و در کل $|V|$ کلمه یکتا داریم، در مجموع مقدار صورت به اندازه $\alpha |V|$ بزرگ‌تر شده است. برای حفظ تعادل این کسر و جلوگیری از نقض قوانین احتمالات، باید مخرج کسر را نیز دقیقاً به همین میزان ($\alpha |V|$) افزایش دهیم.

پاسخنامه مسئله 13.7: ارتباط نایو بیز چندجمله‌ای و مدل زبانی مونوگرام

برای درک عمیق این ارتباط، باید الگوریتم نایو بیز را نه فقط به عنوان یک ابزار تصمیم‌گیرنده، بلکه به عنوان سیستمی که می‌تواند داده تولید کند، مورد بررسی قرار دهیم.

بخش 1) پیوند بنیادین: فرض استقلال کامل

نقطه تلاقی و اتحاد مدل نایو بیز چندجمله‌ای (**Multinomial Naive Bayes**) و مدل زبانی مونوگرام (**Unigram Language Model**) در فرض استقلال آن‌ها نهفته است.

مدل زبانی مونوگرام: در پردازش زبان طبیعی (**NLP**)، مدل مونوگرام ساده‌ترین نوع مدل زبانی است. این مدل

فرض می‌کند که احتمال ظهور هر کلمه در یک متن، کاملاً مستقل از کلمات قبل و بعد از آن است. بنابراین،

احتمال تولید یک جمله (یا سند) صرفاً از ضرب احتمال تک‌تک کلمات آن به دست می‌آید.

نایو بیز: همان‌طور که در مسائل پیشین بررسی شد، نایو بیز نیز دقیقاً همین فرض (استقلال شرطی کلمات) را

اعمال می‌کند تا احتمال $P(d|c)$ را محاسبه نماید.

در واقع، می‌توان گفت محاسبه احتمال درست‌نمایی $P(d|c)$ در نایو بیز، از نظر ریاضی دقیقاً معادل با محاسبه احتمال تولید آن سند توسط یک مدل زبانی مونوگرام است که منحصرراً روی متون کلاس c آموزش دیده است.

بخش 2) رویکرد زایشی: نایو بیز به عنوان تولیدکننده متن

الگوریتم نایو بیز در دسته مدل‌های زایشی (**Generative Models**) قرار می‌گیرد. این یعنی سیستم برای دسته‌بندی یک سند، ابتدا تصور می‌کند که آن سند چگونه توسط یک انسان (یا ماشین) «نوشته» شده است.

فرآیند ذهنی و گام‌به‌گام نایو بیز برای تولید یک سند متنی تصادفی به این شکل است:

1. **انتخاب کلاس (موضوع):** نویسنده ابتدا تصمیم می‌گیرد که می‌خواهد درباره چه موضوعی بنویسد. سیستم این کار را با انتخاب یک کلاس **c** بر اساس توزیع احتمالات پیشین **P(c)** انجام می‌دهد (مثلاً تصمیم می‌گیرد یک ایمیل «هرزنامه» تولید کند).

2. **تعیین طول متن:** نویسنده تصمیم می‌گیرد که متن او چند کلمه داشته باشد (مثلاً طول **n** کلمه).

3. **تولید کلمات (پرتاب تاس):** نویسنده یک تاس بسیار بزرگ در دست دارد که تعداد وجه‌های آن برابر با کل کلمات زبان (واژگان **V**) است. از آنجایی که نویسنده می‌خواهد متن خود را در کلاس **c** بنویسد، این تاس یک تاس منصفانه نیست؛ بلکه وزن‌های آن بر اساس توزیع احتمال کلمات در کلاس **c** تنظیم شده است (همان احتمال شرطی **P(t|c)**).

4. **تکرار مستقل:** نویسنده برای تولید کلمه اول، تاس را می‌اندازد. سپس تاس را برمی‌دارد و مستقل از کلمه اول، دوباره برای کلمه دوم آن را می‌اندازد. این کار را **n** بار تکرار می‌کند تا تمام کلمات متن تولید شوند.

بخش 3) نتیجه‌گیری منطقی الگوریتم در فرآیند طبقه‌بندی

زمانی که یک متن جدید به عنوان ورودی به سیستم داده می‌شود، الگوریتم نایو بیز متن را تولید نمی‌کند، بلکه فرآیند بالا را **مهندسی معکوس** می‌کند.

الگوریتم از خود می‌پرسد:

«من **N** مدل زبانی مونوگرام متفاوت (به ازای هر کلاس یک مدل) در اختیار دارم. احتمال اینکه این متن جدید توسط تاسِ مخصوصِ کدام کلاس تولید شده باشد، از بقیه بیشتر است؟»

هر کلاسی که مدل مونوگرام آن بالاترین احتمال را برای تولید این سند خاص ارائه دهد، به عنوان برچسب نهایی انتخاب می‌شود.

نکته آموزشی:

دلیل نام‌گذاری «چندجمله‌ای» (**Multinomial**) در این مدل دقیقاً به همین رویکرد زایشی برمی‌گردد. پرتاب مکرر یک تاس با وجه‌های مختلف و محاسبه توزیع فراوانی نتایج، در علم آمار با توزیع چندجمله‌ای (**Multinomial Distribution**) مدل‌سازی می‌شود.

پاسخنامه مسئله 13.8: فضای ویژگی‌ها (Feature Space) و نقش دایره لغات

بخش 1: مفهوم فضای ویژگی در مدل‌سازی متن

در الگوریتم‌های یادگیری ماشین، اسناد متنی خام به صورت مستقیم برای کامپیوتر قابل خواندن و محاسبه نیستند. برای حل این مشکل، متون باید به بردارهای عددی تبدیل شوند (**Vectorization**). این فرآیند نگاشت، نیازمند تعریف یک

فضای ویژگی (Feature Space) است که در آن، هر محور یا بُعد (**Dimension**) از فضا، نماینده یکی از ویژگی‌های استخراج‌شده از داده‌ها باشد.

بخش 2: نقش دایره لغات (Vocabulary) در تعیین ابعاد

در رویکردهای استاندارد بازنمایی متن (مانند **Bag of Words** یا **TF-IDF**)، ویژگی‌هایی که قرار است ابعاد فضا را تشکیل دهند، همان کلمات موجود در متن هستند.

دایره لغات (V) مجموعه‌ای است شامل تمامی کلمات یکتا و متمایزی که حداقل **1** بار در کل مجموعه اسناد آموزشی (**Training Corpus**) مشاهده شده‌اند.

نقش اصلی دایره لغات این است که **تعداد ابعاد فضای ویژگی‌ها را به صورت مستقیم و یک‌به‌یک تعیین می‌کند**. به زبان ریاضی، اگر اندازه دایره لغات برابر با $|V|$ باشد، فضای ویژگی ما فضایی $|V|$ -بُعدی ($\mathbb{R}^{|V|}$) خواهد بود و هر سند در قالب یک بردار با $|V|$ مؤلفه نمایش داده می‌شود.

بخش 3: مثال ملموس برای درک نگاشت

فرض کنید کل مجموعه آموزشی ما تنها از **2** سند بسیار کوتاه تشکیل شده باشد:

سند اول: «بازیابی اطلاعات مهم است»

سند دوم: «دقت بازیابی مهم است»

دایره لغات استخراج‌شده از کل این مجموعه برابر است با:

$V = \{ \text{بازیابی، اطلاعات، مهم، است، دقت} \}$

در اینجا اندازه دایره لغات **5** کلمه است ($|V| = 5$). بنابراین، فضای ویژگی‌های سیستم ما دقیقاً **5 بُعدی** خواهد بود و هر سند فارغ از طول خودش، به یک بردار ثابت **5** تایی تبدیل می‌گردد. (برای مثال با استفاده از مدل برنولی، بردار سند دوم $[1, 0, 1, 1, 1]$ خواهد بود).

بخش 4: پیامدهای ساختاری و چالش‌های معماری

وابستگی مستقیم ابعاد فضا به دایره لغات، دو پیامد بسیار مهم در سیستم‌های واقعی (که دایره لغات در آن‌ها شامل ده‌ها یا صدها هزار کلمه است) به همراه دارد:

خلوت بودن یا پراکندگی بردارها (Sparsity): در یک سیستم واقعی، اندازه $|V|$ ممکن است **100,000** کلمه باشد (فضای **100,000** بُعدی). اما یک ایمیل کوتاه شاید تنها شامل **50** کلمه یکتا باشد. این یعنی در بردار آن سند، **50** بُعد دارای مقدار غیرصفر و **99,950** بُعد دیگر مقدارشان صفر است. این پراکندگی نیازمند ساختارهای داده ویژه برای ذخیره‌سازی است تا حافظه سیستم هدر نرود.

نفرین ابعاد (Curse of Dimensionality): با افزایش بی‌رویه لغات، ابعاد فضا به قدری بزرگ می‌شود که محاسبه فواصل بردارها و آموزش مدل‌های ریاضی به شدت کند و پرهزینه می‌شود.

نکته آموزشی و راهکار جبران:

به دلیل چالش‌های فوق، سیستم‌های بازیابی اطلاعات هرگز کل دایره لغات خام را به عنوان فضای ویژگی نمی‌پذیرند. پیش از تشکیل بردارها، از طریق تکنیک‌های پیش‌پردازش مانند **حذف کلمات توقف**

(Stop-words Removal)، ریشه‌یابی (Stemming) و انتخاب ویژگی (Feature Selection)، سعی می‌شود کلمات زائد حذف شده و اندازه $|V|$ (و در نتیجه ابعاد فضا) تا حد امکان بهینه‌سازی و کاهش یابد.

پاسخنامه تشریحی مسئله 13.9: محاسبه مدل برنولی و تأثیر عدم حضور کلمه

بخش 1) تأثیر عدم حضور کلمه در محاسبات

در الگوریتم‌های مبتنی بر مدل برنولی (Multivariate Bernoulli)، برخلاف مدل چندجمله‌ای، سیستم تنها به کلماتی که در متن نوشته شده‌اند توجه نمی‌کند؛ بلکه **غیبت کلمات موجود در دایره لغات (Vocabulary)** نیز به عنوان یک سیگنال اطلاعاتی بسیار مهم در نظر گرفته می‌شود.

هنگامی که یک کلمه در سند غایب باشد، الگوریتم به جای نادیده گرفتن آن، احتمال «عدم رخداد» آن کلمه در کلاس مربوطه را محاسبه کرده و در احتمال نهایی ضرب می‌کند.

اثر این کار اعمال یک **جریمه (Penalty)** است: اگر کلمه‌ای وابستگی شدیدی به یک کلاس خاص داشته باشد (مثلاً انتظار داشته باشیم حتماً در آن کلاس دیده شود)، عدم حضور آن در سند جدید، احتمال تعلق سند به آن کلاس را به شدت کاهش می‌دهد.

بخش 2) فرمول ریاضی مدل برنولی

فرض کنید V مجموعه کل واژگان سیستم باشد و سند ورودی d به صورت یک بردار باینری نمایش داده شود که در آن مؤلفه x_t نشان‌دهنده وضعیت حضور کلمه t در سند است ($x_t = 1$ یعنی حضور دارد و $x_t = 0$ یعنی غایب است). احتمال درست‌نمایی تعلق سند d به کلاس c از طریق فرمول زیر محاسبه می‌شود:

$$P(d|c) = \prod_{t \in V} \left[P(t|c)^{x_t} \times (1 - P(t|c))^{(1-x_t)} \right]$$

کالبدشکافی رفتار فرمول در دو حالت ممکن:

حالت اول (کلمه در متن وجود دارد): در این حالت $x_t = 1$ است. توان عبارت اول در براکت برابر 1 و توان عبارت دوم برابر 0 می‌شود. در نتیجه فرمول تنها مقدار $P(t|c)$ را در محاسبات ضرب می‌کند (احتمال حضور).

حالت دوم (کلمه در متن غایب است): در این حالت $x_t = 0$ است. توان عبارت اول برابر 0 (که کل عبارت را تبدیل به عدد 1 می‌کند) و توان عبارت دوم برابر 1 می‌شود. در این حالت، فرمول مقدار $(1 - P(t|c))$ را در محاسبات اعمال می‌کند (احتمال عدم حضور).

بخش 3) تحلیل تحلیلی با یک مثال ملموس

فرض کنید سیستم در حال طبقه‌بندی یک ایمیل است و دایره لغات ما شامل کلمه «قرعه‌کشی» است. در مرحله آموزش، سیستم یاد گرفته که این کلمه در 90 درصد ایمیل‌های هرزنامه (**Spam**) وجود دارد؛ بنابراین $P(\text{قرعه‌کشی} | \text{Spam}) = 0.9$ است.

حال یک ایمیل جدید وارد سیستم می‌شود که فاقد کلمه «قرعه‌کشی» است:

در مدل چندجمله‌ای: چون کلمه در متن نیست، اصلاً وارد محاسبات این سند نمی‌شود و تأثیری روی نتیجه ندارد.

در مدل برنولی: سیستم متوجه غیبت کلمه می‌شود. طبق فرمول بالا ($x_t = 0$)، مقدار $(1 - 0.9) = 0.1$ محاسبه شده و در احتمال نهایی کلاس **Spam** ضرب می‌شود. ضرب شدن در این عدد کوچک، احتمال هرزنامه بودن ایمیل را به شدت پایین می‌آورد.

منطق سیستم این است: «اگر این ایمیل هرزنامه بود، به احتمال 90 درصد باید این کلمه را می‌داشت؛ حالا که ندارد، این یک نشانه قوی است که احتمالاً هرزنامه نیست.»

نکته آموزشی:

به عملگر ضرب (Π) در فرمول دقت کنید. در مدل برنولی، این حلقه ضرب روی **تمامی اعضای دایره لغات (V)** اجرا می‌شود، در حالی که در مدل چندجمله‌ای، حلقه ضرب صرفاً روی کلمات موجود در همان سند خاص اجرا می‌گردد. به همین دلیل است که پیچیدگی زمانی پردازش هر سند در مدل برنولی با اندازه کل واژگان رابطه مستقیم دارد، نه با طول خود سند.

پاسخنامه مسئله 13.10: اثبات پیچیدگی زمانی آموزش در نایو بیز

بخش 1) تحلیل وظایف الگوریتم در فاز آموزش

برای محاسبه پیچیدگی زمانی (**Time Complexity**)، ابتدا باید بدانیم سیستم در مرحله آموزش (**Training Phase**) دقیقاً چه محاسباتی را باید انجام دهد. هدف الگوریتم نایو بیز در این فاز، استخراج 2 دسته پارامتر از روی داده‌های خام است:

1. **احتمالات پیشین: $P(c)$** برای هر کلاس.

2. **احتمالات شرطی (همراه با هموارسازی): $P(t|c)$** برای تک‌تک کلمات در هر کلاس.

برای انجام این محاسبات، الگوریتم 2 گام اصلی را طی می‌کند که پیچیدگی هر کدام را به صورت مجزا بررسی می‌کنیم.

بخش 2) گام اول: پیمایش داده‌ها و شمارش (Scanning Phase)

در اولین قدم، ماشین باید تمامی متون موجود در مجموعه آموزشی را کلمه‌به‌کلمه بخواند تا دایره لغات (**Vocabulary**) را بسازد و دفعات تکرار هر کلمه در هر کلاس (**T_{ct}**) را بشمارد.

بر اساس مفروضات مسئله، متغیر **$|D|$** برابر با **مجموع طول تمامی اسناد** است (یعنی کل تعداد کلماتی که در فایل‌های متنی آموزشی وجود دارد).

برای خواندن این کلمات و به‌روزرسانی شمارنده‌ها (مثلاً در یک جدول درهم‌سازی یا **Hash Table**)، پردازنده باید کل داده‌ها را دقیقاً 1 بار از ابتدا تا انتها پیمایش کند.

بنابراین، زمان اجرای این گام مستقیماً متناسب با حجم کل متن است و پیچیدگی زمانی آن برابر با **$O(|D|)$** خواهد بود.

بخش 3) گام دوم: محاسبه احتمالات (Computation Phase)

پس از اتمام شمارش، داده‌های خام آماده شده‌اند و سیستم باید مقادیر کسری احتمالات را محاسبه و در حافظه ذخیره کند:

سیستم دارای $|C|$ کلاس مختلف (مثلاً دسته‌های ورزشی، سیاسی، علمی) و $|V|$ کلمه یکتا در دایره لغات است.

برای محاسبه احتمال $P(t|c)$ ، الگوریتم باید روی هر کلاس، یک حلقه تکرار به طول تمامی کلمات واژگان اجرا کند تا فرمول هموارسازی لاپلاس را برای آن کلمه در آن کلاس محاسبه کند.

تعداد کل عملیات ریاضی در این گام برابر است با ضرب تعداد کلاس‌ها در اندازه دایره لغات. پس زمان اجرای این گام $O(|C| \times |V|)$ است.

بخش 4) جمع‌بندی و ساده‌سازی پیچیدگی نهایی

با جمع کردن زمان اجرای 2 گام بالا، پیچیدگی زمانی کل فاز آموزش به دست می‌آید:

$$\text{Total Time} = O(|D| + |C| \times |V|)$$

چرا پارامتر $|C|$ از فرمول نهایی حذف می‌شود؟

در مسائل واقعی و کاربردی طبقه‌بندی متن، تعداد کلاس‌ها ($|C|$) در مقایسه با حجم داده‌ها بسیار ناچیز است و عملاً یک عدد ثابت (**Constant**) محسوب می‌شود (مثلاً 2 کلاس برای تشخیص هرزنامه، یا نهایتاً 20 کلاس برای دسته‌بندی اخبار).

در منطق تحلیل مجانبی (**Asymptotic Analysis**) الگوریتم‌ها، ثوابت کوچک در برابر مقادیر متغیر و عظیمی مانند طول متون ($|D|$) و گستردگی واژگان ($|V|$) نادیده گرفته می‌شوند، بنابراین $O(|C|) \approx O(1)$.

با در نظر گرفتن $|C|$ به عنوان یک ضریب ثابت، عبارت $|C| \times |V|$ به فرم خطی $O(|V|)$ تقلیل می‌یابد و اثبات فرمول کامل می‌شود:

$$\text{Training Time Complexity} = O(|D| + |V|)$$

نکته:

این اثبات ریاضی پرده از یکی از بزرگ‌ترین مزیت‌های نایو بیز برمی‌دارد. رابطه $O(|D| + |V|)$ نشان می‌دهد که زمان آموزش این الگوریتم با افزایش حجم متون کاملاً خطی (**Linear**) رشد می‌کند. به همین دلیل، در مواجهه با کلان‌داده‌ها (**Big Data**) و میلیون‌ها سند، نایو بیز بسیار سریع‌تر از الگوریتم‌های پیچیده‌تری مانند شبکه‌های عصبی عمیق (**Deep Neural Networks**) آموزش می‌بیند.

پاسخنامه تشریحی مسئله 13.11: اثبات پیچیدگی زمانی تست (**Testing**) در نایو بیز

بخش 1) تحلیل وظیفه الگوریتم در مرحله تست

در مرحله تست، سیستم فاز یادگیری را به اتمام رسانده و تمامی احتمالات پیشین $P(c)$ و احتمالات شرطی کلمات $P(t|c)$ در ساختارهای داده‌ای با دسترسی سریع (مانند جداول درهم‌سازی یا **Hash Tables**) ذخیره شده‌اند.

اکنون یک سند کاملاً جدید مانند **d** با طول **L_d** (یعنی سندی که از **L_d** کلمه یا توکن تشکیل شده است) به سیستم داده می‌شود. وظیفه الگوریتم این است که بررسی کند این سند به کدام یک از **|C|** کلاس موجود تعلق دارد.

بخش 2) کالبدشکافی عملیات ریاضی برای یک کلاس مشخص

برای تصمیم‌گیری، نایو بیز از قانون بیشینه‌سازی احتمال پسین (**MAP**) استفاده می‌کند. برای جلوگیری از خطای حد پایین محاسباتی (**Underflow**) ناشی از ضرب اعداد اعشاری بسیار کوچک، در پیاده‌سازی‌های عملی از لگاریتم احتمالات استفاده می‌شود.

فرمول محاسبه امتیاز برای یک کلاس خاص مانند **c** به شکل زیر است:

$$\text{Score}(c) = \log P(c) + \sum_{i=1}^{L_d} \log P(t_i | c)$$

حال بیاید هزینه محاسباتی (تعداد عملیات) این فرمول را محاسبه کنیم:

پیدا کردن مقدار **log P(c)** از حافظه: **O(1)** زمان می‌برد.

حلقه سیگما (**Σ**): الگوریتم باید متن سند جدید را کلمه‌به‌کلمه بخواند. چون سند **L_d** کلمه دارد، حلقه **L_d** بار تکرار می‌شود.

در هر تکرار حلقه، الگوریتم کلمه **t_i** را در جدول جستجو کرده و مقدار **log P(t_i | c)** را واکشی و جمع می‌کند. زمان جستجو در جدول درهم‌سازی **O(1)** است.

بنابراین، کل عملیات محاسبه امتیاز برای فقط یک کلاس نیازمند **L_d** بار جمع ریاضی است. در نتیجه زمان اجرای آن برابر با **O(L_d)** خواهد بود.

بخش 3) تعمیم محاسبات به کل سیستم (محاسبه برچسب نهایی)

سیستم نمی‌داند سند متعلق به کدام کلاس است، بنابراین نمی‌تواند فقط امتیاز یک کلاس را حساب کند. الگوریتم باید فرمول مرحله قبل را برای تک‌تک کلاس‌های موجود در سیستم محاسبه کند و در نهایت کلاسی که بالاترین امتیاز را آورد، به عنوان برچسب نهایی معرفی نماید:

$$c_{map} = \arg \max_{c \in C} [\text{Score}(c)]$$

از آنجا که سیستم دارای **|C|** کلاس مختلف است و محاسبه امتیاز برای هر کلاس به اندازه **O(L_d)** زمان می‌برد، پردازنده باید این فرآیند را **|C|** بار تکرار کند. با ضرب این 2 مقدار، پیچیدگی زمانی کل در مرحله تست اثبات می‌شود:

$$\text{Testing Time Complexity} = O(|C| \times L_d)$$

تفاوت با مدل برنولی

اثبات بالا، یکی از ویژگی‌های درخشان **مدل چندجمله‌ای (Multinomial)** را نشان می‌دهد. در این مدل، ما فقط روی کلماتی که در داخل سند جدید وجود دارند (L_d) پیمایش می‌کنیم. اما اگر از **مدل برنولی** استفاده می‌کردیم، الگوریتم مجبور بود برای تست یک سند جدید، عدم حضور سایر کلمات را نیز بررسی کند! این یعنی در مدل برنولی، حلقه ما باید به جای L_d ، روی کل دایره لغات سیستم ($|V|$) اجرا می‌شد و پیچیدگی زمانی تست به $O(|C| \times |V|)$ افزایش می‌یافت. از آنجا که همیشه طول یک سند بسیار کوچکتر از کل دایره لغات است ($|L_d| \ll |V|$)، مدل چندجمله‌ای در زمان تست (**Run-time**) بسیار سریع‌تر از برنولی عمل می‌کند.

پاسخنامه تشریحی مسئله 13.12: دلیل استفاده از لگاریتم احتمالات (Log Probabilities)

بخش 1) محدودیت سخت‌افزاری و خطای حد پایین (Floating-point Underflow)

در فرمول اصلی الگوریتم نایو بیز، احتمال نهایی از طریق ضرب تعداد زیادی عدد اعشاری (احتمالات شرطی) محاسبه می‌شود:

$$P(d|c) \propto P(c) \times P(t_1|c) \times P(t_2|c) \times \dots \times P(t_n|c)$$

احتمال رخداد هر کلمه همواره عددی بین 0 و 1 است ($0 < P \leq 1$). اگر یک سند متنی مثلاً 1000 کلمه داشته باشد، ما در حال ضرب کردن 1000 عدد اعشاری کوچک در یکدیگر هستیم. از نظر ریاضی، حاصل ضرب این اعداد به سرعت به سمت 0 میل می‌کند (مثلاً عددی مانند 10^{-300}).

در معماری سخت‌افزار کامپیوتر و استاندارد ممیز شناور (مانند **IEEE 754**)، ذخیره اعدادی که تا این حد به 0 نزدیک هستند غیرممکن است. پردازنده کامپیوتر به دلیل محدودیت در تعداد بیت‌های تخصیص‌یافته، این عدد بسیار کوچک را گرد کرده و آن را **مطلقاً 0 (0.0)** در نظر می‌گیرد. به این پدیده، **خطای حد پایین محاسباتی (Underflow)** می‌گویند. زمانی که امتیاز تمام کلاس‌ها در سیستم به دلیل این خطا 0 ارزیابی شود، الگوریتم قدرت تصمیم‌گیری و مقایسه خود را به طور کامل از دست می‌دهد.

بخش 2) راهکار ریاضی: تبدیل ضرب به جمع با لگاریتم

برای دور زدن این محدودیت سخت‌افزاری، از یکی از بنیادی‌ترین خواص تابع لگاریتم استفاده می‌شود: **لگاریتم حاصل ضرب، برابر است با حاصل جمع لگاریتم‌ها.**

با اعمال تابع لگاریتم (معمولاً در مبنای طبیعی **e** یا مبنای 2) بر دو طرف معادله، فرمول از فضای «ضرب اعداد اعشاری» به فضای «جمع اعداد منفی» نگاشت می‌شود:

$$\log \left(P(c) \prod_{i=1}^n P(t_i|c) \right) = \log P(c) + \sum_{i=1}^n \log P(t_i|c)$$

از آنجا که لگاریتم هر عدد بین 0 و 1، یک عدد منفی قابل توجه است (مثلاً $\log_{10}(0.01) = -2$)، جمع کردن هزاران عدد منفی نه تنها باعث خطای **Underflow** نمی‌شود، بلکه کاملاً در محدوده امن پردازشگرهای استاندارد قرار دارد.

بخش 3) چرا لگاریتم گرفتن نتیجه دسته‌بندی را تغییر نمی‌دهد؟

ممکن است این سوال برای دانشجو پیش بیاید که با تغییر فرمول، مقادیر خروجی تغییر می‌کنند؛ پس چگونه الگوریتم هنوز درست کار می‌کند؟

پاسخ در ویژگی **یکنوایی صعودی (Monotonicity)** تابع لگاریتم نهفته است. در دسته‌بندی نایو بیز، ما به دنبال یافتن مقدار دقیق احتمال نیستیم، بلکه فقط می‌خواهیم بدانیم احتمال کدام کلاس از بقیه بیشتر است (عملیات **arg max**). از آنجا که تابع لگاریتم اکیداً صعودی است، اگر مقدار **x** از مقدار **y** بزرگتر باشد، قطعاً **log(x)** نیز از **log(y)** بزرگتر خواهد بود:

$$P(c_1) > P(c_2) \iff \log P(c_1) > \log P(c_2)$$

بنابراین، رتبه‌بندی نهایی کلاس‌ها کاملاً دست‌نخورده باقی می‌ماند و کلاسی که بیشترین احتمال خام را داشته باشد، دقیقاً همان کلاسی خواهد بود که بالاترین امتیاز لگاریتمی را کسب می‌کند.

نکته برای پیاده‌سازی:

در برنامه‌نویسی و توسعه این سیستم‌ها، معمولاً مقادیر لگاریتمی پیش از فاز تست محاسبه شده و در حافظه (جداول **Hash**) ذخیره می‌شوند. این کار باعث می‌شود در زمان طبقه‌بندی اسناد جدید، پردازنده به جای انجام عملیات سنگین و کند لگاریتم‌گیری مکرر، صرفاً مجموعه‌ای از عملگرهای ساده «جمع» را اجرا کند که به شدت کارایی و سرعت سیستم را افزایش می‌دهد.

پاسخنامه مسئله 13.13: تاثیر طول سند بر مقادیر احتمال در نایو بیز

بخش 1) ماهیت ضربی در محاسبه احتمالات

در مدل نایو بیز چندجمله‌ای استاندارد، احتمال تعلق یک سند (**d**) به یک کلاس مشخص (**c**) بر اساس فرض استقلال شرطی، از طریق ضرب احتمال رخداد تک تک کلمات آن سند محاسبه می‌شود:

$$P(d|c) \propto P(c) \prod_{i=1}^{L_d} P(t_i|c)$$

در این فرمول، **L_d** نشان‌دهنده **طول سند** (تعداد کل کلمات موجود در آن) است. این یعنی عملگر ضرب (**Π**) دقیقاً به تعداد کلمات متن تکرار می‌شود.

بخش 2) رفتار ریاضی کسرها در ضرب متوالی

طبق اصول پایه نظریه احتمالات، احتمال شرطی هر کلمه ($P(t|c)$) همواره یک عدد کسری و اکیداً بین 0 و 1 است ($0 < P < 1$).

از نظر ریاضی، زمانی که شما مجموعه‌ای از اعداد کوچکتر از 1 را در یکدیگر ضرب می‌کنید، حاصل ضرب به صورت نمایی (**Exponential**) کوچک شده و به سمت 0 میل می‌کند. هرچه تعداد عواملی که در هم ضرب می‌شوند بیشتر باشد، نتیجه نهایی با سرعت بیشتری به 0 نزدیک می‌شود.

از آنجا که در الگوریتم نایو بیز، تعداد این عوامل ضرب‌شونده مستقیماً با طول سند (L_d) برابر است، 1 سند طولانی‌تر نیازمند ضرب کردن تعداد بسیار بیشتری از این کسرهای کوچک در یکدیگر است. در نتیجه، ارزش عددی نهایی احتمال برای اسناد طولانی، صرف‌نظر از محتوای آن‌ها، همواره به شدت کوچکتر از اسناد کوتاه خواهد بود.

بخش 3) مثال تحلیلی و مقایسه‌ای

فرض کنید در یک سیستم طبقه‌بند، میانگین احتمال شرطی کلمات در یک کلاس خاص حدوداً برابر با 0.1 باشد. رفتار سیستم را برای 2 سند با طول‌های مختلف بررسی می‌کنیم:

سند الف (کوتاه - 10 کلمه): احتمال نهایی این سند متناسب با ضرب 10 عدد اعشاری خواهد بود.

$$P(d_{\text{short}}|c) \approx (0.1)^{10} = 10^{-10}$$

سند ب (طولانی - 100 کلمه): احتمال نهایی این سند متناسب با ضرب 100 عدد اعشاری است.

$$P(d_{\text{long}}|c) \approx (0.1)^{100} = 10^{-100}$$

همان‌طور که مشاهده می‌شود، فقط به دلیل تفاوت در طول متن، احتمال نهایی سند طولانی‌تر، 90 مرتبه بزرگی (**Magnitude**) کوچکتر از سند کوتاه است.

بخش 4) پیامدهای عملی این پدیده (سوگیری طول)

این پدیده که به **سوگیری طول (Length Bias)** معروف است، 2 پیامد مهم در معماری سیستم‌های بازیابی اطلاعات دارد:

1. بروز خطای Underflow: همان‌طور که در مبحث «لگاریتم احتمالات» بررسی شد، این افت شدید مقادیر در اسناد طولانی، سیستم‌های کامپیوتری را با خطای حد پایین (**Floating-point Underflow**) مواجه می‌کند که تنها راه گریز از آن، استفاده از جمع لگاریتمی است.

2. عدم امکان مقایسه بین اسناد (Cross-document Incomparability): در طبقه‌بندی یک سند واحد، این پدیده مشکلی در انتخاب کلاس برنده ایجاد نمی‌کند؛ زیرا طول سند در هنگام مقایسه کلاس‌های مختلف همان سند، ثابت است. اما اگر بخواهیم **2 سند مختلف** را از نظر میزان اطمینان تعلق به یک کلاس مقایسه کنیم (مثلاً بخواهیم اسناد را بر اساس میزان ارتباط رتبه‌بندی کنیم)، مدل نایو بیز خام به شدت گمراه‌کننده است؛ زیرا همیشه اسناد کوتاه‌تر را بدون دلیل منطقی در رتبه‌های بالاتر قرار می‌دهد.

یکی از دلایل اصلی که در سیستم‌های مدرن بازیابی اطلاعات (مانند موتورهای جستجو)، مدل‌هایی نظیر **BM25** جایگزین مدل‌های ساده احتمالاتی شده‌اند، وجود مکانیزم‌های داخلی برای «نرمال‌سازی طول سند» (**Document Length Normalization**) است تا اثر مخرب اسناد بیش از حد طولانی خنثی شود.

پاسخنامه مسئله 13.14: انتخاب ویژگی مبتنی بر فرکانس و کاربرد لیست توقف

بخش 1) هدف از انتخاب ویژگی مبتنی بر فرکانس

در سیستم‌های طبقه‌بندی متن، برای کاهش ابعاد فضای ویژگی‌ها (**Dimensionality Reduction**) و جلوگیری از پیچیدگی بیش از حد محاسبات، از روش‌های انتخاب ویژگی استفاده می‌شود. یکی از ساده‌ترین روش‌ها، انتخاب **K** کلمه برتر است که بالاترین میزان تکرار (**Frequency**) را در کل دادگان آموزشی (**Corpus**) دارند. فرض اولیه این رویکرد آن است که کلمات پرتکرار، سیگنال‌های قدرتمندتری برای درک محتوای متون هستند.

بخش 2) خطر کلمات با فرکانس بالا (پدیده کلمات ساختاری)

اگر بدون هیچ‌گونه فیلتر اولیه‌ای، کلمات را صرفاً بر اساس بالاترین فرکانس انتخاب کنیم، سیستم با خطر «از دست رفتن قدرت تفکیک‌پذیری» (**Loss of Discriminative Power**) و پر شدن فضای برداری با نویز مواجه می‌شود.

بر اساس **قانون زیف (Zipf's Law)** که بر توزیع کلمات در زبان طبیعی حاکم است، کلماتی که بالاترین رتبه‌های فرکانس را در هر متنی به خود اختصاص می‌دهند، کلمات ساختاری و دستوری هستند (مانند: «از»، «به»، «در»، «است»، «و»).

چرا حضور این کلمات یک خطر مهلک برای الگوریتم است؟

توزیع یکنواخت در تمام کلاس‌ها: کلمه‌ای مانند «است» در اسناد ورزشی، سیاسی، پزشکی و اقتصادی با فرکانس تقریباً یکسانی ظاهر می‌شود. از آنجا که طبقه‌بندها (مانند نایو بیز) به دنبال کلماتی هستند که یک کلاس را از کلاس دیگر متمایز کند (**Information Gain** بالا)، کلمات ساختاری هیچ ارزش اطلاعاتی برای تصمیم‌گیری فراهم نمی‌کنند.

اشغال ظرفیت سیستم: این کلمات بی‌ارزش، بخش بزرگی از فضای ویژگی محدود ما را اشغال کرده و باعث می‌شوند کلمات تخصصی‌تر (مانند «الگوریتم»، «تورم»، «ورزشگاه») که فرکانس پایین‌تر اما قدرت تفکیک بسیار بالایی دارند، از چرخه محاسبات حذف شوند.

بخش 3) راهکار حل مسئله: استفاده از لیست توقف (Stop List)

برای خنثی کردن این خطر ریاضی، پیش از اجرای الگوریتم انتخاب ویژگی، از مرحله‌ای به نام **حذف کلمات توقف (Stop-word Removal)** استفاده می‌شود.

لیست توقف، یک فرهنگ لغت از پیش‌تعریف‌شده است که شامل رایج‌ترین کلمات فاقد ارزش معنایی مستقل در یک زبان خاص می‌شود. مراحل اصلاح سیستم به این شکل است:

1. ابتدا تمامی کلماتی که در لیست توقف وجود دارند، از متون خام فیلتر و حذف می‌شوند.

2. سپس فرکانس کلمات باقی‌مانده محاسبه می‌گردد.

3. در نهایت، کلماتی که اکنون بالاترین فرکانس را دارند (که پس از فیلترینگ، معمولاً اسامی، افعال اصلی و صفات کلیدی متن هستند) به عنوان ویژگی‌های نهایی برگزیده می‌شوند.

نکته آموزشی کلیدی:

استفاده از لیست توقف، یک فیلتر معنایی است که باعث می‌شود الگوریتم انتخاب ویژگی، به جای تمرکز کورکورانه بر روی «ساختار گرامری زبان»، دقیقاً روی «محتوای اطلاعاتی» (**Information Content**) متون متمرکز شود و دقت طبقه‌بندی به شکل چشمگیری ارتقا یابد.

پاسخنامه مسئله 13.15: مفهوم اطلاعات متقابل (Mutual Information)

بخش 1) تعریف مفهومی: اطلاعات متقابل چه چیزی را می‌سنجد؟

در سیستم‌های طبقه‌بندی متن، معیار اطلاعات متقابل (**MI** یا **Mutual Information**) به زبان بسیار ساده، «میزان وابستگی» یا «قدرت هم‌بستگی» بین یک کلمه خاص (**t**) و یک کلاس مشخص (**c**) را اندازه‌گیری می‌کند.

این معیار به ما می‌گوید: «اگر من بدانم که کلمه **t** در یک سند وجود دارد، چقدر از ابهام و عدم قطعیت من درباره اینکه آیا سند متعلق به کلاس **c** است یا خیر، کاسته می‌شود؟»

هرچه دیدن یک کلمه، راهنمای بهتر و مطمئن‌تری برای حدس زدن کلاس متن باشد، مقدار اطلاعات متقابل آن کلمه با آن کلاس بیشتر خواهد بود.

بخش 2) تحلیل منطق معیار MI در دو سناریوی ملموس

برای درک بهتر، رفتار این معیار را در مواجهه با کلمات مختلف بررسی می‌کنیم:

سناریوی اول: استقلال کامل (MI نزدیک به 0)

فرض کنید کلاس مورد نظر ما «اخبار ورزشی» است و کلمه مورد بررسی، فعل «است» یا کلمه «دیروز» باشد. دانستن اینکه کلمه «دیروز» در متن آمده، هیچ اطلاعات مفیدی درباره ورزشی بودن یا نبودن متن به ما نمی‌دهد؛ زیرا این کلمه در اخبار سیاسی، اقتصادی و حوادث نیز با همان احتمال ظاهر می‌شود. در این حالت، کلمه و کلاس کاملاً از هم مستقل هستند و فرمول ریاضی، مقدار **MI** را برای این رابطه 0 (یا بسیار نزدیک به 0) محاسبه می‌کند.

سناریوی دوم: وابستگی شدید (MI بسیار بالا)

حال فرض کنید برای همان کلاس «اخبار ورزشی»، کلمه «گلزنی» یا «استادیوم» را بررسی کنیم. حضور کلمه «استادیوم» در یک متن، یک سیگنال بسیار قدرتمند است که به شدت احتمال ورزشی بودن متن را بالا می‌برد و ابهام سیستم را از بین می‌برد. در این حالت، چون رخداد کلمه با رخداد کلاس گره خورده است، مقدار **MI** به شدت بالا می‌رود.

بخش 3) تفاوت اطلاعات متقابل با معیار فرکانس (تکرار خام)

در مسئله قبلی (انتخاب ویژگی مبتنی بر فرکانس) دیدیم که صرفاً تکیه بر تکرار کلمات، باعث انتخاب کلمات ساختاری و بی‌ارزش (مانند حروف اضافه) می‌شود.

برتری و نبوغ معیار **MI** در این است که به جای شمارشِ کورکورانه، **توزیع احتمالات** را می‌سنجد. فرمول **MI** احتمال رخداد همزمان کلمه و کلاس را با حالتی مقایسه می‌کند که فرض کنیم این 2 کاملاً تصادفی و مستقل رخ داده‌اند. به همین دلیل، **MI** به طور خودکار کلمات پرتکرار اما بی‌اثر (مانند کلمات توقف) را جریمه کرده و امتیاز آن‌ها را 0 می‌کند، اما به کلمات تخصصی که مختص به یک کلاس خاص هستند (حتی اگر فرکانس کلی آن‌ها خیلی بالا نباشد) بالاترین امتیاز را می‌دهد.

نکته آموزشی:

در معماری سیستم‌های طبقه‌بند، از معیار **MI** به عنوان یک الک (فیلتر) هوشمند برای **انتخاب ویژگی (Feature Selection)** استفاده می‌شود. سیستم برای هر کلاس، تمام کلمات دایره لغات را بر اساس امتیاز **MI** مرتب کرده و تنها **K** کلمه‌ای را نگه می‌دارد که بالاترین اطلاعات متقابل را با آن کلاس دارند. این کلمات منتخب، تبدیل به بهترین «شاخص‌ها» یا «نمایندگان» آن کلاس برای فاز آموزش و تست خواهند شد.

پاسخنامه مسئله 13.16: مفهوم آزمون کای-دو (χ^2) در انتخاب ویژگی

بخش 1) هدف از کاربرد آزمون χ^2 در طبقه‌بندی متن

در سیستم‌های پردازش متن، ما با هزاران کلمه مختلف مواجه هستیم که پردازش همه آن‌ها از نظر محاسباتی بسیار پرهزینه و باعث کاهش دقت (به دلیل وجود نویز) می‌شود. آزمون آماری **کای-دو (Chi-square)** که با نماد χ^2 نمایش داده می‌شود، یک ابزار قدرتمند برای **انتخاب ویژگی (Feature Selection)** است.

هدف اصلی از استفاده این آزمون، ارزیابی **میزان استقلال یا وابستگی آماری** بین 2 رویداد زیر است:

1. حضور یک کلمه خاص مانند t در سند.

2. تعلق آن سند به یک کلاس خاص مانند c .

الگوریتم با استفاده از این آزمون می‌خواهد کلماتی را پیدا کند که وابستگی شدیدی به یک کلاس دارند (به عنوان مثال، کلمه «مسی» وابستگی شدیدی به کلاس «ورزشی» دارد). این کلمات وابسته، بهترین شاخص‌ها برای تفکیک اسناد هستند.

بخش 2) مفهوم فرضیه صفر (Null Hypothesis)

در علم آمار، هر آزمونی با یک فرض اولیه به نام **فرضیه صفر (H_0)** آغاز می‌شود که سیستم در طول آزمون تلاش می‌کند آن را رد کند یا بپذیرد.

در کانتکست انتخاب ویژگی با آزمون χ^2 ، **فرضیه صفر بیان می‌کند که:**

«رخداد کلمه t و رخداد کلاس c 2 رویداد کاملاً مستقل از یکدیگر هستند.»

به بیان ساده‌تر، فرضیه صفر ادعا می‌کند که توزیع این کلمه در متون، کاملاً تصادفی است؛ بنابراین دیدن یا ندیدن این کلمه در یک متن، هیچ سیگنال یا اطلاعات مفیدی درباره اینکه آیا متن متعلق به کلاس c است یا خیر، به سیستم نمی‌دهد. (مانند کلمات ربط یا حروف اضافه).

بخش 3) مکانیزم تصمیم‌گیری بر اساس نتیجه آزمون

فرمول ریاضی χ^2 ، تفاوت بین «تعداد مشاهدات واقعی» کلمه در آن کلاس را با «تعداد مشاهدات مورد انتظار» (یعنی تعدادی که اگر فرضیه صفر درست بود و کلمه کاملاً مستقل بود، انتظار داشتیم ببینیم) محاسبه می‌کند.

تصمیم‌گیری سیستم برای انتخاب یا حذف یک ویژگی به این شکل انجام می‌شود:

مقدار χ^2 پایین (نزدیک به 0): نشان‌دهنده این است که تفاوت معناداری بین واقعیت و فرضیه صفر وجود ندارد. در نتیجه سیستم فرضیه صفر را می‌پذیرد (کلمه و کلاس مستقل‌اند). این کلمه فاقد ارزش اطلاعاتی است و از فضای ویژگی‌ها حذف می‌شود.

مقدار χ^2 بالا (بزرگتر از مقدار بحرانی): نشان می‌دهد که رفتار واقعی کلمه با فرضیه صفر کاملاً در تضاد است. در نتیجه سیستم فرضیه صفر را رد می‌کند. این یعنی بین کلمه و کلاس یک وابستگی آماری قدرتمند و معنادار وجود دارد، پس این کلمه به عنوان یک ویژگی کلیدی برای آموزش مدل انتخاب می‌گردد.

نکته آموزشی:

یکی از برتری‌های مهم معیار χ^2 نسبت به سایر روش‌ها (مانند **Mutual Information**) این است که به دلیل برخورداری از پشتوانه آمار کلاسیک، توزیع احتمالات آن کاملاً شناخته شده است. این ویژگی به طراح سیستم اجازه می‌دهد تا با استفاده از جداول استاندارد آماری، یک «سطح اطمینان» دقیق (مثلاً 95 درصد اطمینان) برای رد فرضیه صفر و انتخاب کلمات تعیین کند.

پاسخنامه مسئله 13.17: تحلیل سناریوی ماتریس درهم‌ریختگی در فیلتر هرزنامه

بخش 1) کالبدشکافی خطای مثبت کاذب (False Positive) در این سناریو

در ادبیات یادگیری ماشین و به طور خاص در سیستم‌های فیلترینگ ایمیل، معمولاً کلاس «هرزنامه» (**Spam**) به عنوان کلاس مثبت (**Positive**) و کلاس «ایمیل سالم/معتبر» (**Ham**) به عنوان کلاس منفی (**Negative**) تعریف می‌شود. بر اساس این منطق، خطای مثبت کاذب (**FP** یا **False Positive**) زمانی رخ می‌دهد که الگوریتم، یک ایمیل سالم و کاملاً معتبر را به اشتباه به عنوان هرزنامه (مثبت) شناسایی کرده و آن را به پوشه **Spam** منتقل کند.

بخش 2) تأثیر بالا بودن FP بر تجربه کاربری (User Experience)

افزایش نرخ **False Positive** در یک سیستم ایمیل، تأثیری بسیار مخرب و فاجعه‌بار بر تجربه و اعتماد کاربری دارد. در مسائل طبقه‌بندی دنیای واقعی، هزینه و خسارت ناشی از انواع خطاها یکسان نیست.

هزینه خطای منفی کاذب (False Negative): اگر سیستم یک هرزنامه را تشخیص ندهد و ایمیل وارد صندوق ورودی (**Inbox**) شود، کاربر صرفاً با یک مزاحمت جزئی مواجه شده و با یک کلیک آن را پاک می‌کند.

هزینه خطای مثبت کاذب (False Positive): اگر سیستم دچار این خطا شود، پیام‌های حیاتی کاربر (مانند ایمیل پذیرش دانشگاه، رمزهای عبور بانکی، یا پیام‌های کاری مهم) از دسترس او خارج می‌شوند. از دست رفتن اطلاعات معتبر، به سرعت باعث سلب اعتماد کاربر نسبت به قابلیت اطمینان (**Reliability**) سیستم خواهد شد.

بخش 3) تحلیل ریاضی معیارها: Precision در برابر Recall

برای تعیین استراتژی بهینه‌سازی، باید فرمول 2 معیار اصلی ارزیابی سیستم را در کنار هم بررسی کنیم:

معیار دقت (Precision): نشان می‌دهد چه نسبتی از ایمیل‌هایی که سیستم آن‌ها را «اسپم» تشخیص داده، واقعاً اسپم بوده‌اند.

$$\text{Precision} = \frac{TP}{TP + FP}$$

معیار پوشش یا بازیابی (Recall): نشان می‌دهد سیستم چه نسبتی از کل ایمیل‌های اسپم واقعی را توانسته است پیدا و شکار کند.

$$\text{Recall} = \frac{TP}{TP + FN}$$

بخش 4) استراتژی نهایی: کدام معیار در این سناریو مهم‌تر است؟

با توجه به تحلیل هزینه خطاها، هدف قطعی و حیاتی طراح سیستم باید به حداقل رساندن نرخ **FP** (جلوگیری از حذف ایمیل‌های سالم) باشد.

همان‌طور که در فرمول‌ها مشخص است، متغیر **FP** به طور مستقیم در مخرج کسر **Precision** قرار دارد. بنابراین، برای کاهش دادن خطای مثبت کاذب، ما باید اکیداً **معیار Precision** را بهینه‌سازی (بیشینه) کنیم.

در این سناریو، سیستم باید یک رویکرد محافظه‌کارانه (**Conservative**) اتخاذ کند. این بدان معناست که سیستم فقط زمانی یک ایمیل را اسپم اعلام می‌کند که اطمینان بسیار بالایی داشته باشد. در این معماری، فدا کردن مقداری از معیار **Recall** (یعنی اجازه دادن به ورود چند ایمیل تبلیغاتی به صندوق ورودی) بهای کاملاً منطقی و لازمی است که برای تضمین بالا نگه‌داشتن **Precision** و حفظ ایمیل‌های حیاتی کاربران پرداخته می‌شود.

نکته آموزشی:

در الگوریتم‌های یادگیری ماشین، همواره بین **Precision** و **Recall** یک تقابل و بده‌بستان (**Trade-off**) ریاضی وجود دارد. اینکه سیستم را به سمت کدام معیار متمایل کنیم، نه یک تصمیم صرفاً تئوریک، بلکه یک تصمیم وابسته به دامنه کاربرد (**Domain-specific**) است. در تشخیص اسپم، **Precision** اولویت مطلق است؛ در حالی که در یک سیستم هوش مصنوعی برای تشخیص تومورهای سرطانی، به دلیل اهمیت پیدا کردن تمام بیماران احتمالی، معیار **Recall** پادشاهی می‌کند.

پاسخنامه مسئله 13.18: سناریوی محاسبه دقیق اطلاعات متقابل (MI)

بخش 1) تشکیل جدول توافقی 2x2 (Contingency Table)

برای محاسبه اطلاعات متقابل (**Mutual Information**) بین یک کلمه خاص (در اینجا «تخفیف») و یک کلاس مشخص (در اینجا «اسپم»), ابتدا باید توزیع آماری اسناد را در قالب یک جدول توافقی سازماندهی کنیم.

فرض می‌کنیم متغیر تصادفی **U** نشان‌دهنده وضعیت حضور کلمه «تخفیف» (مقدار 1 برای حضور، مقدار 0 برای غیبت) و متغیر تصادفی **C** نشان‌دهنده وضعیت تعلق سند به کلاس «اسپم» (مقدار 1 برای اسپم، مقدار 0 برای غیر اسپم) باشد.

جدول ساختاریافته شمارش اسناد به شرح زیر تشکیل می‌شود:

وضعیت کلمه «تخفیف» (U)	کلاس اسپم (C=1)	کلاس غیر اسپم (C=0)	جمع سطری
حضور دارد (U=1)	N_{11}	N_{10}	$N_{1\bullet}$ (تعداد کل اسناد حاوی کلمه)
غایب است (U=0)	N_{01}	N_{00}	$N_{0\bullet}$ (تعداد کل اسناد فاقد کلمه)
جمع ستونی	$N_{\bullet 1}$ (کل اسپم‌ها)	$N_{\bullet 0}$ (کل غیر اسپم‌ها)	N (تعداد کل اسناد سیستم)

راهنمای متغیرها:

N₁₁: تعداد ایمیل‌های اسپمی که کلمه «تخفیف» در آن‌ها وجود دارد.

N₁₀: تعداد ایمیل‌های سالمی (غیر اسپم) که کلمه «تخفیف» در آن‌ها وجود دارد.

N₀₁: تعداد ایمیل‌های اسپمی که کلمه «تخفیف» را ندارند.

N₀₀: تعداد ایمیل‌های سالمی که کلمه «تخفیف» را ندارند.

بخش 2) بسط ریاضی فرمول اطلاعات متقابل I(U;C)

فرمول پایه و استاندارد اطلاعات متقابل در نظریه اطلاعات به صورت زیر تعریف می‌شود:

$$I(U;C) = \sum_{e_t \in \{1,0\}} \sum_{e_c \in \{1,0\}} P(U = e_t, C = e_c) \log_2 \left(\frac{P(U = e_t, C = e_c)}{P(U = e_t)P(C = e_c)} \right)$$

این فرمول سیگما، در واقع مجموع 4 حالت ممکن (4 سلول جدول توافقی) است. با تبدیل مقادیر احتمال (**P**) به کسرهای مبتنی بر شمارش (**N**) که از جدول به دست آمده‌اند، فرمول را به 4 عبارت مجزا بسط می‌دهیم:

عبارت اول (کلمه حضور دارد و کلاس اسپم است):

$$\frac{N_{11}}{N} \log_2 \left(\frac{N \times N_{11}}{(N_{11} + N_{10}) \times (N_{11} + N_{01})} \right)$$

عبارت دوم (کلمه حضور دارد و کلاس غیر اسپم است):

$$\frac{N_{10}}{N} \log_2 \left(\frac{N \times N_{10}}{(N_{11} + N_{10}) \times (N_{10} + N_{00})} \right)$$

عبارت سوم (کلمه غایب است و کلاس اسپم است):

$$\frac{N_{01}}{N} \log_2 \left(\frac{N \times N_{01}}{(N_{01} + N_{00}) \times (N_{11} + N_{01})} \right)$$

عبارت چهارم (کلمه غایب است و کلاس غیر اسپم است):

$$\frac{N_{00}}{N} \log_2 \left(\frac{N \times N_{00}}{(N_{01} + N_{00}) \times (N_{10} + N_{00})} \right)$$

در نهایت، مقدار $I(\mathbf{U}; \mathbf{C})$ برابر است با مجموع جبری این 4 عبارت.

بخش 3) مثال محاسباتی برای درک بهتر

برای ملموس شدن فرمول بسطیافته، فرض کنید مجموعه‌ای شامل 100 ایمیل ($N = 100$) داریم که توزیع آن‌ها به این شکل است:

30 ایمیل اسپم حاوی کلمه «تخفیف» ($N_{11} = 30$)

5 ایمیل سالم حاوی کلمه «تخفیف» ($N_{10} = 5$)

15 ایمیل اسپم فاقد کلمه «تخفیف» ($N_{01} = 15$)

50 ایمیل سالم فاقد کلمه «تخفیف» ($N_{00} = 50$)

با جایگذاری مقادیر سلول اول (N_{11}) در عبارت اول فرمول بسطیافته خواهیم داشت:

$$\text{Term 1} = \frac{30}{100} \log_2 \left(\frac{100 \times 30}{(35) \times (45)} \right) = 0.3 \times \log_2 \left(\frac{3000}{1575} \right) \approx 0.3 \times 0.93 =$$

پس از محاسبه هر 4 عبارت و جمع آن‌ها، عدد نهایی که واحد آن «بیت» است به دست می‌آید.

نکته:

ساختار لگاریتم در این فرمول به گونه‌ای است که صورت کسر نشان‌دهنده «رخداد همزمان واقعی» و مخرج کسر نشان‌دهنده «رخداد در حالت استقلال کامل» است. اگر کلمه «تخفیف» هیچ ربطی به کلاس «اسپم» نداشته باشد، صورت و مخرج کسر داخل لگاریتم تقریباً برابر می‌شوند. از آنجا که $\log_2(1) = 0$ است، ارزش اطلاعاتی این کلمه 0 شده و در فرآیند انتخاب ویژگی (**Feature Selection**) از سیستم حذف می‌گردد.

پاسخنامه مسئله 13.19: چالش کلمات توقف در مدل برنولی

بخش 1) تحلیل رفتار آماری کلمات توقف در فضای کلاس‌ها

کلمات توقف (مانند «از»، «در»، «است»، «و») کلمات ساختاری زبان هستند که در اکثر متون، مستقل از موضوع و کلاس سند، حضور دارند.

از منظر آماری، اگر مجموعه داده‌ای شامل 2 کلاس c_1 (مثلاً ورزشی) و c_2 (مثلاً سیاسی) باشد، احتمال رخداد یک کلمه توقف مانند t_{stop} در هر 2 کلاس بسیار بالا و تقریباً نزدیک به 1 است:

$$P(t_{stop}|c_1) \approx 1 \quad \text{و} \quad P(t_{stop}|c_2) \approx 1$$

بخش 2) اثر ریاضی بر روی تفکیک‌پذیری (Discriminative Power)

مدل برنولی برای تصمیم‌گیری، به دنبال کلماتی است که در یک کلاس حضور پررنگ و در کلاس دیگر حضور کم‌رنگی داشته باشند تا بتواند مرز تصمیم‌گیری (**Decision Boundary**) را رسم کند. عدم حذف کلمات توقف، تفکیک‌پذیری سیستم را از 2 جهت به شدت تخریب می‌کند:

الف) تولید ضرایب خنثی و بی‌اثر: هنگامی که یک سند جدید برای طبقه‌بندی وارد سیستم می‌شود، مدل برنولی بررسی می‌کند که آیا کلمه t_{stop} در آن وجود دارد یا خیر ($x_t = 1$). چون این کلمه در سند حضور دارد، سیستم احتمال $P(t_{stop}|c)$ را برای تمام کلاس‌ها در محاسبات ضرب می‌کند. از آنجا که این احتمال برای همه کلاس‌ها عدد یکسانی (نزدیک به 1) است، ضرب کردن آن در امتیاز کلاس‌ها، هیچ تفاوتی در رتبه‌بندی نهایی ایجاد نمی‌کند.

ب) غلبه نویز بر سیگنال (Dilution of Signal): اسناد متنی معمولاً شامل تعداد بسیار زیادی کلمه توقف و تعداد محدودی کلمات کلیدی و تخصصی هستند. در مدل برنولی (که بردارها به صورت 0 و 1 هستند)، حضور همزمان ده‌ها کلمه توقف مشترک بین کلاس‌ها باعث می‌شود که بردارهای اسناد کلاس‌های مختلف، شباهت ریاضی بسیار زیادی به یکدیگر پیدا کنند (افزایش شباهت کسینوسی). در این حالت، وزن و تأثیر کلمات اصلی و متمایزکننده، در میان انبوهی از ویژگی‌های تکراری و بی‌ارزش «گم» می‌شود.

بخش 3) خطر پنهان: جریمه‌های نامتناسب در اسناد کوتاه

یکی از ویژگی‌های متمایز مدل برنولی این است که **عدم حضور** یک کلمه را نیز در محاسبات دخالت می‌دهد (از طریق فرمول $1 - P(t|c)$). حال فرض کنید در یک سند متنی بسیار کوتاه (مثلاً یک پیامک یا توییت)، به صورت تصادفی یکی از کلمات توقف (مثلاً کلمه «است») به کار نرفته باشد ($x_t = 0$).

الگوریتم برنولی متوجه غیبت این کلمه می‌شود و فرمول عدم حضور را اعمال می‌کند:

$$1 - P(t_{stop}|c) \approx 1 - 0.99 = 0.01$$

این عددِ بسیار کوچک (0.01) در امتیاز تمام کلاس‌ها ضرب می‌شود و شانس تعلق سند به تمامی کلاس‌ها را به صورت ناگهانی و شدید افت می‌دهد. این یک **جریمه کاملاً غیرمنطقی و مخرب** است؛ زیرا غیبت یک کلمه ساختاری در یک متن کوتاه، نباید ارزش و اعتبار کلمات کلیدی موجود در آن متن را از بین ببرد.

نتیجه‌گیری:

در مدل برنولی، عدم حذف کلمات توقف بسیار خطرناک‌تر از مدل چندجمله‌ای (**Multinomial**) است. در مدل چندجمله‌ای، تکرار بالای کلمات کلیدی می‌تواند تا حدی اثر کلمات توقف را جبران کند، اما در مدل برنولی که همه‌چیز به صورت باینری (0 و 1) است، کلمات توقف به سرعت فضای ویژگی‌ها را اشغال کرده، اسناد کلاس‌های مختلف را به هم شبیه می‌سازند و قدرت تفکیک الگوریتم را از بین می‌برند. به همین دلیل، اجرای **فیلتر لیست توقف (Stop-list)** پیش از تشکیل بردارهای برنولی، یک الزام معماری است، نه یک انتخاب.

پاسخنامه مسئله 13.20: تحلیل سناریوی داده‌های نامتوازن (Imbalanced Data)

بخش 1) محاسبه معیار دقت (Accuracy)

معیار دقت (**Accuracy**) در ارزیابی سیستم‌های طبقه‌بند، از تقسیم «تعداد پیش‌بینی‌های کاملاً صحیح سیستم» بر «کل تعداد اسناد ارزیابی‌شده» به دست می‌آید.

در این سناریو، مجموعه داده‌های ما در مجموع شامل 10,000 سند است (9000 ورزشی + 1000 اقتصادی).

اگر سیستم (بدون خواندن حتی یک کلمه از متون) صرفاً بر اساس یک قاعده ثابت، تمامی 10,000 سند ورودی را «ورزشی» برچسب‌گذاری کند، نتایج به این شکل رقم می‌خورد:

پیش‌بینی‌های درست (اسناد ورزشی): سیستم 9000 سند ورزشی را به درستی «ورزشی» تشخیص داده است.

پیش‌بینی‌های غلط (اسناد اقتصادی): سیستم 1000 سند اقتصادی را به اشتباه «ورزشی» تشخیص داده است.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Documents}} = \frac{9000}{10000} = 0.90$$

بنابراین، مقدار **Accuracy** برای این سیستم کاملاً ناکارآمد، روی کاغذ برابر با **90 درصد** خواهد بود!

بخش 2) چرا معیار Accuracy در این سناریو به شدت گمراه‌کننده است؟

این پدیده در علوم داده و آمار به **پارادوکس دقت (Accuracy Paradox)** معروف است. تکیه بر این معیار در مجموعه داده‌های نامتوازن، به 2 دلیل زیر طراح سیستم را به اشتباه می‌اندازد:

توهم یادگیری و هوشمندی: عدد خیره‌کننده 90 درصد این تصور را القا می‌کند که الگوریتم، الگوهای پیچیده

زبان و تفاوت متون ورزشی و اقتصادی را به خوبی آموخته است. اما در واقعیت، سیستم عملاً هیچ ویژگی یا

کلمه‌ای را پردازش نکرده و نابینا است. الگوریتم تنها متوجه «توزیع پیشین» (**Prior Probability**) داده‌ها شده و

ارزان‌ترین راهکار محاسباتی یعنی «انتخاب همیشگی کلاس اکثریت» را برگزیده است.

نابینایی مطلق در برابر کلاس اقلیت (Minority Class): در بسیاری از مسائل واقعی، کلاسی که داده‌های کمتری دارد، دقیقاً همان کلاسی است که کشف آن حیاتی‌تر است. فرض کنید به جای متون اقتصادی، این 1000 سند، ایمیل‌های حاوی بدافزار باشند. سیستمی که همه ورودی‌ها را «سالم» برچسب بزند، اگرچه دقت 90 درصدی دارد، اما در رسیدن به هدف اصلی خود (پیدا کردن بدافزارها) دچار شکست 100 درصدی شده و یک سیستم کاملاً بی‌مصرف است.

نکته آموزشی:

قانون طلایی در ارزیابی سیستم‌های بازیابی اطلاعات این است که **هرگز نباید از معیار Accuracy برای ارزیابی مجموعه داده‌های نامتوازن (Imbalanced Datasets) استفاده کرد.**

در چنین شرایطی، معماری ارزیابی باید به سمت معیارهای تفکیک‌شده برای هر کلاس، نظیر **Precision** (دقت)، **Recall** (بازیابی) و به ویژه ترکیب هارمونیک آن‌ها یعنی **F1-Score** حرکت کند.

اگر سناریوی فوق را با این معیارهای استاندارد بسنجیم، معیار **Recall** (پوشش) برای کلاس اقتصادی دقیقاً برابر با 0 محاسبه می‌شود که بلافاصله ناکارآمدی و فاجعه‌بار بودن سیستم را برای مهندس طراح افشا می‌کند.

پاسخنامه مسئله 13.21: ارزیابی میکرو در برابر ماکرو در داده‌های نامتوازن

بخش 1) تبیین مفاهیم: تفاوت بنیادین در رویکرد میانگین‌گیری

هنگامی که با یک مسئله دسته‌بندی با بیش از 2 کلاس (**Multi-class Classification**) مواجه هستیم، برای گزارش کردن عملکرد کلی سیستم باید از معیارهای ترکیبی استفاده کنیم. دو رویکرد اصلی برای این کار وجود دارد:

میانگین ماکرو (Macro-F1): در این روش، ابتدا مقدار **F1-Score** برای تک‌تک کلاس‌ها به صورت کاملاً مستقل محاسبه می‌شود و در نهایت، میانگین حسابی ساده‌ای از این مقادیر گرفته می‌شود. در این رویکرد، «ارزش و وزن تمامی کلاس‌ها با یکدیگر برابر است»، فارغ از اینکه هر کلاس چه تعداد سند در خود جای داده باشد.

میانگین میکرو (Micro-F1): در این روش، مرزهای بین کلاس‌ها برداشته می‌شود. سیستم تمام تصمیمات درست (**True Positives**) و تمام خطاها (**False Positives** و **False Negatives**) را از تمام کلاس‌ها روی هم می‌ریزد (تجمیع می‌کند) و در نهایت فقط 1 بار فرمول **F1** را برای کل این توده عظیم داده محاسبه می‌کند. در این رویکرد، «ارزش تمامی اسناد (نمونه‌ها) با یکدیگر برابر است».

بخش 2) پاسخ به سناریو: کدام معیار ضعف سیستم را آشکار می‌کند؟

در سناریوی مطرح‌شده (داده‌های به شدت نامتوازن با عملکرد ضعیف در کلاس‌های اقلیت)، معیار **Macro-F1** به مراتب ضعف سیستم را بهتر و شفاف‌تر آشکار می‌کند.

دلیل و تحلیل منطقی:

معیار **Micro-F1** به شدت تحت تأثیر کلاس اکثریت (**Majority Class**) قرار دارد. از آنجا که حجم عظیمی از داده‌ها متعلق به کلاس اکثریت است، تعداد بالای پیش‌بینی‌های درست (**TP**) در این کلاس، خطاهای رخ داده در کلاس‌های اقلیت را در فرمول تجمیعی می‌پوشاند و خروجی نهایی را به صورت کاذب بالا نشان می‌دهد.

اما معیار **Macro-F1**، به کلاس‌های اقلیت دقیقاً همان‌قدر وزن می‌دهد که به کلاس اکثریت می‌دهد. بنابراین، شکست سیستم در کلاس‌های اقلیت، مستقیماً میانگین نهایی را به شدت پایین می‌کشد و مهندس طراح را از ناکارآمدی سیستم آگاه می‌سازد.

بخش 3) اثبات ریاضی با یک مثال فرضی

برای اثبات این ادعا، سناریوی مسئله را با اعداد مدل‌سازی می‌کنیم:
فرض کنید مجموعه‌ای شامل 1000 سند داریم که به 3 کلاس A (اکثریت)، B (اقلیت) و C (اقلیت) تقسیم شده‌اند.

کلاس 900 A: سند. سیستم در این کلاس عالی کار کرده و هر 900 سند را درست تشخیص داده است.
 $F1_A \approx 0.95$ (TP=900, FP=100, FN=0) → در نتیجه

کلاس 50 B: سند. سیستم در این کلاس کاملاً شکست خورده است. **$F1_B = 0.0$ (TP=0, FN=50) → در نتیجه**
کلاس 50 C: سند. سیستم در این کلاس نیز کاملاً شکست خورده است. **$F1_C = 0.0$ (TP=0, FN=50) → در نتیجه**

حال عملکرد کلی سیستم را با هر 2 معیار محاسبه می‌کنیم:

محاسبه Macro-F1:

فرمول ماکرو صرفاً میانگین امتیاز کلاس‌هاست:

$$\text{Macro-F1} = \frac{F1_A + F1_B + F1_C}{3} = \frac{0.95 + 0.0 + 0.0}{3} \approx 0.31$$

(نتیجه: عدد ضعیف 31 درصد، که به درستی نشان‌دهنده فاجعه‌بار بودن سیستم در 2 کلاس از 3 کلاس است.)

محاسبه Micro-F1:

برای محاسبه میکرو، ابتدا تمام پارامترها را جمع می‌زنیم:

$$\text{مجموع TP} = 900 + 0 + 0 = 900$$

مجموع تمام اسناد = 1000

$$\text{Micro Precision} = \frac{900}{1000} = 0.90$$

$$\text{Micro Recall} = \frac{900}{1000} = 0.90$$

$$\text{Micro-F1} \approx 0.90$$

(نتیجه: عدد فریبنده 90 درصد، که ضعف مطلق سیستم در کلاس‌های اقلیت را به طور کامل پنهان کرده است.)

در پروژه‌های واقعی و دانشگاهی که با داده‌های نامتوازن (**Imbalanced Datasets**) سروکار دارند (مانند تشخیص تقلب‌های بانکی یا دسته‌بندی شکایات)، همواره باید از **Macro-F1** به عنوان معیار اصلی قضاوت و ارزیابی سیستم استفاده کرد تا اطمینان حاصل شود الگوریتم به کلاس‌های نادر اما مهم نیز توجه کافی داشته است.

پاسخنامه مسئله 13.22: مقایسه روش‌های انتخاب ویژگی (کای-دو در برابر فراوانی)

بخش 1) تفاوت بنیادین در منطق انتخاب

برای درک این سناریو، ابتدا باید تفاوت دیدگاه این 2 روش را نسبت به ارزش یک کلمه مشخص کنیم:

روش مبتنی بر فراوانی (Frequency-based): این روش تنها به «حجم و کمیت» نگاه می‌کند. الگوریتم تمام

کلمات مجموعه داده را می‌شمارد و کلماتی که بیشترین تکرار را در کل اسناد دارند، انتخاب می‌کند. این روش توزیع کلمه در کلاس‌های مختلف را کاملاً نادیده می‌گیرد.

روش کای-دو (χ^2): این روش به «وابستگی و قدرت تفکیک» نگاه می‌کند. الگوریتم بررسی می‌کند که آیا حضور یک کلمه، وابستگی و همبستگی شدیدی با یک کلاس خاص دارد یا خیر. در این روش، نحوه توزیع کلمه بسیار مهم‌تر از تعداد کل رخدادهای آن است.

بخش 2) سناریوی وقوع: کلمات نادر اما به شدت تخصصی (Rare but Highly Discriminative)

سناریویی که در آن روش χ^2 کلمه‌ای را انتخاب کرده اما روش فراوانی آن را دور می‌اندازد، زمانی رخ می‌دهد که با یک «کلمه کاملاً تخصصی با فرکانس پایین» مواجه باشیم.

به عبارت دقیق‌تر، این سناریو زمانی شکل می‌گیرد که کلمه‌ای در کل مجموعه دادگان (**Corpus**) به ندرت استفاده شده باشد (بنابراین در رتبه‌بندی فراوانی در انتهای لیست قرار می‌گیرد و حذف می‌شود)، اما همان تعداد اندک رخداد، منحصراً و به طور 100 درصدی فقط در یک کلاس خاص اتفاق افتاده باشد.

بخش 3) مثال ملموس و تحلیلی

فرض کنید در حال ساخت یک سیستم طبقه‌بندی اخبار با 100 هزار سند در 4 کلاس مختلف هستیم: سیاسی، اقتصادی، ورزشی و پزشکی. ما می‌خواهیم تنها 1000 کلمه برتر را به عنوان ویژگی‌های سیستم انتخاب کنیم (کاهش ابعاد).

کلمه «آریتمی» (بی‌نظمی ضربان قلب) را در این مجموعه در نظر بگیرید:

آمار خام: این کلمه در کل 100 هزار سند، تنها 50 بار تکرار شده است. اما تمامی این 50 رخداد، دقیقاً در اسناد

کلاس «پزشکی» قرار دارند و در هیچ کلاس دیگری حتی 1 بار هم دیده نشده‌اند.

رفتار روش فراوانی (Frequency-based):

الگوریتم مبتنی بر فراوانی کلمات را مرتب می‌کند. کلماتی مانند «دولت»، «بازار»، «تیم» و «بیمارستان» هزاران بار تکرار شده‌اند و رتبه‌های 1 تا هزارم را اشغال می‌کنند. کلمه «آریتمی» با فرکانس بسیار پایین 50، در رتبه مثلاً 15000 قرار می‌گیرد. در نتیجه، این روش با بی‌رحمی این کلمه را فاقد ارزش دانسته و آن را از فضای ویژگی‌ها حذف می‌کند.

رفتار روش کای-دو (χ^2):

الگوریتم χ^2 یک جدول توافقی برای کلمه «آریتمی» و کلاس «پزشکی» تشکیل می‌دهد. فرضیه صفر (استقلال) می‌گوید: «از آنجا که این کلمه در کل 50 بار تکرار شده و ما 4 کلاس مساوی داریم، انتظار می‌رود در هر کلاس حدود 12.5 بار دیده شود.»

اما در واقعیت (مشاهدات)، این کلمه 50 بار در کلاس پزشکی و 0 بار در سایر کلاس‌ها دیده شده است. این اختلاف فاحش بین «توزیع مورد انتظار» و «توزیع واقعی»، باعث تولید یک مقدار χ^2 بسیار بزرگ برای این کلمه می‌شود.

الگوریتم متوجه می‌شود که کلمه «آریتمی» یک **سیگنال خالص و بدون نویز** برای تشخیص اخبار پزشکی است. بنابراین فرضیه صفر را رد کرده و این کلمه را با بالاترین امتیاز به عنوان یک ویژگی کلیدی **انتخاب می‌کند**.

نتیجه‌گیری:

روش‌های مبتنی بر فراوانی، به دلیل تمرکز انحصاری بر کمیت، معمولاً فضای ویژگی‌ها را با کلمات عمومی (**General Words**) پر می‌کنند. در مقابل، روش‌های مبتنی بر نظریه اطلاعات و آمار (مانند χ^2 و **Mutual Information**) قادرند اصطلاحات تخصصی و نادر (**Rare Vocabulary**) را که قدرت تفکیک و ارزش معنایی فوق‌العاده بالایی برای یک کلاس خاص دارند، نجات داده و به سیستم معرفی کنند.

پاسخنامه مسئله 13.23: کنترل اثر طغیان کلمات (Burstiness) در مدل چندجمله‌ای

بخش 1) پدیده طغیان کلمات (Burstiness) چیست؟

در تحلیل آماری زبان طبیعی، پدیده‌ای به نام **طغیان یا رگبار کلمات (Burstiness)** وجود دارد. این مفهوم بیان می‌کند که کلمات در یک متن به صورت یکنواخت توزیع نمی‌شوند؛ بلکه اگر یک کلمه خاص (به ویژه یک کلمه تخصصی یا نام خاص) 1 بار در سندی ظاهر شود، احتمال اینکه در همان سند بارها و بارها تکرار شود، به شدت افزایش می‌یابد.

در مدل چندجمله‌ای استاندارد (**Standard Multinomial Model**)، احتمال شرطی هر کلمه دقیقاً به تعداد دفعات تکرار آن در متن (**tf**) ضرب می‌شود. این امر یک آسیب‌پذیری بزرگ ایجاد می‌کند: تکرار بیش از حد و رگباری یک کلمه در متن، می‌تواند به تنهایی احتمال نهایی را به شدت تحت تسلط خود درآورده و سیستم را دچار یک سوگیری (**Bias**) کاذب کند. در واقع، اهمیت حضور 10 باره یک کلمه در متن، لزوماً 10 برابر حضور 1 باره آن نیست.

بخش 2) راهکار ریاضی: مدل چندجمله‌ای دودویی (Binarized Multinomial Model)

برای خنثی کردن اثر مخرب این پدیده، یک تغییر و هرس بسیار ساده در مرحله «نمایش سند» (**Document Representation**) ایجاد می‌شود. به جای استفاده از فراوانی خام کلمات، از **فراوانی برش‌یافته (Clipped Term Frequency)** استفاده می‌کنیم.

در این روش که یک نسخه اصلاح‌شده از مدل چندجمله‌ای است، مقادیر فراوانی هر کلمه در سند (**tf_{t,d}**) پیش از ورود به فرمول محاسباتی، به شکل زیر به یک متغیر باینری (**tf'_{t,d}**) نگاشت می‌شود:

$$tf'_{t,d} = \begin{cases} 1 & \text{if } tf_{t,d} > 0 \\ 0 & \text{if } tf_{t,d} = 0 \end{cases}$$

بخش 3) تغییر در فرمول محاسبه احتمال

با اعمال این تغییر، دیگر نیازی نیست احتمال شرطی یک کلمه را به توان دفعات تکرار آن برسانیم. فرمول محاسبه احتمال درست‌نمایی برای یک سند جدید، به شکل زیر تعدیل می‌شود:

$$P(d|c) \propto P(c) \prod_{t \in V_d} P(t|c)$$

در این رابطه:

V_d : نمایانگر کلمات یکتای موجود در سند d است.

همان‌طور که مشاهده می‌شود، هر کلمه‌ای که در متن حضور داشته باشد (فارغ از اینکه 1 بار تکرار شده باشد یا 50 بار)، احتمال شرطی آن فقط و فقط 1 بار در محاسبات ضرب می‌شود.

بخش 4) مرزبندی با مدل برنولی

استفاده از مقادیر باینری در این روش، معمولاً باعث اشتباه گرفتن آن با «مدل برنولی» می‌شود. اما این 2 مدل تفاوت‌های بنیادین ریاضی دارند:

ویژگی محاسباتی	مدل برنولی	مدل چندجمله‌ای دودویی (Binarized Multinomial)
محاسبه احتمال شرطی $P(t c)$	بر اساس کسر تعداد اسناد $(\text{Document Frequency})$	بر اساس کسر طول کل متون (Term Frequency) در کل کلاس
برخورد با کلمات غایب (عدم حضور)	جریمه می‌کند (عبارت $1 - P(t c)$ محاسبه می‌شود)	کاملاً نادیده می‌گیرد و هیچ تاثیری در فرمول ندارد
حلقه ضرب (Π)	روی تمام کلمات دایره لغات (V) می‌چرخد	فقط روی کلمات موجود در همان سند V_d (می‌چرخد)

نکته:

مدل چندجمله‌ای دودویی، در واقع بهترین ویژگی‌های هر 2 مدل پایه را ترکیب می‌کند. این روش از یک سو مانند مدل برنولی از غلبه ناعادلانه کلمات پرتکرار جلوگیری می‌کند، و از سوی دیگر مانند مدل چندجمله‌ای، مشکل افت شدید احتمال ناشی از جریمه کردن کلمات غایب در اسناد کوتاه را به همراه ندارد. در کاربردهای عملیاتی برای متون نسبتاً طولانی، این نسخه اصلاح‌شده غالباً دقت **(Accuracy)** بهتری نسبت به نسخه استاندارد ارائه می‌دهد.

پاسخنامه تشریحی مسئله 13.24: چالش ویژگی‌های کمیاب در معیار اطلاعات متقابل (MI)

بخش 1) توهم ریاضی: چرا کلمات نادر امتیاز MI بالایی می‌گیرند؟

معیار اطلاعات متقابل **(Mutual Information)**، کاهش ابهام سیستم را اندازه‌گیری می‌کند. فرمول **MI** به گونه‌ای طراحی شده است که اگر حضور یک کلمه بتواند با قطعیت بالا کلاس یک سند را پیش‌بینی کند، بالاترین امتیاز ممکن را به آن

کلمه اختصاص می‌دهد.

حال فرض کنید کلمه‌ای (مثلاً یک غلط املائی مانند «دانشججو») در میان 100,000 سند آموزشی، فقط و فقط 1 بار رخ داده باشد و تصادفاً این کلمه در سندی متعلق به کلاس «علمی» باشد.

از دیدگاه کاملاً فرمولی و خام ریاضی، الگوریتم استدلال زیر را انجام می‌دهد:

«هر زمان که کلمه 'دانشججو' در متنی دیده شده (که کلاً 1 بار بوده است)، آن متن 100 درصد متعلق به کلاس علمی بوده است. پس این کلمه یک پیش‌بینی‌کننده بی‌نقص (**Perfect Predictor**) است!»

در این حالت، احتمال شرطی **P(Class | Word)** دقیقاً برابر با 1.0 می‌شود. فرمول **MI** این «همبستگی 100 درصدی» را به عنوان یک سیگنال اطلاعاتی فوق‌العاده قدرتمند تفسیر کرده و امتیاز بسیار بالایی (حتی بالاتر از کلمات کلیدی و پرتکرار) به این کلمه نادر می‌دهد.

بخش 2) پیامد سیستم: خطای بیش‌برازش (Overfitting)

این پدیده در واقع یک نقص در منطق تئوری اطلاعات نیست، بلکه ناشی از فقدان معناداری آماری (**Lack of Statistical Significance**) است.

در دنیای واقعی، کلمه‌ای که فقط 1 بار دیده شده (چه غلط املائی باشد، چه یک نام خاص بسیار نادر)، یک نویز تصادفی است و هیچ ارزش تعمیم‌پذیری (**Generalization**) ندارد. اگر سیستم فریب امتیاز بالای **MI** را بخورد و این کلمات 1 بار مصرف را به عنوان ویژگی‌های کلیدی برای فاز آموزش انتخاب کند، به شدت دچار خطای بیش‌برازش (**Overfitting**) می‌شود؛ یعنی سیستم جزئیات بی‌ارزش داده‌های آموزشی را حفظ می‌کند اما در مواجهه با اسناد جدید در دنیای واقعی کاملاً فلج و ناکارآمد خواهد بود.

بخش 3) راهکار عملی در معماری سیستم: آستانه فراوانی اسناد (DF Thresholding)

برای رفع این نقص بزرگ، مهندسان سیستم‌های بازیابی اطلاعات هرگز معیار **MI** را مستقیماً روی داده‌های خام اجرا نمی‌کنند.

راهکار استاندارد، استفاده از یک فیلتر پیش‌پردازش بسیار ساده اما حیاتی به نام **حد آستانه فراوانی سند (Document Frequency Thresholding)** است.

مراحل اجرای این راهکار به شکل زیر است:

1. ابتدا تعداد اسنادی که هر کلمه در آن‌ها حداقل 1 بار ظاهر شده است (**Document Frequency** یا **DF**) محاسبه می‌شود.

2. یک حد آستانه (**Threshold**) حداقلی، مثلاً $\theta = 3$ یا $\theta = 5$ تعیین می‌شود.

3. پیش از محاسبه هرگونه معیار ارزیابی ویژگی (چه **MI** و چه کای-دو)، تمامی کلماتی که **DF** آن‌ها کمتر از این آستانه باشد (کلمات نادر و 1 بار مصرف)، به عنوان نویز آماری در نظر گرفته شده و با قاطعیت از دایره لغات سیستم حذف می‌شوند.

4. در نهایت، معیار **MI** تنها برای کلمات باقی‌مانده (که حداقل به اندازه کافی تکرار شده‌اند تا معنادار باشند) محاسبه می‌گردد.

معیار اطلاعات متقابل (MI) به شدت نسبت به احتمالات حاشیه‌ای (Marginal Probabilities) حساس است و تمایل عجیبی به بزرگ‌نمایی ارزش کلمات نادر دارد. این در حالی است که آزمون آماری کای-دو (χ^2) به دلیل ماهیت استنتاجی خود، مقاومت نسبتاً بهتری در برابر کلمات نادر نشان می‌دهد؛ زیرا در فرمول χ^2 ، برای رد فرضیه صفر به «تعداد مشاهدات کافی» نیاز است و 1 مشاهده منفرد معمولاً برای تولید امتیاز χ^2 بالا کفایت نمی‌کند. با این وجود، اعمال فیلتر DF پیش از هر 2 روش، یک استاندارد قطعی در صنعت است.

پاسخنامه مسئله 13.25: انتخاب ویژگی سراسری در طبقه‌بندهای چندگانه

بخش 1) طرح مسئله: نیاز به امتیاز سراسری (Global Score)

در فرآیند انتخاب ویژگی برای سیستم‌هایی با بیش از 2 کلاس، معیار ارزیابی (مانند MI یا χ^2) همواره به صورت «کلاس‌محور» (Class-specific) محاسبه می‌شود. به این معنا که ما برای یک کلمه واحد، به تعداد کلاس‌های سیستم، امتیاز وابستگی خواهیم داشت.

در سناریوی شما، کلمه «پایتون» 100 امتیاز MI متفاوت دریافت کرده است. اما معماری سیستم در نهایت برای تشکیل بردارهای متنی (Vector Space Model) تنها به یک دایره لغات سراسری (Global Vocabulary) نیاز دارد. سیستم باید با تخصیص یک «امتیاز واحد» تصمیم بگیرد که آیا کلمه «پایتون» را در این دایره لغات نگه دارد یا آن را به عنوان نویز حذف کند.

بر اساس مفاهیم استاندارد بازیابی اطلاعات (بخش 13.5.4)، برای تبدیل امتیازات کلاس‌محور $A(t, c)$ به یک امتیاز سراسری $A(t)$ ، 2 استراتژی و فرمول اصلی وجود دارد:

بخش 2) استراتژی اول: رویکرد بیشینه‌سازی (Maximum Strategy)

در این روش، امتیاز سراسری یک کلمه، برابر با بزرگترین امتیازی در نظر گرفته می‌شود که آن کلمه در میان تمامی کلاس‌ها کسب کرده است.

$$A_{\max}(t) = \max_{c \in C} A(t, c)$$

منطق این روش: اگر یک کلمه برای تفکیک حتی 1 کلاس بسیار قدرتمند و حیاتی باشد، یک ویژگی ارزشمند است و باید در سیستم حفظ شود.

تحلیل سناریوی «پایتون»: کلمه «پایتون» احتمالاً در 99 کلاس (مانند آشپزی، ورزش، سیاست و...) امتیاز MI

بسیار پایینی نزدیک به 0 می‌گیرد. اما در کلاس «برنامه‌نویسی»، امتیاز MI آن به شدت بالا خواهد بود. با

استفاده از تابع max، امتیاز سراسری کلمه برابر با همان امتیاز کلاس برنامه‌نویسی خواهد شد و سیستم با

قدرت این کلمه را حفظ می‌کند. این استراتژی بهترین رویکرد برای نجات کلمات تخصصی نادر است.

بخش 3) استراتژی دوم: رویکرد امید ریاضی / میانگین (Expected Value Strategy)

در این روش، امتیاز سراسری به صورت **میانگین وزن دار (Weighted Average)** امتیازات آن کلمه در تمامی کلاس‌ها محاسبه می‌شود. وزن هر کلاس برابر با احتمال پیشین آن کلاس (**P(c)**) است.

$$A_{avg}(t) = \sum_{c \in C} P(c) \times A(t, c)$$

منطق این روش: این فرمول محاسبه می‌کند که به طور میانگین، حضور این کلمه چقدر اطلاعات مفید درباره کل ساختار دسته‌بندی به ما می‌دهد.

تحلیل سناریوی «پایتون»: این استراتژی برای کلماتی مانند «پایتون» **خطرناک و مخرب** است. از آنجا که این کلمه در 99 کلاس امتیاز 0 دارد، هنگام جمع و میانگین‌گیری، آن 1 امتیاز بالا در میان 99 امتیاز 0 «رقیق» (**Diluted**) می‌شود. در نتیجه، امتیاز سراسری کلمه بسیار پایین آمده و احتمالاً سیستم این کلمه تخصصی را حذف خواهد کرد. این روش بیشتر برای یافتن کلماتی مناسب است که در چندین کلاس مختلف توزیع شده و تا حدودی به تفکیک آن‌ها کمک می‌کنند.

بخش 4) استراتژی سوم (معماری جایگزین): رویکرد اجتماع ویژگی‌ها (Union Approach)

به جای اینکه ابتدا یک امتیاز سراسری محاسبه کنیم و سپس کلمات را فیلتر کنیم، بسیاری از سیستم‌های عملیاتی استاندارد از یک معماری 2 مرحله‌ای استفاده می‌کنند:

1. **انتخاب محلی (Local Selection):** ابتدا برای هر کلاس به صورت کاملاً مستقل، **K** کلمه‌ای که بالاترین امتیاز **M_i** را با آن کلاس دارند انتخاب می‌کنیم. (مثلاً 50 کلمه برترِ مختص به هر کلاس). بدین ترتیب برای هر کلاس **c_i**، یک مجموعه کلمات ممتاز به نام **V_i** تشکیل می‌شود.

2. **تجمیع سراسری (Global Union):** دایره لغات نهایی سیستم (**V**) از اجتماع (**Union**) تمامی این مجموعه‌های محلی ساخته می‌شود:

$$V = \bigcup_{c \in C} V_c$$

مزیت این روش: این رویکرد (که از نظر ریاضی خروجی بسیار مشابهی با استراتژی **Maximum** دارد) تضمین می‌کند که هیچ کلاسی در سیستم نهایی بدون نماینده باقی نمی‌ماند. حتی کوچک‌ترین کلاس‌ها نیز مطمئن خواهند بود که بهترین و تخصصی‌ترین کلماتشان (مانند «پایتون» برای کلاس برنامه‌نویسی) با موفقیت به فضای ویژگی‌های نهایی سیستم راه پیدا کرده‌اند.

نکته:

در توسعه سیستم‌های واقعی با کلاس‌های متعدد و داده‌های نامتوازن، استفاده از استراتژی **Maximum** یا رویکرد اجتماع (**Union**) استانداردِ بلامنازع صنعت است. استفاده از میانگین‌گیری (**Average**) معمولاً باعث افت شدید معیار **Recall** در کلاس‌های اقلیت و تخصصی می‌شود.

پاسخنامه تشریحی مسئله 13.26: اثبات خطی بودن مرز تصمیم در نایو بیز

بخش 1) تعریف طبقه‌بند خطی (Linear Classifier)

از منظر هندسی و جبر خطی، یک طبقه‌بند خطی سیستمی است که فضای ویژگی‌ها را به وسیله یک ابرصفحه (Hyperplane) به 2 قسمت تقسیم می‌کند. تصمیم‌گیری در این مدل برای 2 کلاس بر اساس معادله زیر انجام می‌شود:

$$w^T x + b > 0$$

که در آن:

بردار x : بردار ویژگی‌های سند ورودی است.

بردار w : بردار وزن‌ها (Weights) است که اهمیت هر ویژگی را مشخص می‌کند.

متغیر b : مقدار انحراف یا بایاس (Bias) است.

اگر حاصل این معادله بزرگتر از 0 باشد، سند به کلاس 1 و در غیر این صورت به کلاس 2 تخصیص می‌یابد.

بخش 2) فرمول‌بندی قانون تصمیم در نایو بیز

فرض کنید یک مسئله دسته‌بندی دودویی (Binary Classification) با 2 کلاس c_1 و c_2 داریم. الگوریتم نایو بیز بر اساس قاعده پیشینه‌سازی، زمانی سند d را به کلاس c_1 اختصاص می‌دهد که احتمال تعلق آن به c_1 از c_2 بیشتر باشد:

$$\frac{P(c_1|d)}{P(c_2|d)} > 1$$

با بسط دادن این کسر از طریق قضیه بیز و حذف مخرج مشترک $P(d)$ ، به نامعادله زیر می‌رسیم:

$$\frac{P(c_1) \prod_{i=1}^{|V|} P(t_i|c_1)^{x_i}}{P(c_2) \prod_{i=1}^{|V|} P(t_i|c_2)^{x_i}} > 1$$

در این رابطه، $|V|$ اندازه دایره لغات سیستم و x_i نشان‌دهنده فراوانی (تعداد تکرار) کلمه i در سند d است. در واقع، سند d با بردار ویژگی $x = [x_1, x_2, \dots, x_{|V|}]^T$ نمایش داده می‌شود.

بخش 3) اعمال تبدیل لگاریتمی (Log-Odds Transformation)

از آنجا که تابع لگاریتم اکیداً صعودی است، گرفتن لگاریتم از 2 طرف نامعادله، جهت علامت بزرگتر/کوچکتر را تغییر نمی‌دهد. با اعمال لگاریتم (نسبت شانس یا Log-odds)، ضرب‌ها به جمع تبدیل می‌شوند:

$$\log\left(\frac{P(c_1)}{P(c_2)}\right) + \log\left(\prod_{i=1}^{|V|}\left(\frac{P(t_i|c_1)}{P(t_i|c_2)}\right)^{x_i}\right) > \log(1)$$

با ساده‌سازی فرمول و انتقال توان x_i به پشت لگاریتم (و با توجه به اینکه $\log(1) = 0$ است)، به معادله نهایی زیر می‌رسیم:

$$\sum_{i=1}^{|V|} x_i \log\left(\frac{P(t_i|c_1)}{P(t_i|c_2)}\right) + \log\left(\frac{P(c_1)}{P(c_2)}\right) > 0$$

بخش 4) نگاشت ریاضی: معادل‌سازی متغیرهای w و b

با نگاهی دقیق به معادله به دست آمده در گام 3، متوجه می‌شویم که این معادله دقیقاً همان ساختار $w^T x + b > 0$ را دارد. اکنون می‌توانیم متغیرهای مدل خطی را بر اساس احتمالات نایو بیز استخراج کنیم:

الف) مقدار انحراف (Bias):

متغیر b یک عدد ثابت است که به متن سند بستگی ندارد و صرفاً نشان‌دهنده سوگیری سیستم به سمت یکی از کلاس‌ها پیش از خواندن متن است.

$$b = \log\left(\frac{P(c_1)}{P(c_2)}\right)$$

ب) بردار وزن‌ها (Weights):

مؤلفه w_i (وزن مربوط به کلمه i ام در بردار w) به صورت زیر تعریف می‌شود:

$$w_i = \log\left(\frac{P(t_i|c_1)}{P(t_i|c_2)}\right)$$

تحلیل معنایی وزن w_i :

اگر کلمه i در کلاس c_1 بسیار پرتکرارتر از c_2 باشد، کسر بزرگتر از 1 شده و مقدار لگاریتم (وزن w_i) مثبت

می‌شود. حضور این کلمه در متن، کفه ترازو را به نفع کلاس c_1 سنگین می‌کند.

اگر کلمه i در کلاس c_2 بیشتر دیده شده باشد، کسر کوچکتر از 1 شده و مقدار لگاریتم منفی می‌شود. این یعنی

حضور کلمه، امتیازی به نفع کلاس c_2 است.

اگر کلمه در هر 2 کلاس به یک اندازه توزیع شده باشد (مانند کلمات توقف)، کسر برابر 1 و در نتیجه وزن آن 0

می‌شود (هیچ تاثیری در تصمیم‌گیری ندارد).

نتیجه‌گیری:

این اثبات ریاضی نشان می‌دهد که برخلاف تصور اولیه، الگوریتم نایو بیز در زمان تست (Testing Phase) نیازی به درگیر

شدن با مفاهیم پیچیده احتمالات ندارد. فاز آموزشِ نایو بیز صرفاً فرآیندی برای یافتن بهینه‌ترین مقادیر برای ماتریس وزن‌ها (w) و بایاس (b) است. پس از محاسبه این پارامترها، نایو بیز در محیط عملیاتی دقیقاً مانند یک ماشین ضرب نقطه‌ای (**Dot Product**) عمل می‌کند، که راز سرعت بی‌نظیر آن در دسته‌بندی متون نیز همین معماری خطی است.

پاسخنامه تشریحی مسئله 13.27: تحلیل خطای Underflow و بازنویسی لگاریتمی فرمول

بخش 1) کالبدشکافی بروز خطا در متون حجیم

در مدل نایو بیز چندجمله‌ای، احتمال تعلق یک سند d به کلاس c از ضرب احتمال پیشین در احتمالات شرطی تک‌تک کلمات به دست می‌آید:

$$P(d|c) \propto P(c) \prod_{i=1}^{L_d} P(t_i|c)$$

در متون حجیم مانند پرونده‌های حقوقی، طول سند (L_d) می‌تواند به ده‌ها هزار کلمه برسد. از آنجا که احتمال شرطی هر کلمه همواره عددی اعشاری و کوچکتر از 1 است ($0 < P(t|c) < 1$)، پردازنده کامپیوتر مجبور است ده‌ها هزار عدد بسیار کوچک را در یکدیگر ضرب کند.

حاصل این ضرب متوالی با سرعت نمایی به سمت 0 میل می‌کند (مثلاً عددی در مقیاس 10^{-5000}). از آنجا که استانداردهای ممیز شناور در معماری کامپیوتر (مانند **IEEE 754**) قادر به ذخیره‌سازی اعدادی تا این حد کوچک نیستند، پردازنده مقدار را گرد کرده و آن را **مطلقاً 0 (0.0)** در نظر می‌گیرد. به این فروپاشی محاسباتی، **خطای حد پایین (Floating-point Underflow)** گفته می‌شود که باعث می‌شود امتیاز تمامی کلاس‌ها 0 شده و سیستم قدرت تصمیم‌گیری را از دست بدهد.

بخش 2) راهکار ریاضی: انتقال از فضای ضرب به فضای جمع

برای غلبه بر این محدودیت سخت‌افزاری، باید از ویژگی‌های بنیادین تابع **لگاریتم (Logarithm)** استفاده کنیم. مهم‌ترین ویژگی لگاریتم این است که ضرب اعداد را به جمع تبدیل می‌کند:

$$\log(x \times y) = \log(x) + \log(y)$$

از آنجا که هدف الگوریتم نایو بیز یافتن کلاسی با بیشترین احتمال است (قاعده **arg max**)، ما در واقع به مقدار دقیق احتمالات نیازی نداریم، بلکه فقط می‌خواهیم رتبه‌بندی آن‌ها حفظ شود. چون تابع لگاریتم یک **تابع اکیداً صعودی (Monotonically Increasing Function)** است، اعمال آن بر روی خروجی‌ها تغییری در کلاس برنده ایجاد نمی‌کند.

بخش 3) بازنویسی و استخراج فرمول نهایی

قانون تصمیم‌گیری اولیه سیستم برای انتخاب بهترین کلاس (c_{map}) به این شکل است:

$$c_{map} = \arg \max_{c \in C} \left[P(c) \prod_{i=1}^{L_d} P(t_i|c) \right]$$

با اعمال تابع لگاریتم بر عبارت داخل براکت، فرمول را بسط می‌دهیم:

$$c_{map} = \arg \max_{c \in C} \left[\log \left(P(c) \prod_{i=1}^{L_d} P(t_i|c) \right) \right]$$

اکنون با استفاده از قوانین لگاریتم، ضرب بیرونی را به جمع، و عملگر ضرب متوالی (Π) را به عملگر جمع متوالی (Σ) تبدیل می‌کنیم:

$$c_{map} = \arg \max_{c \in C} \left[\log P(c) + \sum_{i=1}^{L_d} \log P(t_i|c) \right]$$

بخش 4) تحلیل پایداری فرمول جدید

در فرمول بازنویسی‌شده:

لگاریتم یک احتمال (عددی بین 0 و 1)، به یک عدد منفی معنادار و قابل مدیریت تبدیل می‌شود (مثلاً $\log_{10}(0.01) = -2$).

به جای ضرب کردن ده‌ها هزار کسر کوچک، پردازنده اکنون ده‌ها هزار عدد منفی (مانند -2، -3.5، -4.1) را با یکدیگر جمع جبری می‌کند.

حاصل این جمع یک عدد منفی بزرگ (مثلاً -15000) خواهد بود که کاملاً در محدوده امن پردازشگرها قرار دارد و خطای **Underflow** را به طور کامل و قطعی خنثی می‌سازد.

نکته:

فرمول لگاریتمی فوق، استاندارد بی‌چون‌وچرای پیاده‌سازی نایو بیز در تمامی کتابخانه‌های معتبر یادگیری ماشین (مانند **Scikit-learn**) است. این بازنویسی نه تنها خطای سرریز را حل می‌کند، بلکه به دلیل سرعت بسیار بالاتر عملگر «جمع» نسبت به عملگر «ضرب» در واحد منطق و حساب پردازنده (**ALU**)، سرعت فاز تست سیستم را نیز به شکل چشمگیری ارتقا می‌دهد.

پاسخنامه مسئله 13.28: معماری بروزرسانی برخط (Online) در مدل نایو بیز

بخش 1) چالش یادگیری دسته‌ای (Batch Learning)

در رویکرد سنتی، سیستم تمام داده‌ها را جمع‌آوری کرده و مدل را یکجا آموزش می‌دهد. اگر بخواهیم با هر پست جدید، مدل را روی تمام داده‌های قبلی به علاوه داده جدید بازآموزی (**Retrain**) کنیم، هزینه پردازشی به شدت بالا رفته و

سیستم در شبکه‌های اجتماعی فلج می‌شود.

خوشبختانه، الگوریتم نایو بیز به دلیل ساختار شمارشی خود، ذاتاً از یادگیری افزایشی (Incremental Learning) یا بروزرسانی برخط (Online Updating) پشتیبانی می‌کند. برای این کار، تنها کافی است معماری ذخیره‌سازی مدل را تغییر دهیم.

بخش 2) تغییر معماری: ذخیره «شمارنده‌ها» به جای «احتمالات»

راز بروزرسانی بلادرنگ نایو بیز این است که ما نباید احتمالات نهایی (اعداد اعشاری مانند $P(t|c)$) را در پایگاه داده ذخیره کنیم؛ زیرا با ورود هر پست جدید، مخرج کسرهای تغییر کرده و باید تمام احتمالات قبلی را دوباره محاسبه کنیم. در عوض، معماری سیستم باید صرفاً بر پایه حفظ و بروزرسانی متغیرهای حالت (State Counters) در یک پایگاه داده سریع (مانند جداول Hash در RAM یا سیستم‌هایی مثل Redis) بنا شود. این متغیرها عبارتند از:

N: کل تعداد پست‌های پردازش شده تاکنون.

N_c: تعداد پست‌های متعلق به هر کلاس **c**.

T_{ct}: تعداد دفعات تکرار کلمه **t** در کلاس **c**.

L_c: مجموع کل کلمات موجود در کلاس **c** (طول کلاس).

|V|: اندازه دایره لغات (تعداد کل کلمات یکتا در سیستم).

بخش 3) الگوریتم بروزرسانی بلادرنگ (مرحله ورود پست جدید)

فرض کنید یک پست جدید (**d**) با طول **L_d** کلمه وارد سیستم شده و کاربر برچسب صحیح آن را کلاس **c_{true}** تعیین کرده است. سیستم بدون نیاز به دیدن پست‌های قبلی، شمارنده‌ها را با هزینه زمانی بسیار ناچیز ($O(L_d)$) به این شکل آپدیت می‌کند:

1. کل اسناد را 1 واحد افزایش بده:

$$N \leftarrow N + 1$$

2. اسناد کلاس مربوطه را 1 واحد افزایش بده:

$$N_{c_{true}} \leftarrow N_{c_{true}} + 1$$

3. طول کل کلاس مربوطه را آپدیت کن:

$$L_{c_{true}} \leftarrow L_{c_{true}} + L_d$$

4. سپس سیستم روی تکتک کلمات (**t**) در پست جدید پیمایش می‌کند:

اگر کلمه **t** جدید است و تاکنون در سیستم نبوده، آن را به دایره لغات اضافه کرده و $|V| \leftarrow |V| + 1$.

$$T_{true,t} \leftarrow T_{true,t} + \text{freq}(t, d)$$

با انجام این چند عملگر جمع ساده، مدل به طور کامل آپدیت شده و پست جدید را آموخته است.

بخش 4) محاسبه احتمالات در زمان تست (On-the-fly Calculation)

حال اگر یک پست بدون برچسب برای طبقه‌بندی وارد سیستم شود، سیستم مقادیر احتمال را از پیش آماده ندارد، بلکه آن‌ها را در همان لحظه (**On-the-fly**) با استفاده از شمارنده‌های آپدیت‌شده و فرمول هموارسازی لاپلاس محاسبه می‌کند:

$$P(c) = \frac{N_c}{N}$$

$$P(t|c) = \frac{T_{ct} + 1}{L_c + |V|}$$

به این ترتیب، الگوریتم نایو بیز همواره از جدیدترین اطلاعات برای تصمیم‌گیری استفاده می‌کند.

نکته آموزشی:

این ویژگی معماری نایو بیز را به یکی از محبوب‌ترین انتخاب‌ها برای فیلترینگ اسپم در سرویس‌های ایمیل و پایش شبکه‌های اجتماعی تبدیل کرده است. سیستم به محض اینکه کاربر روی دکمه «گزارش اسپم» کلیک می‌کند، شمارنده‌ها را در کسری از میلی‌ثانیه تغییر می‌دهد و مدل بلافاصله و بدون هیچ هزینه سنگین محاسباتی، نسبت به آن کلمات هوشیار می‌شود (جلوگیری از فاجعه فراموشی یا **Catastrophic Forgetting**).

پاسخنامه مسئله 13.29: طراحی طبقه‌بند سلسله‌مراتبی و تاثیر آن بر فضای ویژگی‌ها

بخش 1) معماری مدل سلسله‌مراتبی (Hierarchical Design)

در 1 مدل مسطح (**Flat**)، سیستم دارای 1 طبقه‌بند واحد است که تلاش می‌کند سند ورودی را مستقیماً از میان 10ها یا 100ها کلاس نهایی انتخاب کند. اما در مدل سلسله‌مراتبی، ما معماری را به شکل 1 **درخت تصمیم احتمالاتی (Probabilistic Decision Tree)** طراحی می‌کنیم.

برای سناریوی مطرح شده، معماری سیستم شامل 2 طبقه‌بند (**Classifier**) کاملاً مجزا خواهد بود:

1. **طبقه‌بند سطح 0 (گره ریشه):** 1 نایو بیز دودویی که صرفاً وظیفه دارد اسناد را به 2 دسته کلان «علوم» و

«هنر» تقسیم کند.

2. طبقه‌بند سطح 1 (گره علوم): 1 نایو بیز دودویی دیگر که تنها اسنادی را پردازش می‌کند که از فیلتر گره ریشه عبور کرده و برچسب «علوم» گرفته‌اند. وظیفه این گره، تفکیک اسناد علمی به 2 زیرشاخه «فیزیک» و «ریاضی» است.

احتمال نهایی تعلق 1 سند (d) به کلاس «فیزیک» از طریق قاعده زنجیره‌ای احتمالات (Chain Rule) به دست می‌آید:

$$P(\text{Physics}|d) = P(\text{Physics}|\text{Science}, d) \times P(\text{Science}|d)$$

بخش 2) تحول بنیادین در انتخاب ویژگی (Feature Selection)

مهم‌ترین و درخشان‌ترین مزیت معماری سلسله‌مراتبی، تغییر رفتار سیستم در فاز استخراج و انتخاب ویژگی (با استفاده از معیارهایی مانند MI یا χ^2) است. در این معماری، ما به جای 1 «دایره لغات سراسری عظیم»، به «فضاهای ویژگی محلی» (Localized Feature Spaces) دسترسی پیدا می‌کنیم.

این تغییر معماری، 2 تاثیر شگرف بر روی فیلتر کردن کلمات می‌گذارد:

الف) گره ریشه: تمرکز بر ویژگی‌های کلان (Macro-features)

در سطح 1 (تقسیم علوم و هنر)، الگوریتم انتخاب ویژگی به دنبال کلماتی می‌گردد که بتوانند مرز بین 2 دنیای کاملاً متفاوت را رسم کنند.

رفتار معیارها: در این گره، کلماتی مانند «آزمایشگاه»، «پژوهش»، «دانشگاه» (برای علوم) و «رنگ»، «بوم»، «گالری» (برای هنر) بالاترین امتیاز اطلاعات متقابل (MI) را دریافت می‌کنند.

مزیت: سیستم درگیر جزئیات درون‌رشته‌ای نمی‌شود و فقط با استفاده از چند 100 کلمه کلان، با دقت و سرعت بسیار بالا، ترافیک ورودی را به 2 مسیر مجزا هدایت می‌کند.

ب) گره فرزند: ظهور ویژگی‌های خرد و تخصصی (Micro-features)

جادوی اصلی در گره 2 (تفکیک فیزیک و ریاضی) رخ می‌دهد. در این گره، مجموعه آموزشی (Training Set) فقط و فقط شامل مقالات علمی است و تمامی مقالات هنری از محاسبات حذف شده‌اند.

تغییر ارزش کلمات: کلمه‌ای مانند «فرمول» یا «پژوهش» که در گره ریشه 1 ویژگی طلایی بود، در این گره ارزش خود را کاملاً از دست می‌دهد! چرا که هم در مقالات ریاضی و هم در فیزیک به وفور یافت می‌شود (تبدیل به نویز یا کلمه توقف محلی می‌شود) و امتیاز χ^2 آن به 0 میل می‌کند.

نجات کلمات نادر: در عوض، کلماتی مانند «کوانتوم»، «ترمودینامیک»، «انتگرال» و «توپولوژی» که در 1 مدل مسطح (Flat) ممکن بود به دلیل تکرار کم در کل دادگان حذف شوند، در این فضای محلی ایزوله، قدرت تفکیک (Discriminative Power) 100 درصدی پیدا کرده و بالاترین امتیازها را کسب می‌کنند.

بخش 3) مزایای مهندسی سیستم‌های سلسله‌مراتبی

کاهش چشمگیر ابعاد (Dimensionality Reduction): هر طبقه‌بند محلی تنها به کلمات کلیدی مختص به

خود نیاز دارد. این یعنی پردازنده در هر مرحله با بردارهای بسیار کوچکتری سروکار دارد که خطر خطای

بیش‌برازش (Overfitting) و خطای Underflow را به حداقل می‌رساند.

ایزوله سازی خطا (Error Isolation): اگر سیستم در تشخیص «فیزیک» از «ریاضی» دچار چالش شود، این خطا هیچ تاثیری بر دقت دسته بندی مقالات «هنری» نخواهد گذاشت، زیرا مدل های ریاضیاتی آن ها کاملاً از یکدیگر تفکیک شده اند.

حل مشکل داده های نامتوازن: در 1 مدل مسطح، اگر مقالات هنری بسیار بیشتر از مقالات ریاضی باشند، سیستم دچار سوگیری می شود. در مدل سلسله مراتبی، تعادل داده ها در هر گره به صورت مستقل و محلی مدیریت می شود.

پاسخنامه مسئله 13.30: پیاده سازی اسکرپت مدل چندجمله ای (Multinomial Model Script Implementation)

```
multinomial_naive_bayes.py

import math
from collections import defaultdict, Counter

def train_multinomial_nb(docs, labels):
    """
    Trains a Multinomial Naive Bayes classifier with Laplace smoothing.
    Args:
        docs (list of list of str): Tokenized documents.
        labels (list of str): Class labels corresponding to the documents.
    Returns:
        tuple: (log_prior, log_cond_prob)
        - log_prior (dict): Logarithmic prior probabilities of each class.
        - log_cond_prob (dict): Logarithmic conditional probabilities of terms per class.
    """
    total_docs = len(labels)
    class_doc_counts = Counter(labels)
    # 1. Calculate Log Prior Probabilities
    log_prior = {}
    for c, count in class_doc_counts.items():
        # P(c) = N_c / N
        log_prior[c] = math.log(count / total_docs)
    # 2. Build Vocabulary and Term Frequencies
    term_counts_per_class = defaultdict(Counter)
    total_terms_per_class = defaultdict(int)
    vocab = set()
    for doc, label in zip(docs, labels):
        for token in doc:
            term_counts_per_class[label][token] += 1
            total_terms_per_class[label] += 1
            vocab.add(token)
    vocab_size = len(vocab)
    # 3. Calculate Log Conditional Probabilities with Laplace Smoothing
    log_cond_prob = defaultdict(dict)
    for c in class_doc_counts:
        # L_c + |V| (Denominator for Laplace smoothing)
        denominator = total_terms_per_class[c] + vocab_size
        for term in vocab:
            # T_ct + 1 (Numerator for Laplace smoothing)
            term_count = term_counts_per_class[c][term]
            prob = (term_count + 1) / denominator
            # Store in log space to prevent underflow during testing
            log_cond_prob[c][term] = math.log(prob)
    return log_prior, log_cond_prob
```

کالبدشکافی ساختار الگوریتم

بخش 1) استفاده از ساختارهای داده بهینه (Data Structures)

در این پیاده سازی از **Counter** و **defaultdict** از کتابخانه داخلی **collections** استفاده شده است. این ساختارها مبتنی بر جداول درهم سازی (**Hash Tables**) هستند و پیچیدگی زمانی شمارش کلمات و دسترسی به آن ها را به **O(1)** کاهش می دهند که برای پردازش مجموعه داده های عظیم و ساخت ایندکس بسیار حیاتی است.

بخش 2) مکانیزم هموارسازی لاپلاس (Laplace Smoothing)

خطوط پایانی کد مستقیماً فرمول ریاضی لاپلاس را پیاده‌سازی می‌کنند:

$$P(t|c) = \frac{T_{ct} + 1}{\sum_{t' \in V} T_{ct'} + |V|}$$

در این کد:

متغیر **term_count** معادل **T_{ct}** است. اگر کلمه‌ای در کلاس **c** وجود نداشته باشد، **Counter** به طور خودکار

مقدار 0 را برمی‌گرداند و با **+1** جمع می‌شود (جلوگیری از احتمال 0).

متغیر **vocab_size** معادل **|V|** (اندازه کل دایره لغات سیستم) است که به مخرج کسر (**denominator**) اضافه

شده تا مجموع احتمالات شرطی در هر کلاس دقیقاً برابر با 1 باقی بماند.

بخش 3) پیمایش روی کل دایره لغات (Vocabulary Iteration)

دقت کنید که در حلقه محاسبه احتمالات شرطی (**for term in vocab**)، سیستم برای هر کلاس، روی **کل دایره لغات**

پیمایش می‌کند، نه فقط کلمات موجود در آن کلاس. این 1 الزام قطعی در مدل نایو بیز است تا کلماتی که در کلاس‌های

دیگر دیده شده‌اند اما در کلاس فعلی غایب هستند نیز مقدار هموارشده لاپلاس را دریافت کنند.

پاسخنامه مسئله 13.31: پیاده‌سازی ارزیابگر خودکار (Autograder Implementation)

```
autograder.py

class Autograder:
    def __init__(self, y_true, y_pred):
        """
        Initializes the Autograder with true labels and predicted labels.
        Dynamically extracts all unique classes from both lists.
        """
        if len(y_true) != len(y_pred):
            raise ValueError("Error: Length of y_true and y_pred must be identical.")
        self.y_true = y_true
        self.y_pred = y_pred
        # Extract unique classes present in either true or predicted sets
        self.classes = set(self.y_true).union(set(self.y_pred))

    def _get_confusion_elements(self, target_class):
        """
        Calculates True Positives (TP), False Positives (FP),
        and False Negatives (FN) for a specific target class.
        """
        tp = sum(1 for yt, yp in zip(self.y_true, self.y_pred) if yt == target_class and yp == target_class)
        fp = sum(1 for yt, yp in zip(self.y_true, self.y_pred) if yt != target_class and yp == target_class)
        fn = sum(1 for yt, yp in zip(self.y_true, self.y_pred) if yt == target_class and yp != target_class)
        return tp, fp, fn

    def calculate_macro_f1(self):
        """
        Calculates the Macro-F1 score by computing F1 for each class
        independently and then taking the unweighted mean.
        """
        f1_scores = []
        for c in self.classes:
            tp, fp, fn = self._get_confusion_elements(c)
            # Safe division to prevent ZeroDivisionError
            precision = (tp / (tp + fp)) if (tp + fp) > 0 else 0.0
            recall = (tp / (tp + fn)) if (tp + fn) > 0 else 0.0
            if (precision + recall) > 0:
                class_f1 = 2 * (precision * recall) / (precision + recall)
            else:
                class_f1 = 0.0
            f1_scores.append(class_f1)
        # Return the arithmetic mean of all class F1 scores
        return sum(f1_scores) / len(f1_scores) if f1_scores else 0.0

    def calculate_micro_f1(self):
        """
```

```
Calculates the Micro-F1 score by globally aggregating all
TP, FP, and FN across all classes before calculating the metric.
"""

total_tp = 0
total_fp = 0
total_fn = 0
for c in self.classes:
    tp, fp, fn = self._get_confusion_elements(c)
    total_tp += tp
    total_fp += fp
    total_fn += fn
# Calculate global precision and recall
global_precision = (total_tp / (total_tp + total_fp)) if (total_tp + total_fp) > 0 else 0.0
global_recall = (total_tp / (total_tp + total_fn)) if (total_tp + total_fn) > 0 else 0.0
if (global_precision + global_recall) > 0:
    micro_f1 = 2 * (global_precision * global_recall) / (global_precision + global_recall)
else:
    micro_f1 = 0.0
return micro_f1
```

معماری و نکات فنی پیاده‌سازی

استخراج پویا (Dynamic Extraction): با استفاده از متد **set().union()** در تابع سازنده (**__init__**)، کلاس‌ها به صورت خودکار و بدون نیاز به دریافت پارامتر اضافی از خروجی‌های سیستم استخراج می‌شوند. این ویژگی باعث می‌شود اسکرپیت برای مسائلی با تعداد کلاس متغیر کاملاً منعطف باشد.

پایداری محاسباتی (Computational Stability): یکی از رایج‌ترین خطاهای دانشجویان در پیاده‌سازی سیستم‌های ارزیابی، خطای **ZeroDivisionError** در کلاس‌های اقلیت است. در این کد، با افزودن شرط‌های **if (tp + fp) > 0**، در صورتی که مخرج کسر 0 باشد، مقدار نهایی با ایمنی کامل 0 در نظر گرفته می‌شود.

طراحی ماژولار: متد **_get_confusion_elements** به صورت یک متد داخلی (**Internal Method**) طراحی شده است تا از تکرار کدهای مربوط به محاسبه المان‌های ماتریس درهم‌ریختگی در توابع میکرو و ماکرو جلوگیری کند و خوانایی کد را افزایش دهد.

پاسخنامه مسئله 13.32: مهندسی معکوس طبقه‌بند نایو بیز (جعبه سیاه)

بخش 1) تعریف ریاضی خروجی سیستم (امتیازدهی خام)

در معماری استاندارد نایو بیز چندجمله‌ای، سیستم برای جلوگیری از خطای **Underflow**، محاسبات را در فضای لگاریتمی انجام می‌دهد. فرض می‌کنیم خروجی قابل مشاهده سیستم برای هر کلاس (که آن را $S(c_j|d)$ می‌نامیم)، همان امتیاز لاگ-احتمال نرمال‌نشده است.

فرمول داخلی سیستم که برای ما پنهان است، به شکل زیر عمل می‌کند:

$$S(c_j|d) = \log P(c_j) + \sum_{k=1}^{|d|} \log P(w_k|c_j)$$

هدف ما این است که با کنترل ورودی (d)، بخش‌های مختلف این معادله را ایزوله (**Isolate**) کرده و مقادیر $P(c_j)$ و $P(w_i|c_j)$ را کشف کنیم.

بخش 2) فاز اول: استخراج احتمال پیشین کلاس‌ها $P(c_j)$

برای به دست آوردن احتمال پیشین، باید اثر احتمالات شرطی کلمات را به طور کامل از معادله حذف کنیم.

طراحی ورودی ساختگی:

1 سند کاملاً خالی و بدون هیچ کلمه‌ای (Empty Document) با نام $d_\emptyset$ ایجاد کرده و به عنوان ورودی به سیستم ارسال می‌کنیم.

تحلیل ریاضی:

چون سند فاقد کلمه است، بخش سیگما (جمع لگاریتم کلمات) در فرمول بالا برابر با 0 می‌شود. بنابراین خروجی سیستم دقیقاً برابر با لگاریتم احتمال پیشین خواهد بود:

$$S(c_j|d_\emptyset) = \log P(c_j) + 0$$

برای استخراج مقدار واقعی احتمال پیشین، کافی است از تابع نمایی (Exponential) روی خروجی مشاهده‌شده استفاده کنیم:

$$P(c_j) = \exp(S(c_j|d_\emptyset))$$

بخش 3) فاز دوم: استخراج احتمال شرطی کلمات $P(w_i|c_j)$

اکنون که لگاریتم احتمال پیشین $(\log P(c_j))$ را در اختیار داریم، می‌توانیم مقدار احتمال شرطی هر کلمه دلخواه مانند w_i را استخراج کنیم.

طراحی ورودی ساختگی:

1 سند جدید با نام d_{w_i} می‌سازیم که شامل تنها 1 بار تکرار از کلمه هدف (w_i) باشد و هیچ کلمه دیگری نداشته باشد. این سند را به طبقه‌بند ارسال می‌کنیم.

تحلیل ریاضی:

خروجی سیستم برای این سند تک‌کلمه‌ای به شکل زیر محاسبه می‌شود:

$$S(c_j|d_{w_i}) = \log P(c_j) + \log P(w_i|c_j)$$

با 1 جابه‌جایی ساده جبری و استفاده از مقداری که در فاز اول کشف کردیم، لگاریتم احتمال شرطی کلمه به دست می‌آید:

$$\log P(w_i|c_j) = S(c_j|d_{w_i}) - S(c_j|d_\emptyset)$$

و در نهایت، مقدار واقعی احتمال شرطی با اعمال تابع نمایی استخراج می‌شود:

$$P(w_i|c_j) = \exp(S(c_j|d_{w_i}) - S(c_j|d_{\emptyset}))$$

بخش 4) چالش سیستم‌های نرمال‌ساز (Softmax)

روابط فوق با این فرض نوشته شده‌اند که خروجی سیستم، لاگ-احتمال خام باشد. اگر سیستم جعبه سیاه خروجی‌ها را به صورت احتمالات نرمال‌شده بین 0 و 1 (مثلاً با استفاده از تابع **Softmax**) ارائه دهد، ما خروجی $P(c_j|d)$ را مشاهده خواهیم کرد که تحت تاثیر مخرج مشترک (احتمال کل سند یا $P(d)$) قرار گرفته است.

در این حالت، مهندسی معکوسِ مقادیرِ مطلقِ احتمالات غیرممکن می‌شود، اما همچنان می‌توانیم نسبت احتمالات (**Likelihood Ratios**) را استخراج کنیم. اگر اسناد ساختگی را ارسال کنیم، می‌توانیم نسبت وزن 1 کلمه در کلاس اول به کلاس دوم را با تقسیم کردن خروجی‌های سیستم به دست آوریم:

$$\frac{P(w_i|c_1)}{P(w_i|c_2)} \propto \frac{P(c_1|d_{w_i})}{P(c_2|d_{w_i})} \times \frac{P(c_2|d_{\emptyset})}{P(c_1|d_{\emptyset})}$$

جمع‌بندی تحلیلی:

این فرآیند اثبات می‌کند که سیستم‌های مبتنی بر نایو بیز از نظر امنیتی به شدت شفاف (**Transparent**) هستند. با تعداد پرس‌وجوهای خطی (برابر با اندازه دایره لغات سیستم)، 1 مهاجم یا تحلیل‌گر می‌تواند پایگاه داده داخلی سیستم را دقیقاً کپی کرده و مدل را به طور کامل شبیه‌سازی کند.